

A Comprehensive Survey on the Integration of Reinforcement Learning and NLP for Stock Market Trading

Brian J. Ferrell^{1,2,*}, Bridget T. McInnes¹

¹Virginia Commonwealth University, Richmond, VA, United States

²Quantitative Supervision and Research, Federal Reserve Bank of Richmond

Abstract

The integration of Reinforcement Learning (RL) and Natural Language Processing (NLP) has emerged as a creative approach in stock market trading, enabling agents to leverage both numerical data and textual information for informed financial decision-making. Traditional strategies often overlook the contextual insights available in financial news, social media, and corporate disclosures. By combining RL's iterative learning capabilities with NLP's ability to process unstructured text, modern trading systems can potentially optimize for higher profitability and risk management. This survey synthesizes findings from 22 research papers published between 2018 and 2024, analyzing methodologies, data sources, evaluation metrics, and other relevant aspects within an integrated RL–NLP framework for stock trading. A detailed review of foundational RL and NLP concepts is provided to contextualize these advancements. Challenges such as the lack of standardized datasets, limited use of random seeds for reproducibility, the simplicity of state representations and NLP methods, insufficient evaluation procedures, and limited real-world adaptability are explored, alongside opportunities for future research, including leveraging large language models and advancing textually-aware RL systems. This study aims to offer a comprehensive resource for researchers and practitioners to advance the field of RL and NLP in financial decision-making.

Keywords

Literature review, Reinforcement Learning, Natural Language Processing, Stock Market Trading, Financial Decision-Making

1. Introduction

The integration of Reinforcement Learning (RL) and Natural Language Processing (NLP) represents a transformative approach in stock market trading, where decision-making is informed by both structured numerical data and unstructured textual information. Traditional trading strategies often rely solely on historical price trends and technical indicators, which may miss the critical contextual insights embedded in financial news, social media, and corporate disclosures [1]. By harnessing the capabilities of RL and NLP, researchers aim to develop trading systems that not only react to market dynamics but also anticipate them by decoding the nuanced, time-sensitive signals within textual data.

The increasing availability of large amounts of financial data [2], coupled with advancements in computational power [3], has spurred significant attention in this field. As a goal-oriented framework, RL iteratively refines decision-making policies, making it an appealing option for complex tasks like stock market trading. Simultaneously, NLP techniques such as sentiment analysis, knowledge graph extraction, and transformer-based language models have unlocked new opportunities to quantify market sentiment and detect subtle correlations between textual events and market behaviors [4, 5]. The combination of these paradigms enables trading agents to optimize policies that incorporate both price data and contextual sentiment, aiming for improved profitability, risk management, and adaptability to market volatility.

Despite the promise of RL-NLP integration, challenges remain. The non-stationarity of financial markets [6], the inherent noise in textual data [7], and the high computational demands of these models [8] create significant barriers to real-world adoption and generalization. Furthermore, the heterogeneity of RL methodologies, from actor-critic frameworks to policy-based algorithms, complicates standardization and benchmarking [9]. Likewise, NLP techniques vary widely, from simple lexicon-based sentiment scoring to sophisticated transformer architectures, each introducing unique trade-offs in complexity and interpretability.

*Corresponding author.

[†]The views expressed in this paper are those of the authors and do not necessarily reflect the views of the Federal Reserve Bank of Richmond or the Federal Reserve System.

[‡]Data sharing is not applicable to this article as no new data were created or analyzed in this study. None of the authors have a conflict of interest to disclose. No funding was received for this research.

✉ ferrellbj@vcu.edu (B.J. Ferrell)

🌐 <https://github.com/brianferrell787> (B.J. Ferrell)

📄 0000-0003-2301-4926 (B.J. Ferrell)

This survey aims to discuss these challenges by providing a comprehensive synthesis of recent research at the intersection of RL, NLP, and stock market trading. We analyze 22 papers spanning 2018 to 2024, identifying common themes, methodologies, and gaps in the literature. Emphasis is placed on understanding how RL and NLP are synergistically employed to tackle core trading tasks such as portfolio optimization, single-asset trading, and market prediction. By systematically evaluating the state of the field, we offer insights into existing limitations and propose directions for future research, with the ultimate goal of advancing RL-NLP systems in financial decision-making.

The remainder of this paper is organized as follows. Section 2 presents an extensive review of related survey papers, offering a comparative perspective on existing literature and establishing a foundation for this study. Section 3 delves into the foundational concepts of RL, covering Markov Decision Processes (MDPs), classifications of RL methods, practical tips, as well as the history and core concepts of NLP, including the transformer revolution and advancements in the field. This section provides an in-depth exploration of theoretical underpinnings and methodologies that readers familiar with the basics may opt to skip. Sections 4 and 5 analyze the surveyed articles, evaluating the integration of RL and NLP in stock trading, presenting comprehensive insights into applications, baseline methods, data sources, preprocessing techniques, and evaluation metrics, supported by detailed tables. Section 6 identifies challenges and limitations as well as explores future directions, highlighting opportunities to advance the field, and Section 7 concludes this work. This structure provides a holistic yet detailed perspective on RL-NLP integration in stock market trading, catering to both seasoned experts and those new to the domain.

2. Related Survey Papers

The application of RL in financial markets has been extensively studied in recent years, with survey papers spanning from 2018 to 2023 offering a comprehensive examination of methodologies, applications, and trends in the field. These surveys provide critical insights into RL's contributions to tasks such as algorithmic trading, portfolio management, market making, and high-frequency trading (HFT). Furthermore, the integration of NLP into RL frameworks has emerged as a promising avenue for leveraging unstructured data sources, such as news and social media signals, to enhance trading strategies.

While our survey focuses on the intersection of RL and NLP in financial markets, understanding the broader landscape of RL applications in finance is essential for identifying gaps, trends, and opportunities. This section reviews ten key survey papers published between 2018 and 2023, highlighting their major findings, research trends, limitations, and future directions. Specifically, we examine:

- **Key Findings:** Major insights from each survey regarding RL's role in financial trading.
- **Research Trends:** Emerging trends such as the adoption of Deep Reinforcement Learning (DRL) methods and their applications.
- **Gaps and Limitations:** Areas requiring further exploration, such as data scarcity, evaluation protocols, and scalability.
- **Future Directions:** Promising research avenues proposed in the surveys.

By contextualizing these surveys, we aim to establish a foundation for understanding how RL and NLP can jointly address challenges while operating in financial markets.

2.1. Key Surveys on RL in Financial Markets

This subsection reviews the primary objectives, findings, and contributions of each survey paper, incorporating the number of papers reviewed and their scope.

2.1.1. Reinforcement Learning Applied to Trading Systems: A Survey [4]

Kanashiro et al. (2022) reviews 29 papers, classifying RL applications in financial trading and proposing a workflow pipeline for categorization. It emphasizes aligning problem formulations with foundational RL principles, such as the Markov Decision Process (MDP), while highlighting gaps in theoretical clarity, particularly in differentiating contextual bandits from full RL models. Technical indicators and deep learning encoders are noted for preprocessing state representations, but limited exploration of action and reward designs remains a critical challenge.

Key trends include the growing use of actor-critic methods and heuristic techniques to enhance RL, alongside strategies to address the exploration-exploitation trade-off. However, the lack of standardized datasets,

evaluation benchmarks, and synthetic time-series data for controlled experiments hinders comparability and reproducibility.

Future directions include refining problem formulations, comparative studies between contextual bandits and RL models, and enhanced state representations separating market- and agent-state features. Integrating NLP into RL, exploring Partially Observable Markov Decision Processes (POMDPs), and promoting open-source practices are recommended to improve reproducibility and methodological rigor.

2.1.2. Reinforcement Learning for Quantitative Trading [10]

Sun et al. (2023) reviews over 100 papers, offering a taxonomy of RL applications in quantitative trading, including algorithmic trading (AT), portfolio management (PM), order execution (OE), and market making. It highlights the adaptability of RL to market dynamics and its ability to optimize long-term profits while minimizing costs and market impacts. Traditional methods like Q-learning and recurrent RL are contrasted with advanced DRL techniques, including Deep Q-Networks (DQN), Deep Deterministic Policy Gradient (DDPG), and Proximal Policy Optimization (PPO).

Key trends include the shift toward DRL for handling high-dimensional trading environments, with AT focusing on market adaptability, PM on efficient asset allocation, and OE on minimizing trade costs and impacts. Emerging methodologies include model-based RL to address data scarcity, multi-objective RL for balancing profit and risk, and graph learning for modeling stock interrelationships. Techniques such as meta-RL, transfer learning, hierarchical RL, and offline RL are gaining traction, alongside the use of alternative data sources and underexplored areas like intraday and high-frequency trading.

Gaps include the lack of realistic market simulations, standardized datasets, and robust evaluation protocols, which limit reliability and comparability. Data scarcity and advanced RL integration remain underexplored challenges.

Future directions emphasize standardized evaluation metrics encompassing profitability, risk, and explainability, systematic hyperparameter tuning, and leveraging AutoML techniques to streamline RL model development and improve accessibility.

2.1.3. How Are Reinforcement Learning and Deep Learning Algorithms Used for Big Data-Based Decision Making in Financial Industries? [11]

Singh et al. (2022) reviews 64 papers, examining the integration of Deep Learning (DL) and RL in financial decision-making. It highlights DRL's scalability and ability to handle high-dimensional financial datasets, with methods like Deep Deterministic Policy Gradient (DDPG) and Deep Q-Networks (DQN) outperforming traditional approaches in tasks such as trading and news recommendation. DRL's advanced feature extraction and problem-solving capabilities enable more effective decision-making in complex financial scenarios.

Key trends include combining DL and RL to address scalability and high-dimensional challenges, with Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) models widely applied in prediction and strategy optimization. The survey emphasizes developing robust, interpretable models that balance complexity, accuracy, and profitability for real-world applications.

Gaps include the limited use of RL in multi-agent financial markets and the underexplored application of DRL in real-world trading environments, such as investment banks and hedge funds. Current multi-agent RL (MARL) models are insufficient for large-scale financial scenarios.

Future directions focus on advancing MARL to model agent interactions in financial markets, extending DRL applications to practical trading environments, and integrating DRL with big data and AI platforms to enhance scalability and adaptability for tasks like investment research, risk management, and high-frequency trading.

2.1.4. Automated Trading Systems: Statistical and Machine Learning Methods and Hardware Implementation [3]

Huang et al. (2019) reviews 60–80 papers on automated trading systems, categorizing them into technical analyses, textual analyses, and high-frequency trading (HFT). It highlights advancements in telecommunications and computing technologies that have diversified and enhanced these systems. Key findings include the evolution of technical analysis from indicator-based to machine learning-driven methods, the growing role of textual analysis in incorporating financial news, and the prominence of HFT in modern trading despite challenges like market fairness and systemic risks.

Trends include increasing adoption of machine learning, real-time data processing for HFT, and integrating textual and sentiment analysis into trading strategies. However, gaps remain in understanding the long-term effectiveness and market impact of these systems, ethical and regulatory challenges in HFT, and the integration of textual analysis with traditional financial indicators to improve predictive accuracy.

Future research directions emphasize integrating diverse data sources, such as financial news, social media sentiment, and market data, into trading systems to enhance decision-making. Proposals include developing Enterprise Information Systems (EIS) for unified data management, advancing real-time data processing capabilities with secure hardware, and refining data analysis techniques. Ethical and regulatory considerations in HFT are also highlighted as critical areas for ensuring fairness and market stability.

2.1.5. Applications of Deep Learning in Stock Market Prediction: Recent Progress [12]

Jiang (2021) reviews 124 papers on deep learning (DL) for stock market prediction, providing a workflow for categorizing prediction models and emphasizing implementation and reproducibility. It highlights the adoption of multi-source data and advanced DL techniques, offering insights into the predictive process from data collection to model evaluation.

Key findings include advancements in leveraging DL for stock price prediction, with improved performance through Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and integration of diverse data sources like financial news and social media sentiment. The emphasis on implementation and reproducibility addresses critical gaps in prior research.

Trends reveal growing sophistication in predictive models and increasing use of multi-source data. However, gaps remain, including limited exploration of novel architectures (e.g., Transformers), insufficient cross-market analysis, and lack of attention to practical trading aspects like transaction costs and slippage.

Future research directions include exploring Transformer-based models for financial text, combining diverse data sources for robust predictions, and leveraging transfer learning for cross-market analysis. Translating predictions into practical trading strategies using deep reinforcement learning (DRL) is also recommended, alongside expanding research to other financial markets and alternative methods.

2.1.6. A Review on Sentiment Analysis in Reinforcement Learning Models for Stock Market Analysis [13]

Soon et al. (2022) reviews 23 papers on integrating sentiment analysis into RL for financial trading. It highlights the growing use of advanced NLP techniques, such as Long Short-Term Memory (LSTM) networks and transformer models, to extract insights from financial news and social media. RL methods like Q-learning and Deep Q-Networks (DQN) dominate current applications, with Twin Delayed Deep Deterministic Policy Gradient (TD3) identified as a promising algorithm for future sentiment-based trading systems.

Trends indicate a shift from rule-based and machine learning methods to RL-based approaches, driven by their adaptability and integration of sentiment analysis. However, gaps persist, including narrow focus on single stocks, lack of cross-market sentiment analysis, and limited use of diverse or multilingual data sources.

Future research directions include expanding datasets to cover broader markets and timeframes, leveraging pretrained transformers like BERT for improved sentiment analysis, and addressing the exploration-exploitation trade-off in RL. Advanced RL techniques, such as hierarchical RL and meta-RL, are proposed to develop nuanced trading strategies.

2.1.7. Deep Reinforcement Learning for Trading—A Critical Survey [9]

Millea (2021) reviews 152 papers on DRL in trading, with a focus on cryptocurrency markets. It highlights the importance of state representation for DRL agents and addresses challenges in performance quantification and generalization across market conditions. Common practices include using Convolutional Neural Networks (CNNs) for data preprocessing, recent price steps as state representations, and the Sharpe ratio for performance evaluation. Most studies focus on portfolio management, with limited integration of critical cost factors like transaction costs, slippage, and spread.

Trends reveal significant variations in data usage, preprocessing techniques, and a narrow focus on single-market analyses, limiting cross-market generalization. The lack of publicly available code further hinders replication and development.

Gaps include inconsistent datasets, sampling frequencies, and preprocessing methods, making it difficult to attribute performance gains to algorithmic advancements. Methodological inconsistencies, such as varying assumptions about trading constraints (e.g., short-selling), reduce comparability across studies.

Future directions emphasize systematic hyperparameter tuning, robust testing of DRL models, and integration of practical considerations like cost factors and market dynamics. Standardized data usage, preprocessing methods, and reporting practices are recommended to enhance reproducibility. The survey advocates for comprehensive tools and methodologies, such as automated systems for trend analysis and more uniform evaluation frameworks.

2.1.8. An Overview of Machine Learning, Deep Learning, and Reinforcement Learning-Based Techniques in Quantitative Finance [14]

Sahu et al. (2023) reviews over 70 papers on AI applications in quantitative finance, focusing on RL, DRL, and DL methodologies. Key findings include the dominance of traditional DRL algorithms in quantitative trading (QT) and the potential of model-based DRL to address data scarcity by simulating financial markets. Multi-objective DRL methods are highlighted for balancing profit and risk, while areas like high-frequency and pairs trading remain underexplored. High-fidelity market simulations, incorporating factors like market impact and transaction costs, are identified as critical but challenging to achieve.

Trends emphasize model-based DRL for data efficiency, multi-objective DRL for diversified trading policies, and Auto-ML for streamlining DRL model development. The need for realistic market simulators, live-trading platform testing, and understanding DRL behavior in volatile markets are emerging priorities.

Gaps include limited exploration of specific QT domains, challenges in developing high-fidelity market simulations, and a lack of comprehensive real-time testing. Comparisons between DRL strategies and human traders in live markets are notably scarce.

Future directions propose enhancing market simulation methodologies, conducting more live-trading tests, and leveraging Auto-ML and advanced DRL techniques to improve robustness and profitability. Addressing dependencies on historical data and integrating external factors, such as political and economic events, are also recommended to advance the field.

2.1.9. Reinforcement Learning in Financial Markets: A Survey [15]

Fischer (2018) reviews 50 papers, categorizing RL approaches into critic-only, actor-only, and actor-critic frameworks. It highlights the context-dependent effectiveness of these strategies, with actor-only approaches showing promise due to faster convergence and suitability for continuous action spaces, particularly when enhanced with deep learning.

Key trends include integrating deep learning with RL, especially in actor-only and actor-critic methods, to improve feature extraction and decision-making. RL applications in high-frequency trading, such as trade execution optimization, represent a growing focus.

Gaps include limited exploration of actor-critic methods, lack of empirical validation across diverse markets, and insufficient attention to risk management in RL applications. The absence of benchmarking studies comparing RL approaches hinders generalizability, while challenges like noisy financial data and overfitting remain prevalent.

Future research directions propose large-scale benchmarking of RL methods across varied market conditions, advanced techniques for noise reduction and regularization, and exploring actor-critic frameworks for multi-asset portfolio management. Integrating alternative data sources, such as sentiment and macroeconomic indicators, alongside advanced feature extraction techniques, is recommended to enhance RL models further.

2.1.10. Recent Advances in Reinforcement Learning in Finance [2]

Hambly et al. (2023) reviews 100-200 papers, synthesizing a broad range of sources to evaluate RL's potential in improving financial decision-making. It highlights RL's versatility across applications such as robo-advising, where RL adapts to investors' dynamic risk preferences, and smart order routing, optimizing transactions across trading venues. The importance of incorporating risk-sensitive objectives into RL models and integrating domain knowledge from stochastic control literature is emphasized. Key directions include enhancing sample efficiency, leveraging transfer learning for new assets, and exploring multi-objective optimization.

Trends include the growing use of RL in complex financial tasks, the development of risk-sensitive algorithms, and the rise of offline learning for constrained scenarios. Efficient exploration strategies and model-free RL tailored to dynamic environments are also significant trends.

Gaps include underexplored risk-sensitive RL models, challenges with sample efficiency in nonstationary environments, and limited solutions for the cold-start problem with new assets. Overfitting, model mis-specification, and underutilization of financial domain knowledge further limit RL's applicability.

Future research directions focus on improving risk awareness, adapting RL to nonstationary data, and integrating financial modeling with RL to create robust frameworks. Promising areas include model-free robo-advising, multi-period RL for order routing, and transfer learning for asset trading. Developing realistic market simulators is highlighted as critical for training RL models in environments reflecting real-world dynamics.

2.2. Synthesis and Conclusion

The reviewed surveys highlight key themes in the application of RL and DRL to financial tasks, including portfolio management, algorithmic trading, and market making. A significant trend is the integration of deep learning for preprocessing financial data and feature extraction. Balancing profitability with risk management emerges as a core focus, driving interest in multi-objective and risk-sensitive RL models. Methods like actor-critic and model-based RL are increasingly adopted to address data scarcity, sample efficiency, and scalability challenges.

Common gaps include the lack of standardized datasets, insufficient exploration of multi-agent scenarios, limited market realism in simulations, and underdeveloped ethical and regulatory frameworks. Future research suggests leveraging alternative data sources (e.g., text), improving generalization via transfer learning and meta-RL, and developing robust, interpretable, and reproducible RL frameworks. Building on these insights, this survey focuses on the intersection of RL and NLP, exploring how textual analysis, sentiment extraction, and NLP-driven features can enhance RL models for financial decision-making.

3. Foundational Concepts

3.1. Introduction to Reinforcement Learning

Learning through interaction is a fundamental aspect of human experience. From mastering the delicate balance required to ride a bicycle, to adapting to the dynamics of a new workplace, or devising strategies in competitive games, our growth is driven by navigating environments through trial, error, and adaptation. Similarly, in machine learning, RL adopts this iterative process within a computational framework. RL involves an entity, called an agent, that learns through direct interaction with its environment. The agent's goal is to take actions that maximize cumulative rewards over time [16]. Figure 1 illustrates this RL framework, where the agent perceives the environment's state, chooses actions, and receives feedback in the form of rewards.

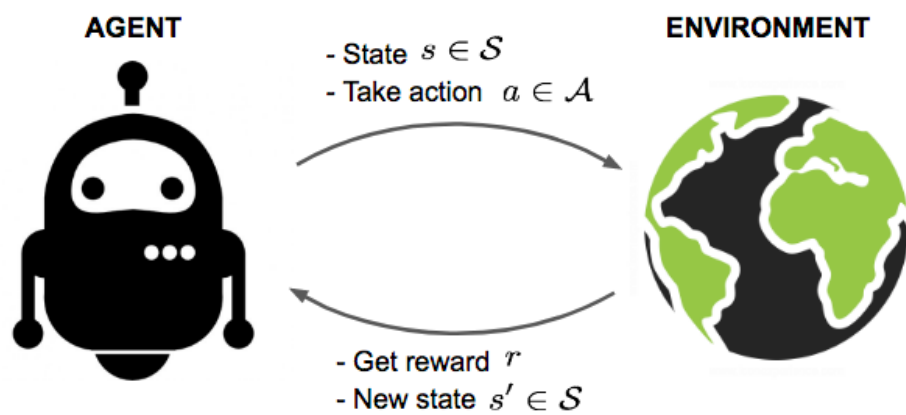


Figure 1: Reinforcement Learning Agent-Environment Interaction (adapted from [17]).

The key components of this RL paradigm include:

- **Agent:** The learner or decision-maker.
- **Environment:** The world the agent interacts with.
- **State:** The environment's current condition, as perceived by the agent.
- **Action:** The agent's response to the state.
- **Reward:** Feedback the agent receives based on its actions.
- **Policy:** The strategy guiding the agent's actions.
- **Exploration vs. Exploitation:** The balance between trying out new actions to discover better strategies and using actions that have already been proven to work.

RL in Action: From Games to Financial Markets

In a game like Tetris, an agent learns to rotate and position falling pieces to maximize cleared lines while avoiding filling up the screen. The reward structure is straightforward: higher scores for clearing more lines. In Mario Kart, an agent's objective could be to complete laps in the shortest amount of time or to finish the race in first place (these depend on what you want to accomplish). The agent must balance speed and control while strategically using power-up items to gain a competitive edge. In the case of this paper, for stock market trading, an agent may learn to execute buy, sell, or hold actions with the goal of maximizing financial returns or managing risk exposure. Here, the reward could be quantified through profit or another risk-adjusted performance metric [18, 19, 20]. For each of these environments the agent learns by experimenting with different actions and observing the outcomes. Over time, through trial and error, it identifies the patterns and strategies that yield the highest returns.

The design of RL systems also allows for customization in optimizing specific objectives. For instance, while one might configure an agent in Mario Kart to minimize average lap times, another goal could be to maximize the acquisition of Item Boxes for a strategic advantage. This level of design flexibility necessitates a creative approach, which must be wisely and methodically balanced with clear-cut objectives. When applied to stock trading, optimizing for a certain metric, such as win percentage, might lead to an agent developing suboptimal trading strategies that do not necessarily translate to maximal financial performance. Therefore, it is imperative to align the RL objectives with meaningful and impactful outcomes, as the agent will inevitably learn to achieve precisely what it has been optimized to accomplish.

Challenges of Real-World Environments

The complexity of an environment can significantly influence the learning process of an RL agent. While certain games offer predictable patterns and limited/finite scenarios [16], allowing the agent to eventually master the environment through sufficient interaction, real-world applications like stock market trading present a more formidable challenge. The stochastic nature of financial markets, coupled with the vast array of influencing factors [21, 22], renders the environment difficult to predict. Training an RL agent in such a setting is not straightforward; an agent might excel in a simulated market based solely on historical price data, yet fail to devise a successful strategy in the real world. This discrepancy can often be due to an oversimplification of the training environment, omitting critical variables that affect market dynamics.

Moreover, real-world applications raise additional concerns regarding the representativeness of simulation environments. For instance, while an Atari game can be directly replicated in a computer program, accurately simulating a stock market is a difficult task due to the complex, dynamic interplay of global economic factors, investor behaviors/market impacts, and regulatory changes [23]. Consequently, an RL agent trained in such a simplified model may not develop a policy that generalizes effectively to the unpredictable nature of real-world trading.

Furthermore, the deployment of a trained agent into a real-world trading system necessitates the consideration of operational intricacies that extend beyond the agent's decision-making logic (which is not something typically addressed in academic research). This includes the integration with real-time data streams for market prices, news scraping for analysis, interfacing with trading APIs, and managing the potential market impact of trade execution. The transition from a simulated training setting to a production environment involves not only algorithmic adjustments but also a comprehensive adaptation to the demanding infrastructural and computational requirements of live financial trading.

The preceding discussion has established the foundational principles of RL with a practical overview. In the sections that follow, we will delve deeper into the core concepts and algorithms that underpin RL, such as Markov Decision Processes, policy optimization, and various learning paradigms, culminating with practical insights for applying RL.

3.2. Reinforcement Learning Components

This section delves into the foundational concepts of Reinforcement Learning, focusing on the dynamics and mathematical frameworks that govern decision-making processes in this field. It aims to provide a comprehensive overview of Markov Decision Processes (MDPs), explore various algorithms pivotal to modern RL that are relevant to this survey, and offer practical insights for addressing RL challenges.

Note to Readers: This subsection is designed for those new to, or seeking a refresher on, the foundational elements of Reinforcement Learning. If you are already well-versed in these principles, you may choose to skip ahead to subsequent sections that discuss the recent research developments and their implications for the topics covered in this survey (i.e. see Section 4).

3.2.1. Markov Decision Processes (MDPs)

Markov Decision Processes (MDPs) serve as the cornerstone mathematical framework for understanding and designing RL systems. These frameworks offer a theoretical foundation for modeling decision-making in uncertain environments. While the equations and concepts presented in this section are essential for grasping RL's underpinnings, they are often simplified abstractions that may not directly translate to practical applications. Real-world RL systems typically adapt or approximate these concepts to address the complexities of partial observability, computational constraints, and dynamic environments.

In probability theory and statistics, there is what is called the Markov property, which states that the future depends only on the present, not the past. For example, in a game like chess, the next move depends on the current board configuration, not the entire sequence of moves leading to it. In other words, a stochastic process is said to have the Markov property if the conditional probability distribution of future states of the process depends only upon the present state, not on the sequence of events that preceded it. Formally, this property can be expressed as:

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, \dots, S_t] \quad (1)$$

This equation, known as the Markov property, simplifies the analysis and computation of decision processes by asserting that the probability of transitioning to the next state S_{t+1} is solely a function of the current state S_t and not of the sequence of states that led to it.

The Agent-Environment Interaction

In the context of RL, this property ensures that the decision-making process at any given step can be modeled as a function of the current state without the need to account for the entire history of an agent's interaction with the environment, as seen in Figure 2:

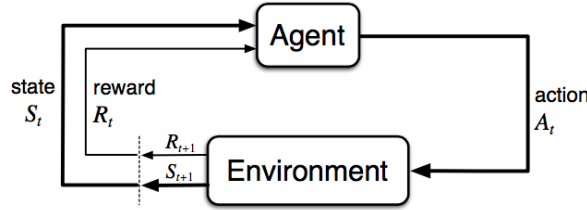


Figure 2: The agent-environment interaction in a Markov decision process (adapted from [16]).

An MDP is characterized by a set of states S , a set of actions A , and a transition probability function P , which together describe the environment in which the agent operates. Additionally, the MDP framework includes a reward function R , which assigns a numerical value to each transition between states, and a discount factor γ , which weighs the importance of future rewards.

As part of the MDP framework, at each time step t , the agent receives some representation of the environment's state s , and takes action a . Which then finds itself in a new state $S_{t+1} = s'$, as well as receiving some numerical reward $R_{t+1} = r$. The transition probability function P captures the dynamics of the environment. This function, given a state-action pair (s, a) , defines the probability of transitioning to each possible next state s' after taking action a while obtaining reward r . For instance, in a weather prediction model, if the current state is 'cloudy' and the action is 'predict rain,' the transition probability reflects how likely it is to move to a 'rainy' state, given the action. This is denoted by $P(s', r|s, a)$ and can be calculated as:

$$P(s', r|s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r|S_t = s, A_t = a] \quad (2)$$

Furthermore, the MDP framework includes a reward function. The expected reward for a state-action pair (s, a) is given by:

$$R(s, a) = \mathbb{E}[R_{t+1}|S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} P(s', r|s, a) \quad (3)$$

This equation predicts the next reward when taking one action a in state s . To capture the expected rewards when transitioning from a state s to a state s' under action a , we use a more granular three-argument function:

$$r(s, a, s') = \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r \frac{P(s', r | s, a)}{P(s' | s, a)} \quad (4)$$

Returns and Episodes

So far, we've informally explored how agents interact with their environment, aiming to maximize their cumulative reward. To formalize this, we introduce the concept of return, denoted as G_t , which represents the total anticipated reward over a trajectory (a sequence of states, actions, and rewards). These future rewards can take different forms depending on whether the environment has identifiable episodes or a terminal state.

In continuing tasks, such as a robot balancing on a tightrope without a predefined endpoint, we employ a discounted infinite-horizon return. This measure aggregates all rewards acquired by the agent, but weighs them by their distance in the future through a discount factor $\gamma \in (0, 1)$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{t=0}^{\infty} \gamma^t R_{t+1} \quad (5)$$

The discount factor, denoted as γ , plays a pivotal role by determining the value of these future rewards. When an agent receives a reward t time steps in the future, its value is discounted by γ^t . When γ is less than 1, it ensures that the sum of discounted rewards converges to a finite value, provided the reward sequence remains bounded. If γ equals 0, the agent adopts a "myopic" strategy, prioritizing immediate rewards over long-term gains. However, this short-sighted approach may limit the agent's ability to maximize its overall return, as actions optimized for immediate rewards might neglect opportunities for larger cumulative gains in the future. Conversely, as γ approaches 1, the agent becomes more forward-thinking, considering future rewards more strongly in its decision-making process. In episodic tasks where $\gamma = 1$, the return is simply the sum of rewards obtained at each step within an episode. For simplicity, a unified mathematical representation that simplifies handling both episodic and continuous tasks can be written as:

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (6)$$

In practical terms, G_t can represent cumulative profit in a trading system or total game score in a video game environment. The discount factor γ helps balance short-term versus long-term rewards, much like prioritizing immediate profits versus long-term growth in investment strategies.

Value Functions, Policies, and Bellman Equations

Value functions estimate the expected return from being in a given state or taking a specific action in that state, serving as a critical tool for evaluating the desirability of various states and actions. Specifically, the state value function, $V_{\pi}(s)$, calculates the expected return starting from state s and following a policy π , while the action value function, $Q_{\pi}(s, a)$, evaluates the expected return after taking an action a in state s under the same policy. A policy π , which guides the agent's decisions, is defined as a probability distribution over actions given a state, denoted as $\pi(a|s)$. This means the policy specifies the likelihood of choosing each possible action in any given state, effectively directing the agent's behavior at each step of the decision-making process. Policies can be deterministic, where the action is chosen directly from the state, or stochastic, involving probabilities that reflect various levels of uncertainty or exploration strategies.

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=t+1}^T \gamma^{k-t-1} R_k \middle| S_t = s \right] \quad (7)$$

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=t+1}^T \gamma^{k-t-1} R_k \middle| S_t = s, A_t = a \right] \quad (8)$$

Value functions in reinforcement learning inherently satisfy recursive relationships, meaning the value of a state, or the expected return from that state, is determined based on the expected returns from future states. This process is mathematically represented by the Bellman Equation, which breaks down as follows:

$$V_{\pi}(s) := \mathbb{E}_{\pi}[G_t | S_t = s] \quad (9)$$

$$= \mathbb{E}_\pi \left[\sum_{k=t+1}^T \gamma^{k-t-1} R_k \middle| S_t = s \right] \quad (10)$$

$$= \sum_a \pi(a | s) \sum_{s', r} P(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \quad (11)$$

$$= \sum_a \pi(a | s) \sum_{s', r} P(s', r | s, a) [r + \gamma V_\pi(s')] \quad (12)$$

This equation effectively illustrates the expected value of the state-value function, $V_\pi(s)$, which unfolds as a comprehensive sum over all possible values of actions a , next states s' , and rewards r . For each combination of these variables, we first compute its probability using the policy and transition function, $\pi(a|s)P(s', r|s, a)$. We then multiply this probability by the sum of the immediate reward r and the discounted value of the next state $\gamma V_\pi(s')$ (i.e. the quantity within brackets), effectively weighting each potential outcome by its likelihood to calculate the expected value. Similarly, the action-value function $Q_\pi(s, a)$ calculates the expected return of taking action a in state s under policy π , and then following π thereafter. The corresponding Bellman equation is:

$$Q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t = a] \quad (13)$$

$$= \mathbb{E}_\pi[R_{t+1} + \gamma \mathbb{E}_{a \sim \pi} Q_\pi(S_{t+1}, a') | S_t = s, A_t = a] \quad (14)$$

This formulation demonstrates that the value of taking a particular action in a given state is determined by the immediate reward and the discounted value of the expected future rewards, where future actions are chosen according to policy π .

Optimal Value and Policy

Optimal policies in reinforcement learning are defined as the strategies that maximize the expected return from any given state within a finite Markov Decision Process (MDP). An optimal policy π^* yields a state-value function $V_{\pi^*}(s)$ that is greater than or equal to that produced by any other policy π for all states $s \in \mathcal{S}$. This concept is expressed through the optimal state-value function $V^*(s)$, defined mathematically as:

$$V_*(s) = \max_{\pi} V_\pi(s) \quad (15)$$

Similarly, the optimal action-value function $Q^*(s, a)$, which provides the maximum expected return for an action taken in a state followed by adherence to an optimal policy, is defined as:

$$Q_*(s, a) = \max_{\pi} Q_\pi(s, a) \quad (16)$$

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a] \quad (17)$$

Optimal policies are derived using the Bellman optimality equations, which provide a repeatable and consistent way to calculate the best possible outcomes from any state or state-action pair. These equations are essential for finding policies that yield the highest expected returns. The Bellman optimality equation for the optimal state-value function $V_*(s)$ is expressed as:

$$V_*(s) = \max_a Q_{\pi_*}(s, a) \quad (18)$$

$$= \max_a \mathbb{E}_{\pi_*} [G_t | S_t = s, A_t = a] \quad (19)$$

$$= \max_a \mathbb{E}_{\pi_*} [R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \quad (20)$$

$$= \max_a \mathbb{E} [R_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a] \quad (21)$$

$$= \max_a \sum_{s', r} P(s', r | s, a) [r + \gamma V_*(s')] \quad (22)$$

Similarly, the optimal action-value function $Q^*(s, a)$ follows the Bellman equation:

$$Q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q_*(S_{t+1}, a') | S_t = s, A_t = a] \quad (23)$$

$$= \sum_{s', r} P(s', r \mid s, a) \left[r + \gamma \max_{a'} Q_*(s', a') \right] \quad (24)$$

Derived from the Bellman optimality equations, optimal policies theoretically solve reinforcement learning challenges by maximizing expected returns. However, their practical application is often hindered by extensive computational demands and a deep need for accurate environmental knowledge. The principle of ‘greediness’—making decisions based solely on immediate rewards—emphasizes the practical limitations of these policies, as they may overlook long-term benefits for short-term gains. Solving the Bellman equations is akin to conducting an exhaustive search through all possible future states, requiring stringent assumptions about environmental dynamics and computational capacity. Moreover, the Markov assumption often proves impractical because real-world states are typically only partially observable, and basing decisions solely on the current state may not lead to achieving an optimal strategy. Consequently, reinforcement learning typically resorts to approximate solutions.

This section outlined the theoretical principles of reinforcement learning framed by MDPs, providing a structured way to conceptualize RL tasks. MDPs enable the modeling of agent-environment interactions through states, actions, rewards, and policies, offering a foundation for decision-making. However, practical applications of RL often deviate from these idealized equations due to the challenges mentioned above. Readers are encouraged to view these principles as a conceptual scaffold, guiding intuition rather than serving as strict prescriptions for implementation. The following sections will explore how these theoretical foundations translate into practical algorithms and advanced research topics.

3.3. Classification of Reinforcement Learning Algorithms

Building on the foundational theoretical framework of Markov Decision Processes (MDPs), this section explores how these principles are realized in practice through RL algorithms. By classifying and examining the strategies employed in RL, we demonstrate how abstract concepts are translated into practical methods for computing optimal policies and value functions, bridging the gap between theory and application.

3.3.1. Dynamic Programming

Dynamic Programming (DP) is a method used in environments where every aspect is completely known to compute optimal policies for finite Markov Decision Processes (MDPs). One popular DP approach involves two main phases: Policy Evaluation and Policy Improvement, which are iteratively applied to refine the policy towards the best possible strategy. The **Policy Evaluation** phase calculates how good each state is under the current policy by iteratively updating state values until there is little change, suggesting that the values accurately reflect the long-term outcomes. The **Policy Improvement** phase then attempts to find better actions than the current ones, updating the policy if better options are available. This process repeats, alternating between evaluating the policy and improving it, until no better policy can be found, indicating that the optimal policy has been achieved. A second popular DP approach is called **Value Iteration**, which quickly finds the optimal policies by directly updating the values of each state. This method merges the steps of evaluating and improving policies into a single process, significantly accelerating the path to finding the best actions to take in each state. Either of these DP methods can be used to compute the most optimal policy and value functions for *finite* MDPs given a complete and perfect knowledge of the environment.

3.3.2. Monte Carlo Methods

Monte Carlo (MC) methods provide an alternative approach for estimating value functions without requiring a detailed model of the environment. Unlike Dynamic Programming, MC methods rely on repeated sampling of complete episodes of interaction to learn from empirical experience.

In MC methods, the agent records the sequence of states visited, actions taken, and rewards received during each episode. The value of a state is then estimated as the average of the returns observed from all visits to that state across multiple episodes. This approach leverages the law of large numbers, ensuring that the estimated values converge to their true values with sufficient sampling.

Monte Carlo methods are particularly useful in environments with complex dynamics, as they eliminate the need for a transition probability model and focus solely on the outcomes of interactions. However, they require episodes to terminate for value estimation, making them less effective for environments with continuous or ongoing tasks. Despite this limitation, Monte Carlo methods are a robust tool for approximating value

functions and policies, especially when paired with advanced techniques to handle continuous tasks or large state spaces.

3.3.3. Temporal-Difference Learning

Temporal-Difference (TD) learning is a pivotal approach in reinforcement learning, combining the strengths of both Monte Carlo (MC) and Dynamic Programming (DP) methods. Like MC methods, TD learning does not require a model of the environment's dynamics, making it applicable to a wide range of problems. However, unlike MC methods, which wait until the end of an episode to update value estimates, TD learning incorporates the bootstrapping concept from DP. This means TD methods can update their value estimates immediately, using estimates of future states rather than waiting for complete outcomes.

To clarify, Monte Carlo methods rely on the total return G_t , representing the cumulative reward from time step t to the end of the episode, to update value function estimates. The constant- α Monte Carlo update rule is:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)] \quad (25)$$

In contrast, TD methods update their estimates after observing the next state, leveraging the reward R_{t+1} and the estimated value of the subsequent state $V(S_{t+1})$. The simplest TD method, known as TD(0), updates the state value function as follows:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)], \quad (26)$$

where α is the learning rate, γ is the discount factor, and R_{t+1} is the reward observed after transitioning from state S_t to S_{t+1} . For action-value functions, the update rule in TD(0) is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)], \quad (27)$$

where A_t and A_{t+1} represent the actions taken in states S_t and S_{t+1} , respectively. The following algorithm demonstrates the simplicity of the TD(0) process:

Algorithm 1 Simple TD(0) Algorithm for Estimating V_π

```

1: for each episode do
2:   Initialize  $t = 0$ 
3:   Start at initial state  $S_0$ 
4:   while the state is not terminal do
5:     Choose action  $A_t$  based on policy  $\pi$  for state  $S_t$ 
6:     Take action  $A_t$ , observe reward  $R_{t+1}$  and next state  $S_{t+1}$ 
7:     Update the value of  $S_t$ :

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

8:     Set  $S_t = S_{t+1}$  and increment  $t$ 
9:   end while
10: end for

```

TD learning offers significant advantages due to its ability to update estimates on-the-fly and learn from incomplete episodes. This makes it highly efficient for many real-world applications, where environments are dynamic and episodic data may be unavailable or impractical to collect in its entirety. By bridging the gap between Monte Carlo and Dynamic Programming, TD methods form a cornerstone of modern reinforcement learning techniques.

SARSA: On-Policy Temporal Difference Control

SARSA (State-Action-Reward-State-Action) is an on-policy TD control algorithm that learns the action-value function $Q(s, a)$ by directly interacting with the environment. SARSA updates Q values based on the sequence of states, actions, and rewards $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, ensuring the updates are consistent with the current

policy. The term "on-policy" reflects that the next action A_{t+1} is selected according to the same policy that the agent is following during learning.

Algorithm 2 SARSA (on-policy TD control) for estimating $Q \approx q_*$

- 1: Initialize $t = 0$
- 2: Start with S_0 and choose action A_0 using ϵ -greedy: $A_0 = \arg \max_{a \in \mathcal{A}} Q(S_0, a)$
- 3: **for** each step of the episode **do**
- 4: Take action A_t , observe reward R_{t+1} and next state S_{t+1}
- 5: Choose next action A_{t+1} using ϵ -greedy: $A_{t+1} = \arg \max_{a \in \mathcal{A}} Q(S_{t+1}, a)$
- 6: Update Q -value:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- 7: Set $t = t + 1$
 - 8: **end for**
-

SARSA ensures that updates to the Q -values reflect the current policy's behavior, making it particularly suited for situations where learning needs to incorporate the exploration inherent in the policy. The ϵ -greedy strategy, which balances exploration and exploitation, plays a crucial role in allowing the agent to refine its policy over time while ensuring it continues to explore new actions.

Q-Learning: Off-Policy Temporal Difference Control

Q-learning, introduced by Watkins and Dayan [24], is a foundational algorithm in reinforcement learning that estimates the optimal action-value function Q^* independently of the policy being followed. As an off-policy method, Q-learning learns the value of the optimal policy π^* while following an exploratory policy, such as ϵ -greedy. This separation allows Q-learning to converge to the optimal policy regardless of the agent's current behavior.

Algorithm 3 Q-learning (off-policy TD control) for estimating $Q \approx q_*$

- 1: Initialize $t = 0$
- 2: Start with S_0
- 3: **for** each step of the episode **do**
- 4: Take action A_t based on Q values using ϵ -greedy: $A_t = \arg \max_{a \in \mathcal{A}} Q(S_t, a)$
- 5: Observe reward R_{t+1} and next state S_{t+1}
- 6: Update Q -value:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- 7: Set $t = t + 1$
 - 8: **end for**
-

In Q-learning, the critical distinction from SARSA lies in its use of the maximum estimated future reward $\max_a Q(S_{t+1}, a)$ to update the Q -value, rather than relying on the action dictated by the current policy. This maximization step enables Q-learning to evaluate the optimal policy π^* , independent of the exploratory behavior of the agent. As a result, Q-learning is well-suited for tasks requiring agents to balance exploration with the drive to find globally optimal strategies.

3.3.4. On-Policy vs. Off-Policy Learning

We can first start off by explaining what exactly is a policy anyway? A policy is an agent's strategy; it's essentially a mapping from perceived states of the environment to actions to be taken when in those states. The goal is to learn the best policy, which defines the agent's way of behaving at a given time. There are multiple ways to assess policies, and reinforcement learning algorithms can be broadly categorized into on-policy and off-policy methods based on how they learn from behavior. We have said that SARSA is on-policy and Q-learning is off-policy, to which understanding the distinction between these approaches is crucial for implementing and analyzing reinforcement learning algorithms effectively. Policy learning in general refers to learning how to optimize decision making (target/update policy), from data generated from navigating an

environment (behavior policy). When it comes to **Off-Policy Learning**, the behavior policy can be different from the update policy [25]. The value function is updated based on the best possible actions, regardless of the actual actions taken by the agent. Q-learning is a quintessential off-policy method, where the agent updates its Q-values based on the maximum possible future reward:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad (28)$$

For **On-Policy Learning**, the behavior policy should be the same as the update policy. This means that the policy used to generate the behavior (i.e., the actions taken) and the policy being improved are the same. The value function is updated based on the actions actually taken by the agent. SARSA (State-Action-Reward-State-Action) is a prime example of an on-policy method, where the agent updates its Q-values based on the current policy's actions. Specifically, the Q-value update in SARSA depends on the action taken and the subsequent action the policy would take:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (29)$$

Off-policy learning offers several advantages:

- **Exploration vs. Exploitation:** The behavior policy can explore the environment more thoroughly, trying out various actions to gather a diverse set of experiences. Meanwhile, the target policy focuses on exploiting the best actions to maximize rewards, effectively balancing exploration and exploitation.
- **Learning Efficiency:** Off-policy methods can reuse past experiences stored in a replay buffer. This allows the agent to learn from a wider variety of experiences, not limited to the current trajectory, significantly speeding up the learning process.
- **Robustness:** By considering the best possible actions at each step, off-policy methods can be more robust to changes in the environment. They aim to learn the optimal policy directly, making them less dependent on the specifics of the behavior policy.
- **Policy Improvement:** Off-policy learning allows for continuous improvement. Even if the behavior policy is suboptimal, the agent can still learn the optimal policy by considering the maximum expected future rewards.

Off-policy algorithms, like Q-learning, decouple the update policy from the behavior policy, allowing the agent to explore a variety of actions, such as random, ϵ -greedy, or greedy¹, while still ensuring that the updates to the Q values are guided by a greedy policy. This flexibility enables the agent to gain diverse experiences and ultimately find the optimal policy by updating Q-values based on the best possible future rewards.

3.3.5. Function Approximation Methods

The methods discussed thus far have been primarily centered around tabular methods, which are suitable for environments with relatively small state spaces. However, many real-world problems (e.g., stock trading) feature state spaces that are combinatorially large or even infinite, rendering tabular methods impractical. In such cases, we must resort to approximate solution methods to efficiently find near-optimal policies. In environments with large state spaces, it is often impossible to encounter every possible state, making it crucial to generalize from previously seen states to new, unseen ones. This process is known as **function approximation**, where the goal is to approximate the value function or policy function using a more compact representation.

Function approximation replaces explicit tables with parameterized models. For example, the value function can be approximated as $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$, where $\hat{v}(s, \mathbf{w})$ denotes the approximated value of state s using a weight vector $\mathbf{w} \in \mathbb{R}^d$. These parameterized models can take various forms, such as linear combinations of state features or complex structures like deep neural networks. Adjusting the weights enables the model to represent a broad range of functions, allowing it to generalize effectively across large state spaces. While this approach introduces significant flexibility, it also adds complexity, as updates to parameters can influence multiple states and actions simultaneously.

The advent of **Deep Reinforcement Learning** (DRL), which leverages deep neural networks for function approximation, has transformed RL applications. DRL allows for the approximation of value functions $\hat{v}(s, \mathbf{w})$,

¹A random policy selects actions uniformly at random. An ϵ -greedy policy chooses the action with the highest estimated value (greedy action) most of the time but selects a random action with probability ϵ to encourage exploration. A greedy policy always selects the action with the highest estimated value, without exploration.

policies $\pi(a|s; \theta)$, or both, enabling RL systems to address challenges in high-dimensional and dynamic environments.

In the sections that follow, we delve into key categories of function approximation methods within Deep Reinforcement Learning (DRL). First, we explore **Actor-Critic** methods, which integrate value estimation and policy optimization in a unified framework. Prominent examples include models such as TRPO, PPO, A3C/A2C, DDPG, SAC, and TD3. Next, we discuss **Policy Optimization** in more detail—an approach focused on directly optimizing policies, which can also incorporate actor-critic elements. Following this, we extend our earlier discussion of Q-Learning to **Deep Q-Learning**, a method that combines value-based learning with deep function approximation. Finally, we examine **Hybrid Approaches**, which merge elements from actor-critic and Q-learning to leverage the strengths of both methodologies.

Actor-Critic

We discussed function approximation methods earlier in this section, mentioning a differentiable state-value function parameterization $\hat{v}(s, \mathbf{w})$ as well as a differentiable policy parameterization $\pi(a|s; \theta)$. Actor-Critic Methods essentially utilize both of these parameterizations. In Actor-Critic methods, we have two main components: the actor and the critic.

The **critic** is responsible for evaluating the action taken by the actor by computing the value function, which could be the action-value function $Q(a|s; \mathbf{w})$ or the state-value function $V(s; \mathbf{w})$. The critic's goal is to provide a signal to the actor about how good or bad the action was.

The **actor** is responsible for updating the policy parameters θ in the direction suggested by the critic. This involves updating the policy to maximize the expected reward, using the gradient of the policy. Here's how it works in an action-value actor-critic algorithm:

Algorithm 4 Template for Actor-Critic Algorithms; adapted from [17], [16], and [26] for updating online (i.e., every timestep)

- 1: Initialize policy parameters θ and value function parameters $\mathbf{w}$ at random.
- 2: Observe initial state s .
- 3: **for** $t = 1 \dots T$ **do**
- 4: **while** s is not terminal **do**
- 5: Draw action $a \sim \pi(a|s; \theta)$.
- 6: Observe reward r_t and next state s' .
- 7: Compute the correction for action-value at time t :

$$\delta \leftarrow r_t + \gamma \hat{v}(s', \mathbf{w}) - \hat{v}(s, \mathbf{w}) \quad (30)$$

- 8: and use it to update value function parameters:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) \quad (31)$$

- 9: and to update the policy parameters:

$$\theta \leftarrow \theta + \alpha_{\theta} \delta \nabla_{\theta} \log \pi(a|s; \theta) \quad (32)$$

- 10: Move into new state $s \leftarrow s'$.
 - 11: **end while**
 - 12: **end for**
-

Policy Optimization

Policy Optimization methods focus on directly optimizing the policy that the agent follows. Instead of learning state/action value functions and deriving policies from them, these methods parameterize the policy itself and adjust the parameters to maximize expected rewards. The parameterized function that represents the policy is often a neural network, and the parameters are updated in the direction that increases the expected reward. Let $\pi(a|s; \theta)$ denote the policy parameterized by θ . The objective is to maximize the expected return, which can be expressed as:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[R] \quad (33)$$

To update the policy parameters, we use the gradient ascent method. The policy gradient theorem provides a way to compute the gradient of the expected return with respect to the policy parameters [27]:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (34)$$

where Ψ_t can be one of the following:

1. $\sum_{t=0}^T r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^T r_{t'}$: reward following action a_t .
3. $\sum_{t'=t}^T r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^{\pi}(s_t, a_t)$: state-action value function.
5. $A^{\pi}(s_t, a_t)$: advantage function.
6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual.

The **advantage function** $A^{\pi}(s_t, a_t)$ is particularly useful. It represents the difference between the Q-value of an action and the value of the state, i.e., $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$. The advantage function helps to quantify how much better (or worse) an action is compared to the average action at a given state, and it remains unbiased and reduces variance.

Note: For a detailed discussion on why this remains unbiased and reduces variance, please refer to the Appendix (Section A.2).

Using this gradient, the parameters are updated iteratively as follows:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t) \quad (35)$$

Note: For a detailed discussion on the policy gradient theorem and derivation for basic Policy Gradient, please refer to the Appendix (Section A.3).

Policy Optimization Algorithms

Below, we will discuss some of the most popular policy gradient algorithms, such as TRPO, PPO, and A3C/A2C.

Trust Region Policy Optimization (TRPO)

When it comes to vanilla policy gradient methods, it is difficult to choose the correct stepsize that will work for the entirety of the optimization process. This is because, as time goes on, the policy is constantly changing; in other words, the statistics (observations and rewards distributions) of your inputs are changing over the training period. Large updates can lead to a bad policy, which can result in the agent making suboptimal decisions, ultimately leading to significant performance degradation that cannot recover. Trust Region Policy Optimization (TRPO) [27, 28], an advanced policy gradient method that improves the stability and performance of policy updates, addresses this issue by restricting the update step to a "trust region," ensuring that the new policy does not deviate too much from the old policy. This is achieved by using a constraint based on the Kullback-Leibler (KL) divergence (a measure of how one probability distribution differs from another), the Conjugate Gradient Method (an optimization technique to efficiently solve the constrained optimization problem), and Line Search (a method to find the maximum step size that satisfies the trust region constraint, ensuring stable and reliable updates).

Algorithm 5 Trust Region Policy Optimization (TRPO); adapted from [29] and [30]

- 1: Initialize policy parameters θ and value function parameters w .
- 2: **Hyperparameters:** KL-divergence limit δ , backtracking coefficient α , maximum number of backtracking steps K
- 3: **for** $k = 0, 1, 2, \dots$ **do**
- 4: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 5: Compute rewards-to-go $\hat{R}_t$.
- 6: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{w_k} .
- 7: Estimate policy gradient as

$$\hat{g}_k = \mathbb{E}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]_{\theta=\theta_k} \quad (36)$$

- 8: Use the conjugate gradient algorithm to compute

$$\hat{x}_k \approx \hat{H}_k^{-1} \hat{g}_k, \quad (37)$$

where $\hat{H}_k$ is the Hessian of the sample average KL-divergence.

- 9: Update the policy by backtracking line search with

$$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{\hat{x}_k^T \hat{H}_k \hat{x}_k}} \hat{x}_k, \quad (38)$$

where $j \in \{0, 1, 2, \dots, K\}$ is the smallest value which improves the sample loss and satisfies the sample KL-divergence constraint.

- 10: Fit value function by regression on mean-squared error:

$$w_{k+1} = \arg \min_w \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_w(s_t) - \hat{R}_t \right)^2, \quad (39)$$

typically via some gradient descent algorithm.

- 11: **end for**
-

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [31] is designed to address the same issue discussed in section 3.3.5 regarding Trust Region Policy Optimization (TRPO), where making too large updates to the policy can lead to performance collapse. This policy gradient method also improves the stability and performance of policy updates but with a much simpler approach compared to TRPO, and is one of the most widely used policy optimization algorithms due to its simplicity and performance. PPO achieves this by introducing a surrogate objective function that restricts the new policy from deviating too far from the old policy. This is done through two main techniques:

PPO-Clip: Unlike with TRPO, PPO-Clip does not use a constrained optimization solution. Instead, there is a clipping mechanism in the objective function that limits the size of the updates to maintain stable performance throughout the training process.

PPO-Penalty: Similar to TRPO, PPO-Penalty approximates a solution towards a KL-constrained update. However, PPO adds a penalty to the objective function instead of imposing a hard constraint. Additionally, the penalty coefficient is automatically adjusted over the course of training.

Among these techniques, the most popular one used in PPO is the **PPO-Clip**. The objective for PPO-Clip can be simplified as:

$$L^{CLIP}(s, a, \theta_{old}, \theta) = \mathbb{E} \left[\min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A^{\theta_{old}}(s, a), g(\epsilon, A^{\theta_{old}}(s, a)) \right) \right] \quad (40)$$

where

$$g(\epsilon, A) = \begin{cases} (1 + \epsilon)A & \text{if } A \geq 0 \\ (1 - \epsilon)A & \text{if } A < 0 \end{cases} \quad (41)$$

- $\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$ denotes the probability ratio between old and new policies.
- $g(\epsilon, A)$ clips the ratio to be no more than $1 + \epsilon$ and no less than $1 - \epsilon$. This forces the ratio to stay within a small interval around 1 (i.e. $[1 - \epsilon, 1 + \epsilon]$) where ϵ is a hyperparameter.

If the advantage $A^{\theta_{old}}$ for a state-action pair is positive, this means the action chosen is better than average, and thus the optimization will want to make $\pi_\theta(a|s)$ increase. However, the clipping term ensures that the update chooses $\min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}, (1 + \epsilon)\right) A^{\theta_{old}}(s, a)$. If the advantage $A^{\theta_{old}}$ for a state-action pair is negative, this means the action chosen is worse than average, and thus the optimization will want to make $\pi_\theta(a|s)$ decrease. However, the clipping term ensures that the update chooses $\max\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}, (1 - \epsilon)\right) A^{\theta_{old}}(s, a)$. While PPO-Clip goes a long way towards ensuring reasonable policy updates, it is still possible to end up with a new policy that is too far from the old policy. Various implementations of PPO include different tricks to handle this issue, but the described method is the simplest form.

Algorithm 6 PPO-Clip; adapted from [32]

- 1: Initialize policy parameters θ and value function parameters w .
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 4: Compute rewards-to-go $\hat{R}_t$.
- 5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{w_k} .
- 6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right) \quad (42)$$

typically via stochastic gradient ascent with Adam.

- 7: Fit value function by regression on mean-squared error:

$$w_{k+1} = \arg \min_w \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_w(s_t) - \hat{R}_t \right)^2, \quad (43)$$

typically via some gradient descent algorithm.

- 8: **end for**
-

Asynchronous/Synchronous Advantage Actor Critic Method (A3C/A2C)

The Advantage Actor Critic methods [33], Asynchronous Advantage Actor Critic (A3C) and Synchronous Advantage Actor Critic (A2C), focus on leveraging parallel computing and efficient software engineering practices rather than introducing new mathematical foundations. Therefore, there is no real need for pseudocode. A3C, the original method, employs a multi-threaded approach where several workers run concurrently across multiple threads or cores, each interacting with its own instance of the environment. This parallelism allows the workers to gather diverse experiences independently, which are then used to update a shared model asynchronously. A2C, a synchronous variant of A3C, simplifies this architecture by synchronizing updates from all parallel environments. Instead of asynchronous updates, A2C aggregates gradients from all versions of the environment and performs a single, synchronized update. This helps prevent workers from playing with policies of different versions (i.e., older parameter versions). Both A2C and A3C are built on the actor-critic framework, where the actor selects actions based on the current policy, and the critic evaluates these actions by estimating the value function. The advantage function is used to guide policy updates, helping the agent learn which actions are better than others relative to the current policy.

When to Use A3C vs. A2C:

- **A3C:** Best suited for CPU-based systems where you can exploit multi-core architectures to run multiple threads independently.
- **A2C:** Best suited for GPU-based systems where the synchronized processing of gradients can be efficiently managed.

Note to Readers: This approach does not refer to multi-agent learning in the traditional sense, where multiple independent agents interact with each other or compete in a shared environment. Instead, A3C uses multiple parallel workers or threads to accelerate learning by exploring different parts of the environment simultaneously.

Deep Q-Learning

Most aspects of Q-learning were discussed in Section 3.3.3. Q-learning methods focus on learning an approximate action-value function, $Q_\theta(s, a)$, that converges to the optimal action-value function, $Q^*(s, a)$. These methods typically utilize an objective function based on the Bellman equation, which enables them to update the action-value estimates efficiently. Q-learning is often performed off-policy, allowing it to use data collected at any time during training, independent of the agent's exploration strategy at the time the data was obtained. The resulting policy, π^* , selects actions based on the learned action-values by choosing actions that maximize the estimated Q value:

$$a(s) = \arg \max_a Q_\theta(s, a) \quad (44)$$

Deep Q-Learning Algorithms

Below, we will discuss some of the most popular Q-Learning algorithms, such as DQN, Double DQN, Dueling DQN, and Rainbow DQN.

Deep Q-Network (DQN)

Due to the instability and divergence issues inherent in Q-learning methods, the Deep Q-Network (DQN) [34, 35] introduces two key mechanisms to stabilize training: experience replay and periodically updated target networks. DQN is an algorithm that combines Q-learning with deep neural networks to solve complex reinforcement learning problems. The primary goal of DQN is to learn a Q-function, which maps state-action pairs to expected future rewards. This Q-function helps determine how much better it is to take a particular action in a given state, with the aim of maximizing cumulative rewards over time. DQN paved the way for further advancements in deep reinforcement learning by addressing some of the core challenges of traditional Q-learning methods.

Key Components of DQN:

- **Q-Function Approximation with Neural Networks:**

Instead of using a table to store Q-values for every state-action pair, which is impractical for large state spaces, DQN uses a neural network to approximate the Q-function. The network takes the current state as input and outputs Q-values for all possible actions. By using a neural network, DQN can generalize the Q-function over large state spaces, allowing it to learn from complex environments like Atari games.

- **Experience Replay:**

Experience replay is a technique where the agent's experiences, or transitions (s_t, a_t, r_t, s_{t+1}) , are stored in a replay memory D_t . During training, random samples are drawn from this memory to update the Q-values. This approach helps to break the correlation between consecutive updates, making the learning process more stable and data-efficient by allowing the agent to reuse past experiences multiple times. Experience replay stabilizes learning by ensuring the agent learns from diverse experiences, preventing overfitting to recent experiences.

- **Periodically Updated Target Network:**

The target Q-network, $\hat{Q}$, is used to compute the target values for the Q-learning updates. This network is kept fixed for a certain number of steps (denoted by C), after which its weights are updated to match the current Q-network Q . This reduces oscillations in the Q-values and stabilizes the learning process by providing consistent target values during the update period. The target network acts as a stabilizing anchor, allowing the network to focus on policy improvement without being misled by rapidly changing targets.

- **Epsilon-Greedy Exploration:**

To balance exploration and exploitation, DQN uses an epsilon-greedy strategy. With probability ϵ , the agent explores by choosing a random action, and with probability $1 - \epsilon$, it exploits by choosing the action with the highest Q-value. The epsilon-greedy strategy ensures the agent explores to discover good policies while exploiting known ones to maximize rewards.

Loss Function:

The loss function for DQN is defined as the mean-squared error between the predicted Q-values and the target Q-values, using a uniform distribution $U(D)$ over the replay memory:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (45)$$

where θ^- are the parameters of the target Q-network, and γ is the discount factor.

Algorithm 7 Deep Q-Network (DQN); adapted from [35]

```

1: Initialize replay memory  $D$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \arg \max_a Q(\phi(s_t), a; \theta)$ 
9:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
13:    Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j + 1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$ 
14:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$ 
15:    Every  $C$  steps reset  $\hat{Q} = Q$ 
16:   end for
17: end for

```

Note: In the pseudocode, $\phi(s)$ denotes a basic preprocessing step towards the state sequence, which aims to reduce the input dimensionality. This step is crucial for stabilizing learning in environments with high-dimensional inputs, like images from video games.

Double Deep Q-Network (DQN)

While DQN has significantly improved the stability and performance of Q-learning, it still suffers from overestimation bias. This bias occurs because when the max operator is used in DQN, it tends to select actions that have overestimated Q-values due to the inherent noise and approximation errors in the Q-function. This leads to the accumulation of overestimation errors over time, resulting in biased Q-value estimates. Double DQN [36] addresses this issue by separating the action selection from the action evaluation, without introducing additional networks. Instead, it leverages the existing target network in the DQN architecture. In DQN, the target Y_t is calculated using the maximum Q-value for the next state-action pair, which is estimated by the target network:

$$Y_t^{DQN} = r_t + \gamma \max_a Q(s_{t+1}, a; \theta^-) \quad (46)$$

Here, r_t is the reward at time step t , γ is the discount factor, and $Q(s_{t+1}, a; \theta^-)$ is the estimated Q-value for the next state s_{t+1} and action a , where θ^- are the parameters of the target network. Double DQN mitigates this bias by using the online network to select the action and the target network to evaluate the Q-value of this selected action. The target Q-value for a given state-action pair in Double DQN is calculated as follows:

$$Y_t^{Double-DQN} = r_t + \gamma Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta^-) \quad (47)$$

In this equation, $Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta^-)$ is the estimated Q-value for the next state s_{t+1} and the action a that maximizes the Q-value, where θ are the parameters of the online network used to select the action, and θ^- are the parameters of the target network used to evaluate the Q-value of the selected action.

Dueling Deep Q-Network (DQN)

While traditional Q-networks learn a single Q-value for each state-action pair, Dueling Deep Q-Networks (Dueling DQN) [37] introduce a more sophisticated architecture that separately estimates two components: the state value and the advantage function. The key insight behind Dueling DQN is that in many states, the choice of action does not always significantly affect the outcome. For instance, in certain scenarios like navigating in an environment, knowing the value of moving left versus right may only be crucial in specific contexts, such as avoiding obstacles. In other states, the specific action taken may have little to no impact on the outcome, making it unnecessary to estimate the value of each action choice. By separating the value of being in a state from the value of taking a specific action, Dueling DQN can learn more efficiently and generalize better across actions. The Dueling DQN architecture consists of two streams that represent the following:

- **State Value Function** $V(s)$: This component estimates the expected return of being in a particular state s , independent of the action taken.
- **Advantage Function** $A(s, a)$: This component estimates the relative importance of each action a in a given state s , i.e., how much better or worse an action is compared to other actions in the same state.

These two streams are combined to produce the final Q-values for each action as follows:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \quad (48)$$

In this equation:

- θ denotes the parameters of the convolutional layers that extract features from the input state s .
- α and β represent the parameters of the fully connected layers in the advantage and value streams, respectively.
- $V(s; \theta, \beta)$ is the state value function parameterized by θ and β .
- $A(s, a; \theta, \alpha)$ is the advantage function parameterized by θ and α .
- The subtraction of the mean advantage ensures that the Q-values are centered around the value function $V(s)$, which stabilizes training and mitigates the effect of noisy advantage estimates.

It's important to note that while $V(s)$ and $A(s, a)$ are two separate estimators, they are implemented as part of the same network, sharing a common convolutional feature learning module (i.e. shared network parameters).

Rainbow Deep Q-Network (DQN)

Since the introduction of Deep Q-Networks (DQN), several extensions have been developed to address various challenges and limitations of the original algorithm. For example, we just reviewed Double DQN, which reduces overestimation bias by separating action selection and evaluation, and Dueling DQN, which separates the value of being in a state from the value of taking a specific action. Rainbow Deep Q-Network (DQN) [38] combines six of these extensions into a single, integrated algorithm, demonstrating the benefits of their combination. The six extensions used in Rainbow DQN are:

- **Double DQN**: To mitigate overestimation bias in action values (see Equation 47).
- **Dueling DQN**: To separate the estimation of state values and advantages (see Equation 48).

- **Prioritized Experience Replay:** In standard DQN, experiences are sampled uniformly from the replay buffer. Prioritized Experience Replay [39] instead prioritizes experiences with higher learning potential, as measured by the magnitude of the temporal difference (TD) error. The priority p_i for each experience i is calculated as:

$$p_i = (|\delta_i| + \epsilon)^\alpha \quad (49)$$

where $\delta_i = r_i + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta)$ is the TD error, ϵ is a small positive constant added for numerical stability, and α is a hyperparameter that determines the degree of prioritization. The probability $P(i)$ of sampling experience i is proportional to its priority relative to the sum of all priorities in the buffer:

$$P(i) = \frac{p_i}{\sum_k p_k} \quad (50)$$

To correct for the bias introduced by prioritized sampling, an importance sampling (IS) weight w_i is applied to each sampled experience during the update step:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta = \left(\frac{p_i}{p_{\min}} \right)^{-\beta} \quad (51)$$

where N is the total number of experiences in the buffer, $p_{\min}$ is the minimum non-zero priority in the buffer, and β is a hyperparameter controlling the strength of the importance sampling correction. Optionally, the IS weights can be normalized within the batch:

$$\tilde{w}_i = \frac{w_i}{\max_j w_j} \quad (52)$$

This specific notation for Prioritized Experience Replay is derived from the implementation in the Tianshou deep reinforcement learning library [40].

- **Multi-Step Learning:** Traditional Q-learning methods update based on the immediate reward plus the discounted expected return of the next state. Multi-step learning extends this by considering the cumulative reward over multiple steps, providing a more comprehensive return $R_t^{(n)}$:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-) \quad (53)$$

This approach helps capture the delayed impact of decisions by accumulating rewards over n time steps, rather than just a single step. The final term incorporates the Q-value to estimate future rewards beyond the n steps, ensuring that the long-term effects of an action are considered, even if they are not immediately visible.

- **Distributional RL:** Instead of predicting just the expected return, Distributional RL [41] models the entire distribution of returns. The return distribution $Z(s, a)$ is represented by a set of discrete values (atoms) z_i and associated probabilities p_i :

$$Z(s, a) = \sum_{i=1}^N p_i \delta_{z_i} \quad (54)$$

This approach is implemented in the Categorical DQN (C51) algorithm, which approximates the return distribution using a fixed set of atoms. The objective is to minimize the Kullback-Leibler (KL) divergence between the predicted and target distributions, allowing the model to capture the variability and uncertainty of returns more effectively. By modeling the entire return distribution, Distributional RL provides a more comprehensive understanding of the value function, leading to better decision-making and improved performance.

- **Noisy Nets:** Traditional Q-learning methods often rely on fixed exploration-exploitation strategies, such as ϵ -greedy, which can result in suboptimal exploration in complex environments where multiple actions have to be executed before any reward is collected. Noisy networks (Noisy Nets) address this limitation by introducing parameterized noise directly into the network's weights. In a noisy network,

the linear transformation in a neural network is augmented by adding noise to both the weights and the biases:

$$y = (b + Wx) + (\mathbf{b}_{\text{noisy}} \odot \epsilon_b + \mathbf{W}_{\text{noisy}} \odot \epsilon_W)x \quad (55)$$

where:

- $\mathbf{b}_{\text{noisy}}$ and $\mathbf{W}_{\text{noisy}}$ are the learnable parameters that represent the magnitude of noise for the bias and weight matrices, respectively.
- ϵ_b and ϵ_W are noise variables, typically drawn from a Gaussian distribution, which introduce randomness to the parameters.
- $\odot$ denotes the element-wise product, which means the noise is applied individually to each element in the weight and bias matrices.

The network learns both the standard parameters (the traditional weight and bias) and the noise parameters during training. This allows the model to control the level of exploration by adjusting the noise over time.

Rainbow DQN successfully integrates these extensions, achieving a more robust and efficient learning process than using any of these techniques individually.

Hybrid Approaches

Hybrid Algorithms

Below, we will discuss some of the most popular hybrid algorithms that carefully trade-off between the strengths and weaknesses of both Policy Optimization and Q-Learning, such as DDPG, TD3, and SAC.

Deep Deterministic Policy Gradient (DDPG)

The Deep Deterministic Policy Gradient (DDPG) algorithm [42] is a hybrid reinforcement learning approach that combines the advantages of both Policy Optimization and Q-Learning. It is particularly well-suited for continuous action spaces, where traditional Q-Learning methods struggle due to the need to discretize actions because of the curse of dimensionality. DDPG addresses this challenge by learning a deterministic policy that directly maps states to specific actions using a neural network instead of outputting the probability distribution across a discrete action space. DDPG is based on the Deterministic Policy Gradient (DPG) algorithm [43], which optimizes a deterministic policy instead of a stochastic one. The main components of DDPG include the actor-critic architecture, replay buffer, target networks, and exploration strategy. These components work together to provide stable and efficient learning in high-dimensional continuous environments. Below describes the key components of DDPG:

- **Actor-Critic Architecture:** The DDPG algorithm uses an actor-critic structure where the actor $\mu(s|\theta^\mu)$ is a deterministic policy that maps states to actions, and the critic $Q(s, a|\theta^Q)$ estimates the action-value function. The actor is updated by maximizing the expected return $J(\theta^\mu) = \mathbb{E}[Q(s, a)|_{s=s_t, a=\mu(s_t)}]$ using the policy gradient:

$$\nabla_{\theta^\mu} J(\theta^\mu) \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s=s_i} \quad (56)$$

The critic, on the other hand, is updated by minimizing the mean squared Bellman error:

$$\text{Loss} = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2 \quad (57)$$

where

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'} \quad (58)$$

- **Replay Buffer:** In reinforcement learning, particularly when using function approximators like neural networks, it is crucial that the data used for training is independently and identically distributed (IID). In an on-policy setting, where data is sequentially generated, this IID property is often violated because consecutive experiences are correlated. To address this issue, DDPG employs a replay buffer to store past experiences (s_t, a_t, r_t, s_{t+1}) . By randomly sampling mini-batches from this buffer, the algorithm effectively breaks the correlations between consecutive experiences, thereby approximating the IID condition. This leads to more stable and efficient learning, as the model is trained on a more diverse set of experiences.
- **Target Networks:** To stabilize learning, similar to the DQN algorithm in Section 3.3.5, DDPG uses target networks for both the actor and the critic. These networks, μ' and Q' , are copies of the actor and critic networks that are slowly updated to track the learned networks through soft updates. Unlike DQN, where the target networks are updated periodically by directly copying the parameters from the main networks, DDPG updates its target networks more gradually using a method called Polyak averaging:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \quad (59)$$

where $\tau \ll 1$. This approach ensures that the target values change slowly, providing greater stability during training. The use of Polyak averaging reduces the risk of divergence, as it maintains a more stable set of target values compared to the hard updates used in DQN.

- **Exploration Strategy:** In reinforcement learning, the approach to exploration differs depending on whether the action space is discrete or continuous. In discrete action spaces, exploration is typically handled by probabilistically selecting random actions, such as through epsilon-greedy. In continuous action spaces, as is the case with DDPG, exploration is achieved by adding noise directly to the actions suggested by the policy. Originally, DDPG employed the Ornstein-Uhlenbeck (OU) process to generate temporally correlated noise for exploration. This noise was specifically designed to maintain smooth and consistent actions over time:

$$\mu'(s_t) = \mu(s_t|\theta^\mu) + \mathcal{N}_t \quad (60)$$

where $\mathcal{N}_t$ is the noise generated by the OU process. However, more recent results suggests that uncorrelated, mean-zero Gaussian noise is equally effective and much simpler to implement [44]. This has led to a shift towards using Gaussian noise for exploration:

$$\mu'(s_t) = \mu(s_t|\theta^\mu) + \mathcal{N}(0, \sigma^2) \quad (61)$$

where $\mathcal{N}(0, \sigma^2)$ is Gaussian noise with zero mean and variance σ^2 .

Algorithm 8 DDPG Algorithm [42]

- 1: **Initialize** critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ
- 2: **Initialize** target networks Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$
- 3: **Initialize** replay buffer R
- 4: **for** episode = 1, M **do**
- 5: **Initialize** a random process $\mathcal{N}$ for action exploration
- 6: **Receive** initial observation state s_1
- 7: **for** t = 1, T **do**
- 8: **Select** action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
- 9: **Execute** action a_t and observe reward r_t and observe new state s_{t+1}
- 10: **Store** transition (s_t, a_t, r_t, s_{t+1}) in R
- 11: **Sample** a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
- 12: **Set** $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
- 13: **Update** critic by minimizing the loss:

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$$

- 14: **Update** the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s=s_i}$$

- 15: **Update** the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

- 16: **end for**

- 17: **end for**
-

Twin Delayed Deep Deterministic Policy Gradient (TD3)

DDPG (Section 3.3.5) is a powerful algorithm for solving continuous control problems, as it combines the strengths of both policy gradient and Q-learning methods. However, it is also known to be sensitive to hyperparameters and requires careful tuning to achieve good performance. One critical issue in DDPG is overestimation bias (also discussed in Section 3.3.5), which occurs due to approximation errors, leading to progressively inflated Q-value estimates over time. Twin Delayed DDPG (TD3) [45] addresses these challenges by introducing three key components:

- **Clipped Double Q-Learning**²: TD3 learns two Q-functions, Q_{ϕ_1} and Q_{ϕ_2} , instead of one, using both to reduce overestimation bias. However, only one target value is used, which happens to be whichever Q-function has the smallest one, which helps mitigate the impact of overestimated values:

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_i, \text{targ}}(s', a'(s')) \quad (62)$$

The two Q-functions are then updated by regressing towards this target:

$$L(\phi_1, D) = \mathbb{E}_{(s,a,r,s',d) \sim D} \left[(Q_{\phi_1}(s, a) - y(r, s', d))^2 \right] \quad (63)$$

$$L(\phi_2, D) = \mathbb{E}_{(s,a,r,s',d) \sim D} \left[(Q_{\phi_2}(s, a) - y(r, s', d))^2 \right] \quad (64)$$

By using the smaller Q-value for the target, TD3 effectively reduces the bias in the Q-function.

- **Delayed Policy Updates**: In TD3, the policy (actor network) is updated less frequently than the Q-functions. Specifically, the policy is updated once for every two Q-function updates. This delayed

²The authors of TD3 observed that the double Q-learning technique used in Section 3.3.5 is less effective in an actor-critic setting, prompting the need for an older implementation of Double Q Learning.

update strategy helps reduce the volatility that arises in DDPG from rapid policy changes, which can destabilize training. By updating the policy less frequently, TD3 ensures that the value (critic) networks have more time to converge towards accurate value estimates before the policy network is adjusted. This approach mitigates the risk of the policy network learning from unstable or overestimated Q-values, thereby enhancing the overall stability and reliability of the training process.

- **Target Policy Smoothing:** To make the policy robust to minor changes in action, TD3 adds noise to the target actions, effectively smoothing the Q-function. The target action is clipped to remain within the valid action range, and is defined as:

$$a'(s') = \text{clip}(\mu_{\theta_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}), \quad \epsilon \sim \mathcal{N}(0, \sigma) \quad (65)$$

where ϵ is Gaussian noise, c is a small constant to limit the noise range, and a_{Low} and a_{High} define the valid action range. This target policy smoothing acts as a regularizer, preventing the policy from overfitting to sharp peaks in the Q-function, which could lead to poor policy performance.

Together, these three components (clipped double Q-learning, delayed policy updates, and target policy smoothing) significantly enhance TD3's performance compared to baseline DDPG by addressing key vulnerabilities related to overestimation bias and policy stability (TD3 also adds noise to the actions at training time to explore better, similar to DDPG, typically using uncorrelated mean-zero Gaussian noise).

Algorithm 9 Twin Delayed DDPG (TD3) Algorithm (adapted from [46])

```
1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta, \phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute action  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   if  $s'$  is terminal, reset environment state then
9:     end if
10:  if it's time to update then then
11:    for  $j$  in range(however many updates) do
12:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
13:      Compute target actions
```

$$a'(s') = \text{clip}(\mu_{\theta_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}), \quad \epsilon \sim \mathcal{N}(0, \sigma) \quad (66)$$

```
14:      Compute targets
```

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', a'(s')) \quad (67)$$

```
15:      Update Q-functions by one step of gradient descent using
```

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2 \quad (68)$$

```
16:      if  $j \bmod \text{policy\_delay} = 0$  then
```

```
17:        Update policy by one step of gradient ascent using3:
```

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi_1}(s, \mu_\theta(s)) \quad (69)$$

```
18:        Update target networks with
```

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2 \quad (70)$$

$$\theta_{\text{targ}} \leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \quad (71)$$

```
19:      end if
```

```
20:    end for
```

```
21:  end if
```

```
22: until convergence
```

Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC)[47, 48] is an off-policy reinforcement learning algorithm extending DDPG-style methods with a maximum entropy (level of unpredictability) framework. This method uses a stochastic policy that samples actions from a probability distribution⁴, enhancing exploration. The key components of SAC include:

- **Entropy Regularization and Connection between Value and Q-Functions:** SAC enhances exploration by combining expected return maximization with entropy regularization, where entropy quantifies the randomness of the policy. The entropy-augmented objective of SAC is:

³In TD3, computational efficiency is achieved by using a single actor network optimized with respect to Q_{ϕ_1} , as outlined on page 4 of [45]

⁴This is actually parameterized outputs from the neural network and not sampling distribution methods you may have seen.

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log \pi(a_t|s_t)) \right] \quad (72)$$

where α is a temperature parameter that adjusts the trade-off between reward maximization and exploration through entropy. The value function $V^{\pi}(s)$ and the Q-function $Q^{\pi}(s, a)$ are intertwined through the entropy term, which modifies the traditional definitions to incorporate policy randomness directly into the value estimates. Specifically, the value function is adjusted by the entropy of the policy:

$$\hat{V}_{\phi}^{\pi_{\theta}}(s_t) = \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot|s_t)} \left[\hat{Q}_{\phi}^{\pi_{\theta}}(s_t, a_t) - \alpha \log \pi_{\theta}(a_t|s_t) \right] \quad (73)$$

This equation shows that $V^{\pi}(s)$ is the expected Q-value under the policy, adjusted for the entropy. The Q-function further incorporates this adjustment by including an entropy bonus at each timestep beyond the initial state:

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \middle| s_0 = s, a_0 = a \right]. \quad (74)$$

- **Clipped Double Q-Learning:** Similar to TD3, SAC uses two Q-functions to mitigate positive bias in the target estimation. The target is computed by taking the minimum of the two Q-values, reducing overestimation errors.
- **Reparameterization Trick:** The SAC policy uses the reparameterization trick, drawing samples from the policy by computing deterministic functions of state, policy parameters, and independent noise:

$$\tilde{a}_{\theta}(s, \xi) = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi), \quad \xi \sim \mathcal{N}(0, I). \quad (75)$$

This allows for backpropagation through the policy's stochasticity.

In the Soft Actor-Critic (SAC) algorithm, the value function is updated using sampled actions without reparameterization to capture the expected Q-values adjusted by entropy. Then the actor is updated using reparameterized actions, allowing gradients to flow through the sampling process, balancing reward maximization and exploration. The Q-functions (critics) are updated using actions from the replay buffer, which ensures they accurately learn from the agent's real experiences. SAC incorporates entropy regularization in all updates to encourage exploration and maintain stability. Variants of SAC exist, with some earlier versions including an additional value function, though modern implementations typically omit it to simplify the architecture.

Algorithm 10 Soft Actor-Critic (SAC) Algorithm (adapted from [47])

- 1: **Input:** initial policy parameters θ , Q-function parameters ϕ_1, ϕ_2 , empty replay buffer $\mathcal{D}$
- 2: **Set** target parameters equal to main parameters $\phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$
- 3: **repeat**
- 4: Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
- 5: Execute action a in the environment
- 6: Observe next state s' , reward r , and done signal d to indicate whether s' is terminal
- 7: Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
- 8: **if** s' is terminal, reset environment state **then**
- 9: **end if**
- 10: **if** it's time to update **then**
- 11: **for** j in range(however many updates) **do**
- 12: Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
- 13: Compute targets for the Q-functions:

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', a') - \alpha \log \pi(a'|s') \right), \quad a' \sim \pi(\cdot|s') \quad (76)$$

- 14: Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2 \quad (77)$$

- 15: Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left[\alpha \log \pi_\theta(a_t|s_t) - \min_{i=1,2} Q_{\phi_i}(s, a_t) \right] \quad (78)$$

- 16: Update target networks with

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2 \quad (79)$$

- 17: **end for**
 - 18: **end if**
 - 19: **until** convergence
-

Conclusion

Throughout this section, we explored a range of RL algorithms, highlighting their diverse mechanisms, applications, and infinitely tuneable parameters. The flexibility of RL is evident in the multitude of approaches it offers—online vs. batch learning, on-policy vs. off-policy methods, stochastic vs. deterministic policies, and varied strategies for estimating value and advantage functions. A critical distinction in RL is the timing of updates: whether they occur after every timestep, episode, or batch of episodes, which significantly impacts learning dynamics and stability. Each algorithm presents unique strengths and challenges. For example, methods like TD3 and SAC incorporate innovative solutions such as clipped double Q-learning and entropy regularization, addressing common pitfalls like overestimation bias and exploration-exploitation balance. Additionally, the integration of neural networks allows for function approximation in complex state spaces, overcoming the limitations of traditional tabular methods. Ultimately, the choice of RL algorithm and configuration depends on the specific problem at hand, including the nature of the action space (continuous or discrete), the desired trade-off between exploration and exploitation, and the computational constraints. As we transition to the next section, we will delve into practical tips for implementing RL methods, ensuring robust and effective deployment in real-world scenarios.

3.4. RL Practical Tips

RL is a powerful tool for solving complex decision-making problems, but the practical implementation of RL algorithms often involves challenges that go beyond theoretical formulations. While academic papers provide the foundational algorithms and their performance benchmarks, real-world applications require careful tuning, robust debugging, and strategic adjustments to deal with the nuances of specific environments. This section aims to bridge the gap between theory and practice by compiling actionable tips and strategies gathered from various sources, including research papers, community discussions, and expert insights. By following these guidelines, you can improve the reliability and efficiency of your RL models, ensuring that they perform not just in controlled settings but also in complex, real-world scenarios. Whether you are a researcher, engineer, or enthusiast, these insights will help you navigate the often unpredictable journey of reinforcement learning [49, 50, 51, 52, 53, 54, 55, 56].

3.4.1. Random Seeds and Trials

RL algorithms are often brittle and unstable, making reproducibility a significant challenge. The variance in results due to different random seeds can be substantial, impacting the training performance and making comparisons between algorithms unreliable. It is crucial to:

- Use multiple random seeds for every experiment (at least 3-10) and report both average performance and standard deviation.
- Regularly evaluate the agent on a separate test environment without exploration noise to obtain an unbiased estimate of performance.
- Re-run experiments with the same setup to ensure consistency and check the impact of hyperparameters and initialization.

3.4.2. Logging, Monitoring, and Visualization

Tracking all relevant metrics during RL training is essential for diagnosing and understanding model behavior. Commonly logged metrics include policy entropy, value function estimates, reward distributions, and gradients. Key practices include:

- Use tools like TensorBoard or Weights & Biases to log and visualize losses, rewards, actions, and learning rates.
- Log arbitrary metrics and track detailed statistics on a per-step or per-episode basis, such as mean, standard deviation, and min/max values.
- Visualize your agent's behavior periodically through videos to identify unexpected actions and learning anomalies.
- Monitor the behavior of loss curves, such as the expected decrease in standard deviations in PPO, to detect healthy learning patterns.

3.4.3. Normalization and Data Preprocessing

Normalization of inputs, rewards, and target values is critical in stabilizing training. Poorly scaled data can cause large gradients and destabilize learning:

- Standardize observations to zero mean and unit variance using running estimates or normalize to a specific range (e.g., $[0, 1]$).
- Rescale rewards appropriately but avoid shifting them, as it may alter the learning dynamics and the agent's incentives.
- Use layer normalization or batch normalization within neural networks to improve stability during training.
- Apply whitening techniques to further standardize data, ensuring consistent preprocessing across observations and rewards.

3.4.4. Hyperparameter Tuning and Sensitivity Analysis

RL algorithms are highly sensitive to hyperparameters, including learning rates, discount factors, and exploration coefficients. Strategies for effective tuning include:

- Conduct random search or grid search for hyperparameter optimization, focusing on key parameters like learning rate and batch size.
- Monitor learning curves to adjust parameters dynamically, such as decaying learning rates or adjusting exploration noise based on performance.
- Use cyclical learning rates or sensitivity analysis to identify optimal ranges for critical hyperparameters.
- Understand the trade-offs between different value estimation methods, update frequencies, and network architectures.

3.4.5. Reward Shaping and Scaling

Designing effective reward functions and managing exploration-exploitation trade-offs are crucial for RL success. Common strategies include:

- Shape rewards to provide informative feedback early in training; consider using intermediate rewards for complex tasks to guide learning.
- Control exploration through mechanisms like entropy regularization, exploration noise, or scheduled decay of exploration parameters.
- Test reward scaling and understand how the final reward value, not just the scaling factor, impacts agent performance.
- Analyze the behavior of different reward structures on learning stability and convergence to avoid misleading feedback signals.

3.4.6. Statistical Testing and Rigorous Evaluation

Proper evaluation and statistical testing are essential to ensure reliable conclusions about algorithm performance:

- Use statistical tests to ensure robust performance comparisons (e.g. Welch's t-test).
- Report averages, standard deviations, and confidence intervals rather than just best-case results to provide a comprehensive view of performance.
- Use rigorous evaluation procedures, including multiple trials and careful data management, to reduce Type-I (false positive) and Type-II (false negative) errors.

3.4.7. Baseline Comparisons and Benchmarking

Benchmarking against well-known baselines is essential for assessing RL algorithms' true performance:

- Run standard baseline algorithms (e.g., cross-entropy method, well-tuned policy gradient methods) for comparison.
- Fine-tune baselines to ensure fair comparisons; do not rely on default parameters, as they often do not yield optimal performance.
- Regularly evaluate your RL agent against these baselines to validate improvements and ensure robustness across different scenarios.

3.4.8. Debugging and Validation

RL models are notoriously difficult to debug due to their non-deterministic nature and complex interactions between components. Effective debugging tips include:

- Conduct sanity checks on initial outputs, gradient norms, action distributions, and weight initializations to catch early errors.
- Test critical components independently, such as the policy, value functions, and replay buffer logic, to ensure each works as expected.
- Use unit tests and controlled environments to validate the consistency of key functions, especially when implementing custom algorithms.

3.4.9. Compute Resources and Experiment Management

Effective use of compute resources and organized experiment management can greatly accelerate the RL research process. Best practices include:

- Leverage cloud computing platforms (AWS, Google Cloud) to run large-scale experiments efficiently and manage computational costs.
- Run experiments in parallel with multiple seeds to quickly gather statistically significant results and identify performance trends.
- Maintain an organized experiment log with details on each run, including hyperparameters, random seeds, and specific changes in the code.

3.4.10. Scaling Up from Simple Tasks

When developing new algorithms or techniques, begin with simple tasks that allow for rapid experimentation and debugging:

- Test on small-scale problems like CartPole or basic grid environments before moving to more complex tasks.
- Use insights gained from simpler tasks to refine your methods and gradually scale up to larger, more realistic environments.
- Avoid overfitting algorithms to toy problems; ensure that methods can generalize to broader, more varied contexts.

3.4.11. Community and Knowledge Sharing

Engaging with the RL community can provide valuable insights, help identify common pitfalls, and offer new perspectives on complex problems:

- Participate in forums like Reddit's RL community, GitHub discussions, or research groups to exchange knowledge and discuss experimental results.
- Regularly review the latest literature and open-source implementations to stay updated on emerging best practices and innovative techniques.
- Share your findings and contribute to open-source projects to foster a collaborative and supportive RL research environment.

By implementing these practical tips, researchers and practitioners can navigate the complexities of reinforcement learning more effectively, ensuring that their models not only function well in controlled experimental setups but also maintain robustness and scalability in real-world applications.

3.5. Natural Language Processing

NLP is a field of artificial intelligence that focuses on the interaction between computers and human languages. It involves enabling machines to understand, interpret, and generate human language in a valuable way. NLP combines computational linguistics, machine learning, and deep learning to process and analyze large amounts of natural language data, which has broad applications across industries, including finance, healthcare, and customer service [57]. The evolution of NLP can be traced through several key milestones, each marking significant advancements in the field [58]. Early NLP systems were rule-based, relying on hand-crafted

linguistic rules and heuristics that could perform basic syntactic analysis but struggled with the complexity and variability of human language [59]. The rise of statistical methods in the 1990s introduced more flexibility, allowing models to learn patterns from data rather than depending solely on manually defined rules [60]. The next major leap came with the introduction of word embeddings such as Word2Vec and GloVe in the 2010s, which allowed words to be represented as dense vectors capturing semantic relationships [61, 62]. This innovation was followed by the development of deep learning architectures like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, which improved the ability to handle sequential data and context in language processing tasks [63]. The transformative moment in NLP occurred with the advent of the Transformer architecture, introduced by Vaswani et al. in 2017 [64]. Transformers, characterized by their self-attention mechanisms, significantly improved the performance and efficiency of NLP models, paving the way for large-scale pre-trained language models. BERT (Bidirectional Encoder Representations from Transformers) was one of the first models to fully exploit this architecture, enabling contextualized word embeddings that greatly enhanced the performance of downstream NLP tasks [65]. Since BERT, numerous variants and improvements have emerged, pushing the boundaries of what NLP models can achieve. Today, NLP is not only foundational in AI but continues to evolve rapidly, driven by advancements in model architectures, optimization techniques, and the increasing availability of vast amounts of data. The field's journey from rule-based systems to sophisticated deep learning models illustrates a broader shift toward more adaptive and intelligent language processing systems, making NLP one of the most critical components of modern AI applications. This section will explore the key stages of NLP development, starting with **Foundational Text Representation Methods**, including Bag-of-Words, TF-IDF, N-grams, and classic word embeddings like Word2Vec, GloVe, and FastText. We will then dive into the **Transformer Revolution**, with a focus on self-attention mechanisms and contextual embeddings. Finally, we will examine the newest generation of **Large Language Models (LLMs)**, such as ChatGPT, which have further pushed the boundaries of what NLP models can achieve, leading to increasingly sophisticated language processing systems.

3.5.1. Traditional Methods in NLP

Classical Text Representation Methods

Bag-of-Words (BoW)

Bag-of-Words is one of the simplest and most commonly used techniques for text representation in the early days of NLP. In BoW, a document is represented as a vector of word counts, disregarding grammar, word order, and context but preserving frequency information. The process involves creating a vocabulary of all unique words from a collection of documents and then representing each document as a vector where each entry corresponds to the frequency of a word in that document. For example, if the vocabulary consists of the words *[apple, banana, fruit]*, then the sentence "apple and banana" would be represented as *[1, 1, 0]*. Despite its simplicity, BoW can effectively capture the presence of important terms in text data and can be used in tasks like text classification/information retrieval [66], and basic sentiment analysis where context is less critical.

Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents. It improves upon BoW by not just counting word occurrences but also adjusting for how common or rare a word is across all documents. The term frequency (TF) component captures how often a word appears in a document, while the inverse document frequency (IDF) component downweights words that are frequent across many documents, reflecting their lower discriminative power. Mathematically, TF-IDF is calculated as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right) \quad (80)$$

where t is the term, d is the document, N is the total number of documents, and $\text{DF}(t)$ is the number of documents containing the term t . TF-IDF remains popular for feature extraction in text mining and information retrieval [67].

N-grams

N-grams are contiguous sequences of n items (typically words or characters) extracted from a given text. For example, a 2-gram (bigram) representation of the sentence "machine learning" would be *["machine learning"]*,

while a 3-gram (trigram) could be [*deep learning models*]. N-grams capture local context by considering adjacent word pairs or groups, thus incorporating some level of word order information. N-grams can be used to generate features for machine learning models [67] or even to identify duplicated pairs of documents using high n-gram overlap [68]. While higher-order N-grams can capture more context, they also lead to increased dimensionality and processing time.

Current Relevance and Use

Despite advancements in deep learning and contextual embeddings, these traditional methods remain relevant for informational retrieval and preprocessing steps, especially when computational simplicity, interpretability, and low resource consumption are priorities. Moreover, hybrid approaches often incorporate these techniques as baseline features or initial layers in more complex models [69, 70], demonstrating the enduring utility of these foundational methods in NLP.

Classic Word Embedding Text Representation Methods

Word Embeddings

Word embeddings revolutionized NLP by representing words as dense vectors in a continuous vector space, where semantically similar words are closer to each other. Unlike Bag-of-Words and TF-IDF, embeddings capture the semantic meaning of words and their contextual relationships. Common traditional word embedding models include Word2Vec [61], GloVe [62], and FastText [71].

Word2Vec

Word2Vec, introduced by Mikolov et al. (2013), revolutionized how word meanings are represented in natural language processing by learning dense word embeddings through a shallow neural network (just 2-layers). Word2Vec has two key models: Skip-gram and Continuous Bag of Words (CBOW). The Skip-gram model predicts the context words (words surrounding a target word) given the target word, making it suitable for smaller datasets. CBOW, on the other hand, predicts the target word given its context, which is more effective for larger datasets. Both models learn word relationships by analyzing large amounts of text and placing words in a high-dimensional vector space where similar words are close together. To enhance computational efficiency, Word2Vec employs techniques like negative sampling, which simplifies the training process by focusing on a subset of relevant words.

GloVe

GloVe (Global Vectors for Word Representation), introduced by Pennington et al. (2014), is a word embedding technique that builds on the strengths of Word2Vec but emphasizes capturing the global statistical relationships between words in a corpus. Unlike Word2Vec, which focuses on learning local context by predicting nearby words, GloVe constructs a word co-occurrence matrix that summarizes how frequently words appear together in a large dataset. Using matrix factorization, GloVe derives word vectors that encode both local and global context. This allows GloVe to efficiently learn embeddings for large-scale corpora while capturing semantic relationships across the entire dataset. For example, GloVe can represent analogies like “man is to king as woman is to queen” by modeling the relationships between words mathematically.

FastText

FastText, developed by Facebook’s AI Research lab, extends the capabilities of Word2Vec by incorporating subword information into word embeddings. Unlike traditional methods that treat each word as a discrete unit, FastText breaks words into character n-grams, allowing it to better capture word morphology and meaning, particularly for rare or out-of-vocabulary words. This feature is especially useful for languages with complex forms or when dealing with small datasets where rare words might not have sufficient representation. By learning from the internal structure of words, FastText can generalize better to unseen words or those with multiple forms (such as plurals or conjugations). FastText retains computational efficiency similar to Word2Vec and is highly scalable to large datasets.

Current Relevance and Use

While Word2Vec, GloVe, and FastText improved on earlier methods by encoding semantic relationships, they are limited by their inability to capture multiple meanings of a word based on context (e.g. the word ‘bank’ could mean a river bank or a financial institution). Nevertheless, these embeddings laid the groundwork

for more advanced models and continue to be widely used today for feature extraction and as initial layers in neural networks. In particular, Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and other sequential models often rely on pre-trained embeddings as input. These embeddings transform input words into dense vector representations, enabling models to process language more efficiently. Depending on the task, embeddings can either be pre-trained on large corpora like Word2Vec or GloVe [72], or fine-tuned on specific datasets to capture domain-specific relationships. This integration of embeddings with sequence models has been a common practice in various NLP tasks like machine translation, text classification, and sentiment analysis; however, RNNs and LSTMs struggle to capture long-range dependencies in sentences due to issues like vanishing gradients, making it hard to retain context for longer sequences [63]. Although LSTMs improved over standard RNNs with mechanisms like gating, they still faced challenges when dealing with very long sequences. This limitation paved the way for attention-based models, which can directly attend to all words in a sequence, irrespective of the distance between them, thereby overcoming the weaknesses of RNN-based architectures and leading to the development of the Transformer model.

3.5.2. Transformer Revolution: From Attention to BERT and BERT Variants

While traditional methods like Bag-of-Words, TF-IDF, and early word embeddings such as Word2Vec, GloVe, and FastText laid the foundation for representing textual data, they come with significant limitations. These techniques, particularly BoW and TF-IDF, produce static and context-independent word representations that fail to capture the nuances of word meaning in different contexts. Even advanced word embeddings like Word2Vec and GloVe, while providing richer representations, still offer a single embedding for each word, regardless of its meaning in different sentences. As the complexity of natural language understanding tasks grew, so did the need for models that could better handle the contextual dependencies of words. This led to the development of more sophisticated architectures, culminating in the Transformer model, which introduced self-attention mechanisms capable of dynamically adjusting word representations based on their context within a sentence. The introduction of the Transformer revolutionized NLP, moving beyond the limitations of static embeddings to context-aware models that better capture the relationships between words over long distances. In the next section, we explore the advent of the Transformer architecture and how it marked a turning point in NLP, leading to powerful models like BERT and its variants, which continue to push the boundaries of language understanding.

Attention Mechanism Background

Before the breakthrough of Transformer models, the attention mechanism was introduced as a key innovation to address the limitations of sequence-to-sequence (seq2seq) models in handling long-range dependencies. In seq2seq models, such as those used in neural machine translation (NMT), the encoder compresses an entire input sequence into a single fixed-length context vector, which the decoder uses to generate output. This bottleneck limits the model's capacity to handle long or complex sentences effectively (due to memory constraints), as crucial information could be lost. The attention mechanism, used alongside recurrent or convolutional neural networks, solves this by allowing the model to dynamically focus on specific parts of the input sequence during decoding. Rather than relying on a fixed context vector, attention calculates alignment scores between the current output word and all input words. This creates a weighted sum of the input, where relevant words receive higher attention weights. This innovation greatly improved tasks like translation, question answering, and summarization by enabling the model to "attend" to the most relevant words in a sentence [73, 74]. This early version of attention set the stage for a more advanced form of attention—the self-attention mechanism, which would be introduced in the Transformer architecture.

The Transformer Architecture

In their influential paper, *Attention is All You Need*, Vaswani et al. [64] introduced the Transformer model, the first architecture to rely entirely on self-attention mechanisms—sometimes referred to as intra-attention, which computes a representation of a sequence by relating different positions within that sequence—for both encoding and decoding. This mechanism allows the model to weigh the importance of each word or token relative to others, capturing dependencies and relationships across the entire sequence. Unlike previous models, which relied on sequence-aligned RNNs or convolutions, the Transformer processes input data in parallel, significantly boosting training speeds and making it more efficient for handling long-range dependencies in language tasks. As with most sequence-to-sequence models, the Transformer follows an encoder-decoder architecture, utilizing stacked self-attention layers and point-wise, fully connected layers for both the encoder

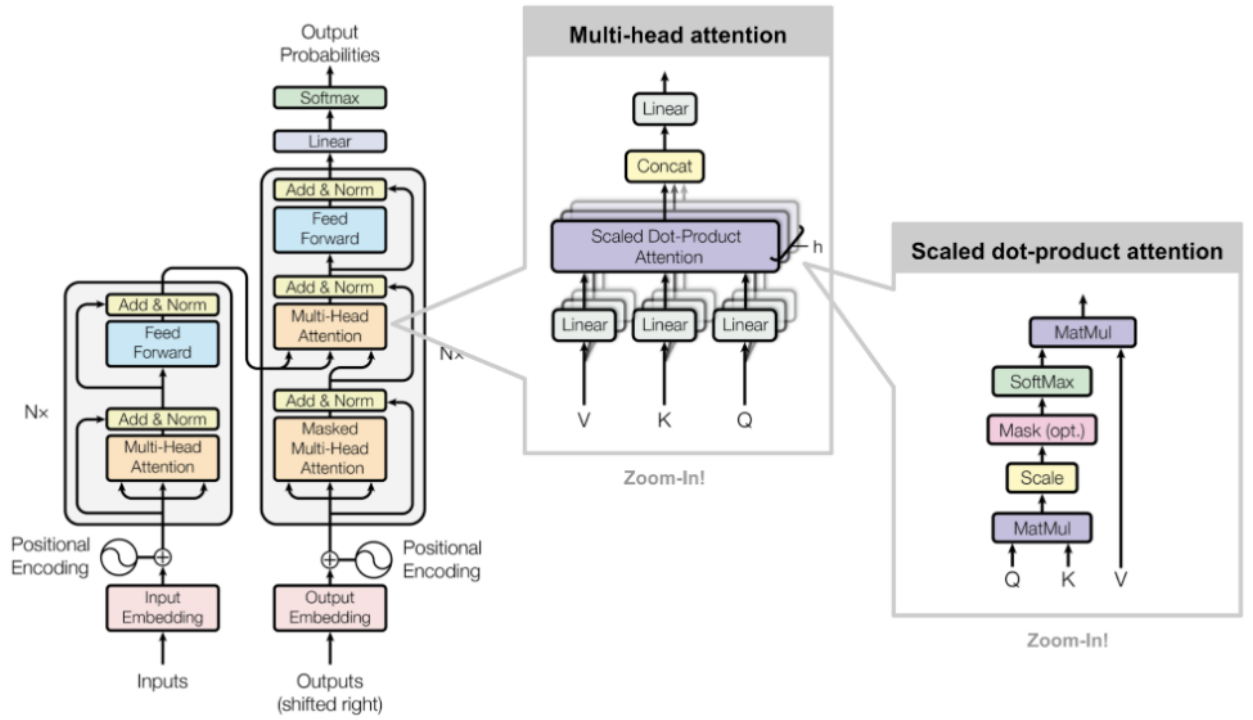


Figure 3: The full model architecture of the transformer; adapted from [64, 75]

and decoder, as illustrated in Figure 3. To aid in understanding, you can explore the model visually at [this link](#), but below, we'll describe the process in plain language.

Just like the name "encoder-decoder" suggests, the Transformer model has an **Encoding component** and a **Decoding component**. For example, in a translation task, an input (e.g., a sentence in French) is processed by the encoder, then passed to the decoder to generate the output (e.g., the translation in English). The encoding component is composed of 6 identical encoders, each having two sub-components: a **multi-head self-attention mechanism** and a **fully connected feed-forward network**. All 6 encoders are structured the same way but have different learned weights. Here's the flow for each encoder:

- **Self-Attention Layer:** Inputs first pass through a self-attention layer (we will talk about this more in depth this later on in this section).
- **Position-wise Feed-Forward Neural Network (FFNN):** The outputs of that self-attention layer become the inputs of the feed forward NN. The same feed-forward network is separately and identically applied to each input (input means each word in a sentence). This consists of two linear transformations with a ReLU activation in between.
- **Layer Normalization:** After each of the sub-components (self-attention and feed-forward layers), layer normalization is applied to stabilize and accelerate training [76].
- **Residual Connections:** A residual connection is applied around each sub-layer (self-attention and feed-forward) to help preserve the original input information and prevent the degradation of the gradient during training, especially for deep networks [77]. This is done by adding the input to the output of the sub-layer:

$$\text{LayerNorm}(x + \text{Sublayer}(x)) \quad (81)$$

where x is the input to the sub-layer, and $\text{Sublayer}(x)$ is the output of the sub-layer (either from the self-attention or the feed-forward network). The residual connection ensures that the input signal flows through the network without vanishing or exploding gradients, improving stability in deep architectures.

- **Output Dimensions:** Due to these residual connections, all sub-components (including the embedding layers) produce outputs with the same dimension, ensuring that the model maintains consistent dimensionality throughout. In the original Transformer, this dimension is set to $d_{\text{model}} = 512$.

The decoder component has a similar structure to the encoder, consisting of 6 identical decoders. Each decoder follows the same process as the encoder, but with one key difference: in addition to the self-attention,

feed-forward layers, and residual connections, the decoder includes a third sub-component—**encoder-decoder attention**. This attention layer helps the decoder focus on relevant parts of the input sequence by attending to the encoder's output. This ensures that information from the encoder is effectively utilized during the decoding process and the decoder generates predictions based on both the previously generated outputs and the encoded input sequence:

- **Masked Self-Attention Layer:** Similar to the encoder, the decoder applies self-attention. However, it masks future positions to prevent the decoder from "cheating" by looking ahead at future words when predicting the next word in the output sequence.
- **Encoder-Decoder Attention Layer:** This layer uses the encoder's final output, which is transformed into a set of attention vectors K and V , allowing the decoder to attend to specific parts of the input sequence. This ensures the decoder focuses on the most relevant parts of the source sentence when generating its predictions.

The **Self-Attention Layer** in the Transformer enables each word in a sentence to incorporate information from other words when being processed. For instance, in the sentence "I did not like that dog because it was mean," the word "it" refers to "dog." While this is intuitive to humans, machines require complex computation to capture this relationship. Self-attention assigns different weights to each word, determining how much focus each word should receive based on its relevance to the word being processed. This mechanism ensures that the context of surrounding words is captured, resulting in more contextually-aware word representations.

Step 1: Creating Query, Key, and Value Vectors: For each word in a sentence, three vectors are created: a query vector, a key vector, and a value vector. These are derived by multiplying the word's embedding with three weight matrices (W^Q , W^K , and W^V), which are learned during training. For example, the word "it" is projected into query, key, and value vectors (q_1 , k_1 , and v_1).

Step 2: Calculating Scores: The model calculates a score to determine how much focus to place on other words in the sentence. This score is the dot product of the query vector with each key vector in the sequence. For example, the score for "it" is calculated by multiplying q_1 with k_1 , k_2 , and so on for the other words in the sequence.

Step 3: Scaling and SoftMax: The scores are divided by the square root of the dimensionality of the key vectors (in this case, $\sqrt{64} = 8$), a technique that prevents the dot products from becoming too large, which would make the softmax function output have extremely small gradients (i.e. ensures more stable training). These scaled scores are then passed through a SoftMax function to normalize them, converting the scores into probabilities that sum to 1.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (82)$$

Where Q is the query vector, K is the key vector, and V is the value vector. d_k is the dimension of the key vectors.

Step 4: Weighting Value Vectors: The SoftMax scores are used to weight the value vectors for each word. Words that are more relevant to the current word receive higher weights, while less relevant words are downweighted.

Step 5: Summing Weighted Value Vectors: The weighted value vectors are summed to produce the output for the **word being processed**. This new vector represents the word's contextualized encoding, taking into account the relationships with other words in the sentence.

This process is repeated for each word in the sentence, enabling the model to generate rich, context-aware vector representations for the entire sentence (see Appendix Section A.4 for a simple example of this process).

One limitation of using a single attention head is that it might focus too much on specific parts of the sentence. To address this, the Transformer uses **multi-head attention**. Instead of running the self-attention mechanism once, it runs it multiple times in parallel (e.g., 8 times), each with a different set of learned weights. Thus, instead of one set of the three weight matrices (W^Q , W^K , and W^V) with their corresponding calculations, there are 8 sets of these matrices for each self-attention layer in both the encoder and the decoder, as well as for the "encoder-decoder attention" layers in the decoder. The outputs from each of the attention heads are concatenated and then passed through an additional linear layer with its own weights matrix W^O . This allows the model to capture different types of relationships and patterns between words in parallel, ultimately enhancing the richness and quality of the word representations.

As for how the inputs work, each word in the input sequence is embedded into a fixed-size vector of 512 dimensions. These vectors are called word embeddings, and in the Transformer, they are **learned embeddings**, meaning that the model learns the optimal representation of each word during training. For example, in the sentence "I like food", each word is represented as a vector of 512 numbers. These vectors capture various aspects of the word, and are crucial for enabling the model to work with numerical data rather than raw text. Each word embedding serves as the input to the first encoder in the Transformer model. As the input sequence flows into the encoder, the embeddings are processed separately, meaning that the self-attention and feed-forward layers process each word one by one. Although each word is processed individually at first, the self-attention mechanism ensures that information from the entire sentence is incorporated into each word's representation, enriching the model's understanding of how the words in the sentence relate to one another. In addition, unlike RNNs and LSTMs, which have an inherent understanding of word order through sequential processing, the Transformer processes input tokens in parallel. To ensure that the model captures the order of the words in a sentence, **positional encodings** are added to the word embeddings. These encodings inject information about the relative or absolute position of tokens in the sequence, allowing the model to account for word order. The positional encodings have the same dimensionality as the word embeddings, $d_{model} = 512$, so that the two can be summed together. The Transformer paper uses sinusoidal functions to generate these encodings. The authors hypothesized that sinusoidal encodings would allow the model to generalize to longer sequences than it encountered during training because the sinusoidal pattern provides a continuous and smooth representation of positions. The positional encodings are defined as follows:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (83)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (84)$$

The pos symbol is the position of the token (word) in the sequence (sentence), and i is the dimension of the embedding. Each dimension of the positional encoding corresponds to a curve with the form of a sine wave (sinusoid). The function alternates between sine for even dimensions and cosine for odd dimensions, adding variation to the positional signals and improving the model's ability to capture relative positions. In simpler terms, the sinusoidal encodings create continuous wave patterns across each dimension, allowing the model to distinguish word order and handle sequences of varying lengths.

Finally, once the decoder stack has processed the input sequence and generated its final output (a vector of floats), the goal is to turn this output into an actual word from the model's learned vocabulary. This is achieved through the **Final Linear Layer** and **Softmax Layer**. The Linear Layer is a fully connected neural network layer that projects the decoder output into a much larger vector called the logits vector. The size of this logits vector corresponds to the size of the model's output vocabulary. For instance, if the model has learned 25,000 unique words, the logits vector will have 25,000 elements, where each element represents a score for a specific word in the vocabulary. After producing the logits vector, it is passed through a Softmax Layer, which converts the logits into probabilities. The softmax function ensures that the scores are normalized to sum to 1, turning them into probabilities. The word associated with the highest probability is selected as the model's prediction for that particular time step.

The training process of the Transformer involves both an encoder and decoder, working together to translate an input sentence (e.g., French) into a target sentence (e.g., English). Here is the step-by-step process:

- **Input Sentence (French):** The process begins with the input sentence in French, for example, "*Je suis étudiant*". This sentence is passed through the encoder.
- **Encoding the Input:** The encoder processes the entire French sentence, passing it through multiple layers of self-attention and feed-forward networks. The result is a set of encoded representations, which capture the context and meaning of the French sentence but do not yet represent the English translation.
- **Decoder Input (English):** The decoder takes two inputs: (1) the encoded representation from the encoder, and (2) a partially completed English sentence (for training purposes, this is the correct translation). For example, if the target translation is "*I am a student*", the decoder is initially given the word "*I*". This step is known as "teacher forcing," where the correct English word is provided during training to help the model learn.

- **Prediction (English):** The decoder then combines the encoded French sentence with the partial English sentence to predict the next word in the sequence. For instance, it may predict *"am"* as the next word following *"I"*. This prediction is made by sampling from a list of possible English words (i.e., the target vocabulary), and the word with the highest likelihood is selected.
- **Sequence of Predictions:** This process repeats for each word in the sentence. After predicting *"am"*, the decoder is given *"I am"* and asked to predict the next word, such as *"a"*, and so on, until the entire sentence *"I am a student"* is generated.
- **Feedback and Weight Updates:** During training, the model's predictions are compared to the correct target words for an entire batch of sentences. To avoid overfitting and encourage better generalization, **label smoothing** is applied. Instead of assigning a probability of 1 to the correct target word and 0 to all others, label smoothing distributes a small portion of the probability mass across incorrect labels⁵. For instance, if the smoothing factor is 0.1, 90% of the probability is assigned to the correct word, and 10% is spread among the incorrect ones. This helps prevent the model from becoming overly confident and improves its performance on unseen data. Afterward, the model updates its internal parameters (weights) to minimize future errors across the entire batch.
- **Inference (Post-Training):** During inference (i.e., when the model is used in practice), the model no longer has access to the correct target sentence. Instead, it relies on its own previous predictions as input for generating the next word in the sequence, without teacher forcing⁶.

In summary, the training process involves encoding the input sentence into contextual representations, decoding one word at a time, and predicting the next word in the target language by sampling from a target vocabulary. The model continuously updates its weights based on feedback, improving the accuracy of predictions over time.

BERT and its Variants

With an understanding of the Transformer architecture and the key role that attention mechanisms play in sequence processing, we now turn to one of the most impactful applications of the Transformer framework: BERT. BERT (Bidirectional Encoder Representations from Transformers) [65] revolutionized natural language understanding by leveraging bidirectional training of Transformer encoders and novel pre-training objectives. In the following sections, we will explore BERT's foundational methods, as well as introduce several variants of BERT that have further refined its capabilities.

BERT (Bidirectional Encoder Representations from Transformers)

BERT, introduced by Devlin et al. in 2018, revolutionized the field of NLP by introducing a method for pre-training deep bidirectional representations from unlabeled text. Unlike unidirectional models, BERT is bidirectional, meaning it captures both past and future contexts at every layer of its Transformer encoder. This bidirectionality allows BERT to generate rich, **high-quality contextualized word embeddings**, understanding their meanings based on surrounding words from both directions. While BERT shares several similarities with the Transformer architecture described in Section 3.5.2—such as the use of multi-layered Transformer encoders, positional encodings, and the self-attention mechanism—there are key differences. BERT is an **encoder-only** model, whereas the original Transformer architecture includes both an encoder and a decoder for sequence generation tasks. Since BERT's goal is representation learning rather than sequence generation, the decoder component is absent. BERT also introduces several key innovations that were not part of the original Transformer setup, including **Masked Language Modeling (MLM)**, and **Next Sentence Prediction (NSP)**. One of BERT's most significant contributions is its ability to be pre-trained on large amounts of text and later fine-tuned on specific downstream tasks with relatively small amounts of labeled data. This approach allows BERT to be easily adapted to tasks such as text classification, question answering, and named entity recognition with just a limited amount of task-specific data. As a result, BERT significantly reduces the data requirements for fine-tuning, making it a highly versatile and efficient model for many NLP applications. BERT originally came in two main variants: **BERT_{BASE}** and **BERT_{LARGE}**. BERT_{BASE} has 12 encoder layers (Transformer blocks), each with 12 attention heads, and a hidden size of 768. BERT_{LARGE}, on the other hand,

⁵The role of label smoothing in stabilizing probability distribution updates during training is conceptually similar to the use of KL divergence in reinforcement learning, where KL divergence helps control the divergence between large policy updates, ensuring smoother learning.

⁶During inference, the authors actually implement beam search to explore multiple top translations instead of just one, then selected whichever produced less error.

significantly increases model capacity with 24 encoder layers, 16 attention heads per layer, and a hidden size of 1024. These configurations make BERT_{LARGE} more powerful but also more computationally expensive compared to BERT_{BASE}, providing flexibility for different resource and performance trade-offs.

The original Transformer architecture is designed for tasks like translation or language generation, where predicting the next word in a sequence is done in a unidirectional, left-to-right manner. BERT, in contrast, performs bidirectional encoding of text, allowing it to build representations that account for both the left and right contexts. We illustrate this difference with the following simple example:

- **Transformer (Unidirectional, Left-to-Right Prediction):**
 - Input: *I am a _.*
 - Task: Predict the next word based on the previous context.
 - Prediction: *student.*
- **BERT (Bidirectional, Masked Language Modeling):**
 - Input: *I [MASK] a student.*
 - Task: Predict the masked word using both left and right contexts.
 - Prediction: *am.*

BERT introduces two novel pre-training objectives to improve upon the bidirectional representation learning: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). These tasks allow the model to better capture word-level and sentence-level semantics, making it highly effective for downstream NLP tasks.

Masked Language Modeling (MLM) Procedure

Masked Language Modeling (MLM) is a key component of BERT's pre-training that allows it to learn bidirectional representations by randomly masking some tokens in the input sequence and then training the model to predict these masked tokens. During pre-training, 15% of the tokens in each input sequence are selected for masking, but they are not always directly replaced by a [MASK] token. Instead, the masking procedure is as follows:

- 80% of the time, the selected token is replaced with the [MASK] token.
- 10% of the time, the selected token is replaced with a random token.
- 10% of the time, the selected token is left unchanged.

This variation forces the model to create robust contextual representations, as it needs to handle masked, random, and unchanged tokens effectively.

Example Consider the following sentence: *The man went to the store to buy some milk.*

- Input with MLM: *The man went to the [MASK] to buy some [MASK].*
- Target output: *store, milk.*

BERT processes this sentence using bidirectional attention, predicting the masked words (*store, milk*) based on the entire context of both preceding and following words. This kind of prediction is not possible with unidirectional models, which can only rely on past tokens. The model's predictions for the masked tokens are generated by applying a classification layer to the output of the encoder stack. A softmax function is then used to calculate the probabilities of each token in the vocabulary, and the token with the highest probability is chosen as the prediction.

MLM equips BERT with the ability to utilize both prior and future tokens when predicting missing words, enhancing the model's performance in tasks requiring a deep understanding of contextual information. However, this approach slows down convergence compared to left-to-right models because of the need to mask and reconstruct words in the sequence. One significant advantage of MLM is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words. This uncertainty forces the model to maintain a robust contextual representation of *every* token in the input sequence. This approach ensures that the model doesn't overly rely on specific tokens but instead learns a broader understanding of context for all words in the sequence. Additionally, only a small fraction (15%) of tokens are masked, and of those, only a portion (80%) are directly replaced with the [MASK] token. The remaining tokens are either left unchanged or replaced with random tokens. This technique encourages the model to generalize better across varying contexts and helps avoid overfitting to a particular training condition. As a result, MLM provides more robust and context-aware language representations that are crucial for downstream NLP tasks.

Next Sentence Prediction (NSP) Procedure

Next Sentence Prediction (NSP) is designed to help BERT capture relationships between sentences, which is particularly useful for tasks like question answering and natural language inference. The NSP task teaches the model to understand whether two sentences in a pair follow each other or are randomly sampled. During pre-training, BERT receives pairs of sentences as input:

- 50% of the time, the second sentence is the actual sentence that follows the first one in the text.
- 50% of the time, the second sentence is a random sentence sampled from the corpus.

The model is trained to predict whether the second sentence logically follows the first or is a random sentence.

Example

- **Sentence A:** The man went to the store.
- **Sentence B:** He bought a gallon of milk.
- **Label:** *IsNext*.

For a randomly sampled sentence:

- **Sentence A:** The man went to the store.
- **Sentence B:** Penguins are flightless birds.
- **Label:** *NotNext*.

The input sequence for NSP includes a special [CLS] token at the beginning of the first sentence and a [SEP] token separating the two sentences. The [CLS] token is used to predict whether Sentence B logically follows Sentence A, using a classification layer applied to the encoder's output of the [CLS] token. The output is a binary label (*IsNext* or *NotNext*).

Combined Loss Function

The masked language model (MLM) loss is calculated by comparing the predicted token probabilities, `log_probs`, with the actual masked token labels, `one_hot_labels`. Specifically, each prediction's loss is scaled by label weights, averaged over the real predictions to handle padding, and divided by the sum of weights to yield the final MLM loss:

$$\text{MLM Loss} = \frac{\sum(\text{label_weights} \times \text{per_example_loss})}{\sum(\text{label_weights}) + \epsilon}. \quad (85)$$

The next sentence prediction (NSP) loss is computed as a binary classification task, where the log probabilities for each sentence pair are compared to the actual labels, `one_hot_labels`, and averaged across examples to yield the final NSP loss:

$$\text{NSP Loss} = \frac{1}{N} \sum_{i=1}^N - \sum (\text{one_hot_labels}_i \cdot \log_probs_i). \quad (86)$$

The overall loss function, Total Loss, used to train BERT is the sum of the masked language model and next sentence prediction losses:

$$\text{Total Loss} = \text{MLM Loss} + \text{NSP Loss}. \quad (87)$$

BERT jointly trains MLM and NSP by minimizing the sum of the mean MLM likelihood and the mean NSP likelihood. This allows the model to simultaneously learn to predict masked tokens and sentence relationships, making it highly versatile for various NLP tasks. The joint optimization strategy enhances the model's ability to generalize well across different types of downstream tasks, as demonstrated in numerous fine-tuning applications.

Full Pre-training Procedure

BERT's pre-training follows a two-step process. First, the input sequences are sampled from a large corpus of text. Each sequence consists of two segments: Sentence A and Sentence B. The task is to predict whether Sentence B logically follows Sentence A in the original text (Next Sentence Prediction, NSP). Sentence A and B are packed into a single sequence, separated by a special token [SEP]. The input sequence is tokenized using WordPiece embeddings, which allow BERT to handle rare and subword units effectively by breaking down words into smaller components. BERT uses a vocabulary of 30,000 tokens built from the corpus using this WordPiece tokenization strategy [78]. Additionally, a classification token [CLS] is added at the beginning of each sequence, and segment embeddings (A or B) are added to differentiate between the two segments. The Masked Language Modeling (MLM) task complements NSP, where 15% of tokens in the input sequence are randomly selected for prediction. Of those selected, 80% are replaced with the [MASK] token, 10% with a random token, and 10% remain unchanged. This combination encourages BERT to develop a robust contextual representation of every token in the sequence, as the model cannot rely solely on the presence of the [MASK] token for predictions.

Pre-training Data

BERT was pre-trained on two large-scale corpora: the BooksCorpus, consisting of 800 million words, and the English Wikipedia, containing approximately 2,500 million words. Wikipedia texts were cleaned to remove tables, lists, and headers, and only the main textual content was used. BERT's training focused on longer, contiguous sequences rather than shuffled sentence-level corpora, allowing it to better capture document-level coherence, which is crucial for downstream tasks.

Vocabulary and MLM Sampling

The WordPiece vocabulary consists of 30,000 tokens generated from the training corpus. This strategy enables BERT to handle both common and rare words by splitting infrequent words into subword units. For example, a rare word like "unaffordable" may be split into "un," "afford," and "able." In MLM, predictions are made for each masked token by sampling from this vocabulary. The model must use the surrounding context to infer which token fits the masked position, leading to stronger language understanding abilities.

Fine-tuning Process

After pre-training, BERT can be fine-tuned for specific downstream tasks such as question answering, text classification, or sentiment analysis. Fine-tuning is relatively simple and cost-effective because BERT retains the general language understanding learned during pre-training. During fine-tuning, task-specific data is fed into BERT, and all parameters are fine-tuned end-to-end. For example, in text classification, the output of the [CLS] token is used as the aggregate representation of the sequence and passed through a task-specific classification layer. The model can adapt quickly to new tasks because the fine-tuning procedure usually involves fewer steps compared to pre-training, which leverages massive unlabeled datasets.

In conclusion, BERT's two-phase approach—pre-training on vast amounts of text data using self-supervised objectives followed by task-specific fine-tuning—has made it a versatile model for a wide range of NLP applications, significantly reducing the need for large amounts of task-specific labeled data. BERT's success stems from its ability to generate high-quality contextualized word embeddings, making it a valuable tool in modern NLP.

BERT Variants

BERT's pre-training and fine-tuning framework has significantly advanced the field of NLP by providing robust, high-quality contextualized word embeddings that can be easily adapted to a variety of downstream tasks. However, as with any model, there is always room for improvement, and numerous variants of BERT have emerged, each focusing on enhancing specific components of the model or addressing its limitations. BERT variants typically seek to optimize different aspects of the model such as training efficiency, model size, or the use of additional objectives during pre-training. In this section, we will explore some of the most popular BERT variants, organized based on their primary area of innovation, including model efficiency, long-sequence handling, and enhanced representation capabilities.

Model Size Reduction and Parameter Efficiency: DistilBERT and ALBERT

DistilBERT [79] introduces a lighter and faster variant of BERT, designed to make the model more accessible

in terms of computational efficiency. DistilBERT is created using *knowledge distillation* [80, 81], a process where a smaller model (the *student*) learns to mimic the behavior of a larger model (the *teacher*), which in this case is BERT. The goal is for the student model to capture essential patterns and knowledge from the teacher, allowing it to perform similarly on downstream tasks despite its reduced size. The distillation process trains the student model with a combination of losses (i.e. triple loss combining distillation, language modeling, and cosine-distance losses) that encourage it to match the teacher model’s predictions while also learning general language patterns:

- **Distillation Loss** (L_{ce}): This loss minimizes the cross-entropy between the soft probability distribution predicted by the teacher and the student’s predictions. A temperature parameter T is applied in the softmax function to smooth the teacher’s probability distribution:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (88)$$

where z_i is the teacher’s output for class i . A higher temperature T results in a smoother probability distribution, which provides richer information to the student about class similarities. The cross-entropy distillation loss between the teacher and student’s softened distributions is then given by:

$$L_{ce} = - \sum_i p_i \log(s_i) \quad (89)$$

where p_i is the probability from the teacher model and s_i is the probability predicted by the student model for class i .

- **Masked Language Modeling Loss** (L_{mlm}): Similar to BERT, DistilBERT includes a masked language modeling (MLM) loss, where tokens are masked and the model predicts these tokens based on the context. This objective enhances the model’s language understanding.
- **Cosine Embedding Loss** (L_{cos}): DistilBERT introduces a cosine embedding loss to align the hidden state representations of the student and teacher models. This loss encourages the student model’s hidden states to be closer to those of the teacher, which further improves its language understanding.

DistilBERT also incorporates structural modifications to further enhance efficiency compared to BERT. For example, it has only 6 transformer layers, in contrast to the 12 layers of BERT-base. Instead of reducing the hidden size, which would significantly impact performance, the number of layers is halved, preserving representational capacity and reducing computational cost. In addition, the token-type embeddings and pooler layer from BERT are removed in DistilBERT, which simplifies the model and reduces computational overhead. DistilBERT was trained on the same dataset as BERT (English Wikipedia and the Toronto Book Corpus), but several optimizations made the training more efficient; such as large batch sizes with gradient accumulation and dynamic masking (each input sequence saw different masking patterns) without the next sentence prediction objective. With 40% fewer parameters and 60% faster inference times than BERT, DistilBERT is optimized for resource-constrained environments, including mobile devices and on-device applications. Despite the reduction in parameters, it retains approximately 97% of BERT’s performance on the GLUE benchmark and performs comparably on other NLP tasks, such as sentiment analysis on IMDB and question answering on SQuAD. For an example calculation of DistilBERT’s loss components, please refer to the Appendix (Section A.5).

ALBERT (A Lite BERT) [82] addresses the issue of BERT’s increasing parameter size, which can lead to memory inefficiencies. ALBERT reduces the number of parameters by sharing weights across layers and using factorized embeddings. This design makes ALBERT highly parameter-efficient, while still achieving comparable or superior performance to BERT on many NLP benchmarks. ALBERT’s modifications make it a powerful alternative for applications requiring efficient memory usage without sacrificing model capacity.

ALBERT introduces two main innovations to reduce model parameters and enhance efficiency:

- **Factorized Embedding Parameterization:** In BERT, the vocabulary embedding size is tied to the hidden layer size, which leads to high memory requirements. ALBERT decouples these by using a smaller embedding dimension E , which is then projected to the larger hidden size H . This two-step embedding process reduces the parameter count from $O(V \times H)$ to $O(V \times E + E \times H)$, where V is the vocabulary size. This setup allows ALBERT to maintain expressiveness with far fewer parameters.
- **Cross-Layer Parameter Sharing:** ALBERT employs parameter sharing across layers, meaning that instead of having separate parameters for each layer, it reuses the same parameters across all layers.

This technique prevents the model's parameter size from scaling with the number of layers, thereby reducing redundancy and computational cost. ALBERT's default configuration shares all parameters across layers, but variations exist that only share either feed-forward or attention parameters, each with trade-offs for performance and efficiency.

Additionally, ALBERT replaces BERT's Next Sentence Prediction (NSP) task with a **Sentence Order Prediction** (SOP) task. Unlike NSP, which attempts to predict if two sentences are consecutive, SOP predicts whether two consecutive sentences are in the correct order or swapped. This change provides a clearer signal on discourse coherence, improving ALBERT's performance on tasks involving multi-sentence understanding. Thanks to these architectural optimizations, ALBERT configurations achieve substantial parameter reduction. For example, ALBERT-large has 18 million parameters, compared to BERT-large's 334 million. This results in faster training times (about 1.7 times faster) and reduced memory usage, allowing ALBERT to perform effectively on resource-constrained hardware without significant performance loss on NLP benchmarks such as GLUE, SQuAD, and RACE. ALBERT models demonstrate that efficient design choices can mitigate the computational challenges associated with scaling, offering a versatile solution for both research and practical applications in NLP.

Enhanced Contextual Understanding for Long-Form Texts: Longformer and BigBird

Longformer [83] addresses the limitations of traditional transformer models in processing long sequences due to memory and computational inefficiencies. Transformer-based models, such as BERT, face challenges with self-attention, which scales quadratically with sequence length. This makes it infeasible for tasks requiring long documents to be processed in a single pass. Longformer modifies the self-attention mechanism to scale linearly with sequence length, thus enabling the processing of thousands of tokens without the need to truncate or split the document. Longformer introduces three key innovations in its attention mechanism, similar to techniques used in Convolutional Neural Networks (CNNs):

- **Sliding Window Local Attention:** Each token attends to its neighboring tokens within a fixed-size window, allowing the model to capture local context efficiently. By stacking multiple layers of this windowed attention, Longformer expands its receptive field progressively, allowing higher layers to incorporate information from the entire input. The complexity of this attention is $O(n \times w)$, where n is the sequence length and w is the window size, making it significantly more efficient for longer sequences.
- **Dilated Sliding Window Attention:** To further increase the receptive field without adding to computational cost, Longformer employs a dilated sliding window approach. In this setup, tokens attend to others within a window but with gaps (or dilations) between them. This enables the model to capture a wider range of contextual information across layers while keeping the computation cost similar to standard local attention.
- **Global Attention for Task-Specific Tokens:** To handle tasks where certain tokens require a broader context, Longformer includes a small number of globally attended tokens. These tokens can attend to all tokens across the sequence and vice versa, allowing the model to aggregate global information where needed. For example, in classification tasks, the [CLS] token receives global attention, and in question-answering tasks, global attention is given to question tokens, facilitating a full-sequence understanding.

This combination of local, dilated, and global attention allows Longformer to balance efficiency with representational capacity, capturing both fine-grained local details and high-level global structures. This design enables Longformer to achieve state-of-the-art results on tasks involving long documents, such as WikiHop and TriviaQA. Additionally, Longformer can serve as a drop-in replacement for self-attention in pre-trained transformers, allowing efficient fine-tuning on long-document tasks.

BigBird [84] introduces an innovative sparse attention mechanism that addresses the computational inefficiencies associated with standard Transformers, particularly for tasks involving long sequences. By replacing quadratic complexity with linear complexity in attention operations, BigBird enables processing of sequences significantly longer than traditional models, such as BERT, while preserving strong theoretical and practical performance. BigBird, like many other models designed for long sequences, approximates the full attention matrix while balancing computational efficiency with performance. The key innovations of BigBird's attention mechanism are as follows:

- **Sliding Window Attention:** Each token attends to a fixed number of w local tokens within its immediate neighborhood. Specifically, for a query token at position i , attention is computed over the $\frac{w}{2}$ tokens preceding it and the $\frac{w}{2}$ tokens following it. This sliding window ensures that local dependencies are effectively captured, which is essential for most natural language tasks as semantic relationships often exist within close proximity in text. For example, in the sentence "The stock market is highly volatile," words like "highly" and "volatile" would attend to each other due to their close proximity semantic relationship.
- **Random Attention:** To complement local attention and capture long-range dependencies, BigBird randomly selects r tokens from the sequence for each query token to attend to. This mechanism provides an efficient way to propagate information across distant tokens in the sequence. For instance, in a document summarization task, tokens at the beginning of the document may need to relate to tokens near the end. Random attention approximates dense attention by forming indirect connections between such distant tokens.
- **Global Attention:** Certain key tokens, such as the classification token ([CLS]), are designated as global tokens that both attend to all tokens in the sequence and are attended to by all tokens. This setup allows global tokens to act as information hubs, facilitating the aggregation of sequence-wide information. For instance, in a question-answering task, the [CLS] token could attend to both the question and relevant parts of the context, serving as a central mechanism for extracting the answer.

BigBird combines these attention mechanisms into a sparse attention pattern that achieves linear complexity while retaining the core advantages of full self-attention, including its theoretical guarantees. One of BigBird's key architectural designs is its use of **block sparse attention**. This approach partitions the sequence into blocks and applies attention patterns within and across these blocks. Specifically, sliding window attention operates within a block, random attention connects distant blocks, and global attention spans the entire sequence. This block sparse design ensures that the model maintains strong connectivity across the sequence while significantly reducing computational overhead. Additionally, BigBird introduces two different strategies that can be used:

- **Internal Transformer Construction (ITC):** In this configuration, a subset of tokens from the input is designated as global tokens. These tokens are pre-determined and fixed based on their importance to the task (e.g., [CLS]).
- **Extended Transformer Construction (ETC):** This configuration introduces additional global tokens, such as special task-specific markers, and ensures that these tokens attend to all input tokens. ETC is particularly useful for tasks like question answering, where the question tokens serve as global tokens and attend to the entire context.

Through a combination of theoretical analysis and empirical validation, BigBird demonstrates the efficacy of its sparse attention mechanism in approximating dense attention. Notably, BigBird achieves state-of-the-art results on a wide range of tasks, including document summarization and question answering, and extends its applicability to non-NLP domains like genomics, where sequences are naturally long. The model's ability to process sequences of up to 4,096 tokens with computational efficiency and strong performance makes it a groundbreaking advancement in transformer architecture.

Refined Representation and Contextual Nuance: RoBERTa and DeBERTa

RoBERTa [85] builds upon the foundational architecture of BERT by refining its pre-training strategies, resulting in enhanced performance across various NLP benchmarks. Although RoBERTa shares the same architecture as BERT, it introduces significant adjustments in its training procedure that collectively optimize its language modeling capability. Key modifications in RoBERTa include the removal of the Next Sentence Prediction (NSP) objective, dynamic masking, training with larger batch sizes, and an expanded vocabulary.

- **Removal of Next Sentence Prediction (NSP):** In BERT, the NSP task is designed to predict whether two sentences follow each other, intending to enhance the model’s understanding of sentence relationships. RoBERTa eliminates this task, as empirical analysis demonstrated that NSP had a negligible effect on downstream task performance. By removing NSP, RoBERTa reduces the complexity of its training process and reallocates focus to tasks that directly impact performance.
- **Dynamic Masking Strategy:** Unlike BERT’s static masking, where specific tokens are masked once and reused across epochs, RoBERTa implements dynamic masking. In this approach, different tokens are masked each time a text sequence is presented to the model. Formally, if a token sequence $\mathbf{x} = (x_1, x_2, \dots, x_n)$ is masked, the masked tokens $\mathbf{x}_{mask}$ will vary with each training instance, effectively increasing the diversity of training samples. This adjustment reduces redundancy and enables the model to encounter a wider variety of masked contexts, enhancing its ability to predict masked tokens accurately and resulting in improved language representation capabilities.
- **Increased Batch Size and Training Duration:** RoBERTa expands the batch size to 8,000 and increases the training duration compared to BERT’s 256-batch size and fewer iterations. Larger batch sizes enable more stable gradient updates and improved generalization, although they require higher computational resources. To balance the increased batch size, RoBERTa also employs gradient accumulation, allowing efficient parallelization without overloading memory resources. Additionally, RoBERTa’s extended training period—reaching up to 500,000 steps—contributes to a more nuanced understanding of language structure and vocabulary usage.
- **Byte-Pair Encoding and Vocabulary Expansion:** RoBERTa utilizes byte-pair encoding (BPE) with a larger vocabulary of approximately 50,000 tokens, compared to BERT’s 30,000 tokens. This increase reduces the frequency of out-of-vocabulary tokens and improves the model’s ability to represent complex or rare words accurately. The model learns from diverse sources, including CommonCrawl News, OpenWebText, and CC-News, totaling over 160 GB of text data, significantly more than BERT’s 13 GB. This diverse dataset provides RoBERTa with a broader linguistic foundation and better generalization capabilities.

Collectively, these innovations enable RoBERTa to outperform BERT on several NLP benchmarks, such as GLUE, SQuAD, and RACE, without altering the fundamental transformer architecture. By focusing on training optimizations and data diversity, RoBERTa demonstrates that refining the pre-training methodology can yield substantial performance gains in NLP tasks.

DeBERTa (Decoding-enhanced BERT with Disentangled Attention) introduces advanced mechanisms to enhance the representation of contextual information in language processing. DeBERTa separates the representations of word meaning (content) and position (location) in sentences, a disentangled approach that significantly improves the model’s ability to handle token and positional embeddings, especially in complex language tasks requiring nuanced comprehension. Through iterative advancements across its versions (v1, v2, and v3), DeBERTa has demonstrated strong performance on NLP benchmarks, further pushing the boundaries of BERT-based models.

DeBERTa v1: Disentangled Attention and Enhanced Mask Decoder

DeBERTa v1 [86] introduces two primary innovations to improve upon BERT’s architecture: a disentangled attention mechanism and an enhanced mask decoder.

- **Disentangled Attention Mechanism:** This allows the model to treat content and position embeddings independently. In traditional BERT-based models, token embeddings and position embeddings are summed before being processed by the attention layers. DeBERTa, however, applies attention to token and positional embeddings separately using three distinct interactions: *content-to-content*, *content-to-position*, and *position-to-content*. This design enables DeBERTa to capture how words relate to each other based on both their meanings and their relative positions in the sentence. Specifically, the attention score $\tilde{A}_{i,j}$ between two tokens i and j is decomposed into three components:

$$\tilde{A}_{i,j} = \underbrace{\mathbf{Q}_i^c(\mathbf{K}_j^c)^\top}_{\text{(a) content-to-content}} + \underbrace{\mathbf{Q}_i^c(\mathbf{K}_{\delta(i,j)}^r)^\top}_{\text{(b) content-to-position}} + \underbrace{\mathbf{K}_j^c(\mathbf{Q}_{\delta(j,i)}^r)^\top}_{\text{(c) position-to-content}} \quad (90)$$

where $\delta(i, j)$ represents the relative distance between tokens i and j , which determines the position embedding applied in each term. Unlike models like BERT, which rely on absolute position embeddings

for each token based on its fixed position in the sequence, DeBERTa's disentangled attention uses **relative position embeddings**. This means that instead of focusing on each token's specific position, DeBERTa emphasizes the *distances between tokens* to encode their relative locations within the sequence. To handle this, DeBERTa defines a unique position embedding for each relative distance within a range from $-k$ to $+k$, where k is the maximum distance of interest in the model's attention mechanism. For example, if $k = 6$, DeBERTa has embeddings for relative distances from -6 to $+6$. Any pair of tokens with a distance greater than k will share a common "far-away" embedding, which is applied to all distances beyond k . This approach allows the model to flexibly capture both close and far dependencies in a computationally efficient way, as each specific distance within $-k$ to $+k$ is assigned its own distinct embedding.

Indexing Relative Distances

To efficiently index these distances in the embedding matrix, DeBERTa maps each relative distance $\delta(i, j)$ to an index in the range $[0, 2k)$. Specifically, the mapping is defined as:

$$\delta(i, j) = \begin{cases} 0 & \text{if } i - j \leq -k \\ 2k - 1 & \text{if } i - j \geq k \\ i - j + k & \text{otherwise.} \end{cases} \quad (91)$$

This mapping ensures that each relative distance from $-k$ to $+k$ has a unique position embedding index within $[0, 2k)$, making it easy to look up the corresponding embedding for any distance.

- **Content-to-Content (standard)**: This is the standard attention term similar to BERT, where each word (or token) pays attention to other words based on their meanings alone. It essentially asks, "How important is the meaning of word j to word i ?" This term determines the attention score based purely on the content of the tokens, without considering their positions.
- **Content-to-Position**: Here, the content of word i interacts with the position of word j . This term adjusts the attention based on *where* word j is relative to word i . In essence, it helps word i decide how much weight to give to nearby positions (rather than words), based on what the content of word i indicates it needs. So, this term considers "Given the meaning of word i , how important are the positions around it?"
- **Position-to-Content**: This term allows the positional information of word i to affect how much attention it pays to the content of word j . It answers the question, "Given the location of word i , how much should it pay attention to the meaning of word j ?" Here, word i looks at its position in the sentence and uses that information to decide how relevant the meanings of other words are, based on their relative positions.

By combining these three terms, DeBERTa can capture more nuanced relationships between words, as it doesn't rely on a simple sum of content and position. Instead, each term plays a specific role:

- **Content-to-content** gives a general importance based on meanings.
- **Content-to-position** adjusts attention based on where words are, relative to what the content wants.
- **Position-to-content** allows position information to influence attention based on meanings, depending on location.

By incorporating these distinct attention components, DeBERTa effectively captures more complex linguistic relationships, especially for tasks where word order and sentence structure are crucial (see Appendix Section A.6 for a simple example of this process).

- **Enhanced Mask Decoder**: Like BERT, DeBERTa is trained with a Masked Language Modeling (MLM) objective. However, DeBERTa incorporates absolute positional embeddings in the mask decoder just before the softmax layer. This adjustment allows the model to leverage both the content and absolute positional information of words to predict masked tokens. For instance, in a sentence like "a new store opened beside the new mall," the model must recognize "store" as the subject based on its absolute position, while "mall" plays a different syntactic role.
- **Virtual Adversarial Training (VAT)**: To improve robustness and generalization on downstream tasks, DeBERTa introduces VAT during fine-tuning, which applies task-relevant perturbations to the input data, leading to better performance on complex NLP tasks.

DeBERTa v2: Enhanced Vocabulary and Structural Optimizations

Building upon v1, DeBERTa v2 introduces several structural and architectural refinements:

- **Expanded Vocabulary and Tokenization:** DeBERTa v2 increases the vocabulary size to 128K tokens and switches from the GPT-2 tokenizer to the SentencePiece tokenizer, allowing it to capture a broader range of linguistic patterns and rare words more effectively.
- **n-gram Induced Encoding (nGiE):** To better capture local dependencies, v2 includes an additional convolutional layer alongside the initial transformer layer. This layer encodes n-grams within the input, enhancing the model's ability to understand token sequences at a finer granularity.
- **Shared Content and Position Projection Matrices:** In v2, the content and position weight matrices in the attention layers are shared, which reduces the parameter count without compromising performance.
- **Bucketed Relative Position Encoding:** To encode relative positions, v2 adopts a log-bucketed relative position encoding similar to that used in T5. This modification helps the model generalize better over long sequences by grouping relative positions in a logarithmic scale, which enhances computational efficiency and contextual understanding.
- **Larger Model Variants:** DeBERTa v2 scales up with larger models (900M and 1.5B parameters) to improve downstream task performance, notably achieving near-human-level performance on SuperGLUE.

DeBERTa v3: Gradient-Disentangled Embedding Sharing and Replaced Token Detection (RTD)

break

DeBERTa v3 [87] introduces notable improvements over previous versions, focusing on more efficient pre-training objectives and embedding management:

- **Replaced Token Detection (RTD) Objective:** Inspired by ELECTRA [88], DeBERTa v3 replaces the traditional Masked Language Modeling (MLM) objective with the Replaced Token Detection (RTD) objective for pre-training. In RTD, the model is trained as a discriminator to identify replaced tokens within a corrupted input sequence, making the model more sample-efficient and accelerating learning. Unlike MLM, which focuses only on masked tokens, RTD enables the model to learn from all tokens in the sequence, optimizing data usage during training. Given an input sequence $X = (x_1, x_2, \dots, x_n)$, the RTD framework consists of two key steps:

1. **Generator Step:** A generator model replaces a subset of tokens in X with predicted replacements, resulting in a corrupted sequence $\tilde{X} = (\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n)$. Importantly, the generator replaces tokens with plausible, contextually similar alternatives rather than random words. For example, if the original sentence is "The stock market is volatile today," the generator might replace "volatile" with "stable" or "risky," yielding the corrupted sentence: "The stock market is stable today." This ensures that the discriminator's task of identifying replaced tokens remains challenging, enhancing the model's ability to understand nuanced context. The generator's objective (MLM) loss is given by:

$$L_{MLM} = \mathbb{E} \left(- \sum_{i \in C} \log p_{\theta_G} (x_i = \tilde{x}_i \mid \tilde{X}_G) \right) \quad (92)$$

where C is the set of masked tokens, θ_G represents the generator's parameters, and $\tilde{X}_G$ is the input to the generator, which is derived from masking a certain percentage (e.g., 15%) of tokens in X .

2. **Discriminator Step:** The discriminator, parameterized by θ_D , processes $\tilde{X}$ and attempts to determine if each token $\tilde{x}_i$ is original or has been replaced by the generator. The discriminator's goal is to distinguish between original and contextually similar replacements, which creates a tension in training objectives: while the generator aims to pull semantically similar tokens closer, the discriminator's RTD objective pushes them apart to optimize classification accuracy. The discriminator's objective (RTD) loss is given by:

$$L_{RTD} = \mathbb{E} \left(- \sum_i \left[y_i \log p_{\theta_D} (\tilde{x}_i \mid \tilde{X}_D) + (1 - y_i) \log(1 - p_{\theta_D} (\tilde{x}_i \mid \tilde{X}_D)) \right] \right) \quad (93)$$

where y_i is a binary label indicating whether $\tilde{x}_i$ is an original token (1) or a replaced token (0), p_{θ_D} denotes the probability predicted by the discriminator for each token, and $\tilde{X}_D$ is the corrupted input provided to the discriminator.

By jointly training the generator and discriminator in the RTD framework, DeBERTa v3 enhances the quality of learned representations, leading to improved performance on downstream NLP tasks. The overall loss function combines both generator and discriminator losses as:

$$L = L_{\text{MLM}} + \lambda L_{\text{RTD}} \quad (94)$$

where λ is a hyperparameter that scales the discriminator loss.

- **Gradient-Disentangled Embedding Sharing (GDES):** In standard embedding sharing, as used in ELECTRA, the generator and discriminator share the same token embeddings, which can create conflicting gradients during training. This is referred to as a "tug-of-war" effect because the Masked Language Modeling (MLM) loss of the generator and the Replaced Token Detection (RTD) loss of the discriminator pull embeddings in different directions: the generator seeks similarity in semantically related embeddings, while the discriminator tries to distinguish them. GDES addresses this by sharing the generator's embeddings with the discriminator but stopping the discriminator's gradients from flowing back to the generator's embeddings. To understand GDES, we define the generator's embeddings E_G and the discriminator's embeddings E_D . GDES re-parameterizes the discriminator embeddings as:

$$E_D = \text{sg}(E_G) + E_{\Delta} \quad (95)$$

where $\text{sg}(\cdot)$ is the stop-gradient operator, which prevents gradients from flowing through E_G and back-propagating into the generator. E_{Δ} is a residual matrix that allows the discriminator to have flexibility in its embeddings without affecting the generator's embeddings. The training process in GDES proceeds as follows:

1. During each iteration, the generator updates its parameters, including E_G , based on the MLM loss.
2. The discriminator updates E_D (through E_{Δ}) using the RTD loss.
3. By using the stop-gradient operator $\text{sg}(E_G)$, GDES ensures that the discriminator's RTD loss does not affect the generator's embeddings E_G . This preserves the semantic coherence of the generator's embeddings while still allowing the discriminator to learn effectively using E_{Δ} to distinguish between original and replaced tokens.

This approach, GDES, preserves the semantic information encoded in the shared embeddings E_G while preventing gradient conflicts between the generator and discriminator, and it provides a balance between parameter sharing and model independence, yielding more effective and coherent embeddings that improve training efficiency without compromising performance. Empirical results indicate that GDES outperforms other embedding-sharing techniques (such as Embedding Sharing (ES) and Non-Embedding Sharing (NES)) in terms of both model convergence speed and performance on downstream tasks. The GDES approach matches the efficiency of NES, but provides more effective embeddings by addressing the gradient conflict issue present in ES.

- **Improved Cross-Lingual Capabilities with Multilingual DeBERTa (mDeBERTa):** Leveraging the enhancements in GDES and RTD, DeBERTa v3 extends to cross-lingual tasks by training a multilingual variant, mDeBERTa. With the same architectural optimizations, mDeBERTa achieves state-of-the-art results on cross-lingual benchmarks, such as XNLI, demonstrating DeBERTa v3's robust generalization across different languages. The multi-lingual pre-training uses extensive cross-lingual datasets to build token embeddings that capture semantic meaning across languages, significantly outperforming previous models like XLM-R.

These combined advancements in DeBERTa v3 refine both the model's training efficiency and its ability to generalize, providing a highly optimized architecture for natural language understanding tasks across languages and contexts. In fact, DeBERTa's progression from v1 through v3 demonstrates a commitment to refining attention mechanisms, embedding sharing strategies, and training objectives. These enhancements collectively make DeBERTa models some of the most robust and versatile transformers available, suited for a wide range of complex NLP tasks.

Each of these variants builds upon the original BERT framework, introducing innovations that either improve performance, computational efficiency, or applicability to specialized tasks. Together, BERT and its variants have become cornerstone models in NLP, offering flexible solutions for a wide range of language understanding applications.

3.5.3. The New Era of NLP: Large Language Models (LLMs)

The field of NLP has experienced a transformative leap with the advent of Large Language Models (LLMs). While earlier models like BERT and its variants pioneered bidirectional context understanding and refined pre-training objectives, the newer generation of LLMs has fundamentally redefined the landscape. Models such as OpenAI's GPT series [89, 90], Meta's LLaMA [91], Google's PaLM [92], and Hugging Face's BLOOM [93] have achieved remarkable scalability, leveraging parameter counts in the tens or hundreds of billions, massive datasets, and unprecedented computational resources. These advancements mark a paradigm shift, positioning LLMs as general-purpose AI systems capable of tackling tasks far beyond the scope of their predecessors.

Unlike BERT, which excels in encoding tasks through bidirectional Transformers, LLMs typically adopt an autoregressive framework, focusing on generative capabilities. This design enables them to perform tasks like text generation, conversational AI, and creative content creation with remarkable fluency. Autoregressive models like GPT-3, GPT-4, and the conversationally optimized [94] ChatGPT⁷ leverage their ability to predict sequences token by token, facilitating in-context learning—a capability that allows models to understand and perform new tasks based solely on prompts and examples without further fine-tuning. This architectural divergence from BERT, combined with scaling laws, has unlocked emergent abilities such as few-shot learning, step-by-step reasoning, and cross-domain generalization, which were previously unattainable.

The success of LLMs extends beyond single-modal text processing. Models like GPT-4V [95] have integrated multimodal inputs, enabling systems to process and reason across text and visual data. These multimodal models expand the horizons of AI applications, supporting use cases like document understanding, image captioning, and complex decision-making across diverse data formats. At the same time, the rapid evolution of embedding techniques and the daily emergence of new models underscore the dynamism of this field. From domain-specific adaptations like Codex [96], optimized for programming, to multilingual systems like LLaMA and PaLM, LLMs are becoming increasingly specialized and versatile.

The development of LLMs represents a convergence of research and engineering, blurring the lines between academic inquiry and industrial application. Their impact is reshaping not only NLP but also adjacent fields like information retrieval, where traditional paradigms are giving way to conversational search systems, and computer vision, where text-vision integrations hold transformative potential. For a more detailed exploration of the technical advancements and methodologies underlying LLMs, readers are referred to the comprehensive survey by Zhao et al. [58], which offers an in-depth review of pre-training techniques, alignment strategies, and emerging applications.

3.5.4. Practical Advances in NLP

The deployment and practical application of modern NLP models, particularly Large Language Models (LLMs), require overcoming challenges related to computational efficiency, scalability, and adaptability to diverse environments. Alongside these concerns, the ability to fine-tune models for specific domains and leverage embeddings for various tasks has driven the evolution of NLP into a practical and widely applicable technology.

Deployment Challenges and Optimization

Deploying NLP models, especially LLMs, often involves addressing issues of latency, memory constraints, and scalability. These models are computationally intensive, requiring sophisticated infrastructure to deliver real-time or near-real-time performance. Latency is particularly critical for applications such as conversational AI or search systems, where even slight delays can diminish the user experience. Techniques such as model sharding, pipeline parallelism, and batching help optimize computational resources and ensure efficient inference. For large-scale deployments, container orchestration platforms like Kubernetes enable scalable systems capable of handling high workloads effectively. Edge deployments—where models operate on devices like smartphones or IoT platforms—introduce additional constraints, including limited memory, processing power, and energy efficiency. Lightweight models such as DistilBERT [79] and MobileBERT [97] exemplify architectures that retain the performance of larger models while reducing size and power consumption. Optimization techniques like pruning, quantization, and knowledge distillation are critical for these scenarios:

⁷<https://openai.com/blog/chatgpt/>

- **Pruning** involves removing redundant neurons or weights in a model, resulting in smaller architectures with minimal performance loss.
- **Quantization** reduces the precision of weights and activations, transitioning from higher-precision format to a lower-precision format, significantly decreasing memory requirements and inference times without substantial accuracy degradation.
- **Knowledge Distillation** transfers the knowledge from a larger, more complex "teacher" model to a smaller "student" model. The student replicates the teacher's behavior while operating more efficiently, making this technique ideal for resource-constrained environments.

These optimizations, alongside emerging methods like low-rank factorization and sparsity, enable LLMs to be deployed across diverse settings, from high-performance cloud environments to low-power edge devices.

Domain-Specific Fine-Tuning and Embedding Techniques

While pre-trained language models provide strong general capabilities, their true strength lies in their adaptability to specific tasks and domains. Fine-tuning allows these models to specialize in tasks like sentiment analysis, topic modeling, or named entity recognition, effectively enabling them to handle tailored requirements. For example, a general model like BERT can be fine-tuned on a labeled dataset to classify sentiment, extract topics, or identify entities such as people, locations, or organizations within text. Domain-specific fine-tuning further enhances this adaptability by training models on specialized datasets to address the unique challenges of particular industries. Examples like FinBERT [98] and SciBERT [99] illustrate this approach. FinBERT, fine-tuned on financial texts, captures nuances such as interpreting "m&a" as "mergers and acquisitions," a level of understanding that general-purpose models like BERT might overlook. Similarly, SciBERT leverages its training on scientific literature to understand complex academic language, making it invaluable for research contexts. This ability to fine-tune models for industry-specific jargon and context is critical for fields like finance, healthcare, and law, where precise understanding of specialized terminology is essential.

Embeddings further extend the versatility of NLP models by providing robust representations of language for downstream applications. Contextual embeddings from models like BERT or Sentence-BERT [100] encode text into high-dimensional vectors that capture semantic meaning, enabling tasks such as semantic search, clustering, and topic modeling. Embeddings are foundational for numerous applications:

- Retrieval-Augmented Generation (RAG) dynamically incorporates external factual knowledge into generative models [101].
- Semantic search identifies relevant documents by comparing query and text embeddings [102].
- Clustering and classification group or label text based on shared semantic properties [103].
- Deduplication eliminates redundant text entries by measuring embedding similarity [104].
- Anomaly detection identifies unusual patterns in datasets [105].
- Topic modeling uncovers underlying themes in large corpora [67].

Specialized tools like Flair [106] offer additional flexibility by generating contextual word embeddings that adapt to varying usage, while Sentence-BERT [100] focuses on sentence-level semantic representations. Together, these embedding techniques and fine-tuning approaches unlock the potential of pre-trained models, allowing them to tackle industry-specific problems efficiently and accurately.

In summary, modern NLP advancements hinge on a dual focus: optimizing models for deployment efficiency and adapting them for task-specific and domain-specific needs. By combining computational innovations with the flexibility of fine-tuning and embedding techniques, NLP models continue to expand their impact across industries, addressing challenges ranging from real-time translation to highly specialized applications in finance, law, and healthcare.

3.5.5. Ethics and Bias in NLP

The increasing deployment of NLP models has brought concerns about biases inherent in these systems and their potential societal implications [107]. Biases often stem from imbalances or stereotypes in training datasets, which are subsequently reflected in model predictions. For example, if the training data overrepresents certain demographic groups or reinforces stereotypes, the model may produce outputs that perpetuate these biases [108].

Addressing these biases requires a multifaceted approach. Fair data curation is a critical first step, ensuring that training datasets represent diverse perspectives and avoid over-reliance on biased sources. Another

promising avenue is adversarial training [109], where models are exposed to adversarial examples designed to challenge their robustness and reduce susceptibility to biased outputs. Despite these efforts, challenges remain in ensuring that NLP models are both fair and effective across a wide range of applications.

Transparency and accountability are also essential in deploying NLP models responsibly. Clear documentation of training data sources, explicit explanations of model decisions, and adherence to ethical guidelines can help build trust in these systems. As the use of NLP models continues to expand, particularly in sensitive domains like hiring [110], healthcare [111], and legal decision-making, prioritizing fairness and ethical practices is more critical than ever.

3.5.6. Future Directions in NLP

The future of NLP is being shaped by several key trends and challenges that aim to make models more adaptable, transparent, and sustainable. One promising direction is continual learning [112], where models can adapt to new information incrementally without forgetting previously acquired knowledge. This capability is vital for applications in dynamic environments, such as real-time data analysis or updating systems with evolving information.

Explainability and transparency are becoming critical requirements as NLP models are integrated into high-stakes domains like healthcare, finance, and law [113]. Users and stakeholders need to understand how decisions are made, which necessitates the development of interpretable models and robust tools for tracing decision-making processes. Models with high explainability will also help mitigate ethical concerns by providing clearer accountability for their outputs.

4. Survey Methodology

This survey synthesizes findings from 22 research papers published from 2018 onward, including a few additions from 2024, to explore the integration of RL with NLP for stock market trading. The search strategy was carefully designed to ensure relevance and focus, employing Google Scholar with keywords such as *"text embeddings reinforcement learning stock trading"*, *"trading stocks 'reinforcement learning' NLP"* and *"Deep Reinforcement Learning NLP Stock Trading"*. Papers were selected based on their alignment with the survey's focus, prioritizing studies that integrated RL with NLP and evaluated actual trading algorithms using trading-specific metrics, such as profit, loss, or return on investment.

Inclusion criteria emphasized recent publications that demonstrated NLP contributions, including text embeddings or advanced sentiment analysis techniques, and their integration with RL for trading. Exclusion criteria filtered out papers focused solely on stock price prediction without implementing trading algorithms, those lacking substantial NLP-RL integration, unusually short papers, and limited certain master's theses. This approach allowed the review to concentrate on studies contributing significant insights to the field.

Selected papers were systematically analyzed to extract insights into their methodologies, contributions, and findings. Common themes, challenges, and trends in RL-NLP integration were synthesized to provide a comprehensive overview of the current state of the field.

Despite these efforts, limitations exist. The reliance on Google Scholar may have excluded relevant works from other databases, and the specificity of keywords might have overlooked studies employing alternative terminology. Additionally, the exclusion of papers without explicit trading metrics may have missed innovative but indirectly relevant approaches. While every effort was made to include relevant works from 2024, the primary focus of this survey was on papers published between 2018 and 2023, as substantial time was dedicated to the development of the foundational concepts section. Given these constraints, 2024 papers were subjected to an even higher quality threshold compared to earlier years. Only those that demonstrated significant and innovative NLP contributions beyond traditional sentiment analysis or basic integrations were included. Papers failing to meet these elevated standards, or those lacking sufficient novelty, were excluded. Despite this selective approach, the recurring themes, gaps, and challenges identified in this survey will still remain representative of the field and provide a robust synthesis of RL and NLP integration for stock market trading. This method ensures that the few included 2024 papers offer truly meaningful insights without compromising the survey's representation of the field in its current state.

5. Applications of Reinforcement Learning and NLP in Stock Trading

5.1. Overview

Recent advances in machine learning have seen a growing body of research that leverages RL and NLP to guide decision-making in stock trading. Traditional quantitative trading strategies have long relied on historical price data, technical indicators, or classification/regression tasks, yet these approaches can be seen as too simple [1] and may fail to fully capture the nuanced signals embedded in unstructured textual data such as financial news, social media posts, and corporate filings. The integration of NLP into RL-based trading frameworks aims to distill complex (time-dependent) textual information into actionable insights, allowing trading agents to identify profit opportunities, mitigate risks, and adapt to shifting market conditions.

A common theme across these studies is the use of sentiment and contextual information derived from textual data to complement trade signals. This integration has demonstrated the potential to improve profitability and decision quality significantly. For example, BERT-based models and simpler sentiment tools such as FinBERT and VADER [114] are widely used to incorporate market mood into trading strategies [115, 106, 116, 117, 118]. Unlike traditional trading methods that often first predict prices, RL-based frameworks directly optimize objectives such as cumulative returns, risk-adjusted profits, or Sharpe ratios [119, 120].

The surveyed literature employs a variety of RL methods, including policy gradient algorithms such as Proximal Policy Optimization (PPO) [121], value-based methods like Deep Q-Networks (DQN) [122], and actor-critic approaches such as Deep Deterministic Policy Gradient (DDPG) [123]. Sentiment signals often form a critical input to these models. Some studies focus on long-term investment strategies informed by financial news [121], while others leverage social media sentiment for short-term trading gains [116]. Hierarchical attention mechanisms, time-aware LSTMs, and multi-modal data fusion are common techniques for capturing the temporal and contextual complexities of textual data [1, 124].

For instance, Sawhney et al. [1] proposed a Time-Aware RL model that uses hierarchical attention mechanisms and LSTMs to handle irregularities in financial news and social media data. Similarly, Nan et al. [122] employed DQN with knowledge graphs to reveal hidden relationships in market dynamics. Other works combine recurrent convolutional networks (RCNN) for sentiment extraction with DDPG-based trading strategies [125], or leverage modular designs like Evolving Agent Modules (EAMs) for scalable portfolio management [126].

A significant challenge is the inherent noise and imbalance in textual data, as not all news or sentiment signals are equally predictive. To address this, many studies use domain-specific sentiment models (e.g., FinBERT [127]) or filtering mechanisms to identify "spotlight" news [128]. Additionally, state augmentation techniques, such as embedding sentiment and predicted price movements into the RL state, are often employed to enrich the agent's understanding of the environment [119].

The outcomes reported in these studies consistently show that combining RL with NLP-driven signals enhances trading performance. Improvements in Sharpe ratios, risk-adjusted returns, and profitability are frequently cited. However, these gains are contingent on well-designed systems: poor feature selection or sentiment extraction methods may fail to add value. Models must also contend with the non-stationary and volatile nature of financial markets, a challenge often addressed through techniques such as multi-seed experiments or robust evaluation frameworks [129].

In summary, the integration of RL and NLP for financial trading represents a promising yet challenging frontier. The surveyed papers demonstrate that text-derived signals can significantly enhance trading agents' performance, especially when combined with innovative RL architectures and state representation techniques. As NLP tools and RL algorithms continue to evolve, we anticipate more robust, interpretable, and scalable systems for tackling the complexities of modern financial markets. A detailed summary of the reviewed works, including their key methods, objectives, and findings, is presented in Table 1.

Table 1: Survey of RL+NLP Approaches in Stock Trading

Citation	Keywords	Objective/Purpose	Key Findings & Methods
[1] 2021	DDPG; Custom Policy; Time-Aware LSTM; LSTM; Attention; Risk Adverse; BERT; Chinese BERT	DRL approach that makes time-aware trading decisions using textual data to optimize profits.	- Text-based signals rival or surpass pure price-based strategies. - RL framing (vs. regression/classification) improves profitability. - Hierarchical attention and time-aware LSTMs handle the irregularities in the release of news/tweets. - Strategy design is centered around maximizing profit and reducing investor risk by preferentially allocating resources to less volatile stocks. - Custom DDPG-based policy network.
[122] 2022	Sentiment Analysis; Knowledge Graphs; Deep Q-Network (DQN)	RL-based trading policy by integrating price data with news headline sentiment and leveraging knowledge graphs to reveal implicit relationships impacting market dynamics.	- Sentiment-informed RL agent outperformed baseline agents lacking sentiment data. - Used knowledge graphs to uncover hidden connections in news, improving policy decisions. - Employed a DQN architecture. - Action space included Buy, Sell, and Hold; sentiment scores guide more profitable trading, and there was a heavy emphasis on reward shaping to discourage the model from passively trading for long periods.
[125] 2019	DDPG; LSTM; sentiment; RCNN	Emphasized swing trading using DRL with sentiment analysis to interpret financial news for improved stock trend forecasting.	- DDPG model combined with a RCNN for sentiment classification. - Sentiment classifier included an embedding, CNN, LSTM, and output layer for robust feature extraction and binary sentiment prediction. - Agent actions included buy, sell, and hold, with a focus on discounted rewards to optimize trading strategies. - RCNN improved sentiment analysis by capturing both contextual and semantic features, outperforming traditional RNNs. - Utilized a binary-reward system (i.e., reward agent if action was profitable or not), yielding the best performance.
[121] 2019	Long-term portfolio management; PPO	Leveraged historical prices and news/tweets for long-term investment decisions using RL, assessing various data observation methods.	- PPO is used for RL. - Implements two approaches: independent training of sentiment prediction and trading, and joint training. - Employed a binary-reward system to avoid reward sparsity and prioritize confident investment decisions. - Sentiment classification uses CNNs with BERT-initialized word embeddings, correlating sentiment labels with stock price movements. - Identifies challenges like overfitting, flawed sentiment labeling methods, and random guess performance in testing phases.
[115] 2021	Influence model	Develop a multimodal RL trading system that integrates sentiment analysis and an influence model to capture news impact on market decisions.	- Utilized MDRNN preprocessing framework with separate channels for price and news features, merging them into a joint multimodal influence vector for decision-making. - Employed VADER for sentiment polarity scoring, and integrating it with price data using LSTMs. - Demonstrated significant profit increases, especially in the Information Technology sector.

(Continued on next page)

Citation	Keywords	Objective/Purpose	Key Findings & Methods
[129] 2021	A2C; sentiment correlation signal; rolling window	Developed the Sentiment-Aware Reinforcement Learning (SentARL) system, which integrates a market sentiment momentum strategy to enhance the consistency and profitability of single-asset trading.	- Integrated A2C with various features. - Very thorough experiments were conducted to evaluate the influence of news on SentARL performance, and discussed assets with most news coverage and how it impacts the approach. - Evaluated performance across 20 assets during five periods, including the COVID-19 pandemic, using no-penalty and high-penalty transaction cost scenarios. - Addressed challenges of market unpredictability by leveraging sentiment momentum and optimizing look-back windows.
[119] 2020	SARL; data heterogene- ity; asset movement prediction; financial news embedding; DPG	Developed SARL, a State-Augmented Reinforcement Learning framework, enhancing Portfolio Management by addressing data heterogeneity and uncertainty, with real-world validation demonstrating improved risk-adjusted profits and decision-making.	- SARL achieved significant Portfolio Value (PV) improvements: 140.9% on Bitcoin and 15.7% on HighTech datasets compared to state-of-the-art RL methods like DPM. - Utilized a hierarchical attention network (HAN) for financial news embedding, predicting asset movement for state augmentation. - Demonstrated the impact of prediction accuracy and label density on PM performance. - Highlighted hidden correlations in news data, leveraging them for improved PM strategies, particularly in volatile markets.
[130] 2018	Q-learning; Sentiment- driven investment; text mining; RL; financial news impact	Demonstrated how news sentiment enhances automated trading strategies, evaluating rule-based and ML approaches for more profitable investment decisions.	- Observed superior returns in machine learning-based methods compared to traditional benchmarks. - Supervised and RL approaches yielded significantly higher returns compared to simple news trading and momentum-based strategies. - Integrated sentiment analysis of regulated ad hoc announcements to predict stock price movements, quantifying polarity as positive or negative. - Addressed risk factors in machine learning strategies due to prediction errors, emphasizing the importance of model selection and strategy design.
[131] 2022	Deep RL; knowledge graphs; sentiment analysis; Q-learning	Developed a DRL method combining sentiment and knowledge graphs for enhanced stock prediction in the Turkish market.	- Proposed a deep reinforcement learning framework integrating community-aware (CA) sentiment analysis and knowledge graphs for stock price prediction. - Compared traditional DQN models (DQN, DDQN, DDDQN) with community analysis (CA) enhanced variants (CA-DQN, CA-DDQN, CA-DDDQN) on datasets of Turkish banks (GARAN, AKBNK, ISCTR). - CA-based models consistently outperformed traditional models in profitability and Sharpe ratios, with CA-DQN and CA-DDDQN achieving the best performance. - Demonstrated the effectiveness of integrating community sentiment with RL for accurate price predictions and profitable trading strategies.
[132] 2021	Portfolio allocation; DDPG; Adaptive DDPG; sentiment	Developed a DRL-based portfolio allocation method that incorporates market sentiment, improving returns and risk measures for Dow Jones stocks.	- Proposed a sentiment-aware deep reinforcement learning approach, extending the Adaptive DDPG algorithm. - Integrated sentiment signals from Google News and social media (weighted by retweets and likes) into the RL state and reward function to dynamically leverage market sentiment. - Limited extreme decisions by capping trades at 5 stocks per action and scaled sentiment scores for a stronger influence on rewards.

Citation	Keywords	Objective/Purpose	Key Findings & Methods
[126] 2022	MARL; modular design; EAM; SAM; DQN; PPO	Introduced a modularized multi-agent RL-based system (MSPM) for portfolio management, integrating Evolving Agent Modules (EAMs) and Strategic Agent Modules (SAMs).	- EAMs, using DQN-based agents, process heterogeneous data to generate actionable signals, while SAMs (PPO-based) reallocate capital across portfolios, enabling scalability and parallel processing. - Demonstrated superior performance of EAM-enabled MSPM over baselines. - Incorporated transfer learning by pre-training foundational EAMs with AAPL data and reusing them across other assets, reducing training inefficiencies. - Enhanced system robustness with risk-adjusted return as the reward function and "news_buzz" attributes to address the unbalanced-news issue in sentiment data.
[127] 2022	DRL; sentiment; twin-delayed DDPG	Developed a DRL-based automated trading system that integrates prediction and allocation, minimizing risk and maximizing profit in dynamic stock markets.	- Proposed a simulated trading environment reflecting real-world conditions, incorporating ten technical indicators and sentiment analysis scores from financial news. - Utilized Twin Delayed Deep Deterministic Policy Gradient (TD3) for solving the Partially Observed Markov Decision Process (POMDP) with high-dimensional, continuous action spaces, outperforming the baseline DDPG algorithm. - Enhanced state representation by integrating sentiment probabilities calculated using a fine-tuned FinBERT model, assigning positive or negative scores to daily sentiment.
[120] 2023	Portfolio management; sentiment; DQN; DDQN; Dueling DQN	Developed a framework combining RL and sentiment analysis to automate and enhance portfolio decisions, outperforming traditional methods in returns and risk management.	- Applied Deep Q-Network (DQN) alongside enhancements like Dual DQN (DDQN) and Dueling DQN to improve decision-making in dynamic market environments. - Integrated NLP for sentiment analysis using GPT-3, extracting insights from news and social media for better-informed trading decisions. - Claimed improved performance over traditional portfolio management strategies.
[124] 2024	Multi-modal; cryptocurren- cies; PPO; LSTM	Developed a multi-modal DRL approach that fuses price and sentiment data to improve trading performance and address overfitting, measured by profit and loss.	- Combined price and sentiment features using embedding layers to improve financial trading performance. - Proposed learnable parameters a (price data) and b (sentiment data) to adjust the influence of each feature. - Demonstrated higher profitability and stability compared to traditional trading methods, supervised learning models, and other RL approaches. - Incorporated minute-based OHLC price features, time features (day, week, month), and sentiment features (e.g., averages, previous values, differences). - Resampled data hourly, aggregating price and sentiment statistics to mitigate noise and overfitting risks.
[133] 2023	DRL; variational mode decom- position; sentiment; BERT; gold market	Combined ANN, LSTM, NLP, and DQN for improved gold stock value predictions, demonstrating higher accuracy and effectiveness than traditional methods.	- Utilized sentiment analysis through fine-tuned BERT. - Incorporated VMD for noise removal and LSTM for predicting decomposed signals, enhancing price prediction accuracy. - Achieved superior performance across metrics such as annualized wealth rate, drawdown, and action decision accuracy compared to baseline models.

Citation	Keywords	Objective/Purpose	Key Findings & Methods
[116] 2022	RL; DQN; sentiment; social media; FinBERT	Developed and evaluated an RL trading model integrating sentiment analysis from news and social media, showing potential to outperform strategies lacking textual data.	- Employed a Deep Q-Network (DQN) approach, extended to Double DQN (DDQN), integrating sentiment analysis from financial news (using FinBERT) and social media (using TweetEval) for algorithmic stock trading. - Analyzed sentiment-based features derived from textual sources, including social media metrics like emoji-based sentiment and article/tweet counts, alongside technical indicators and financial data from Yahoo Finance. - Hypothesis-driven insights: Found text analysis (especially combining social media and news sentiment) improves trading profitability for certain stocks, but generalization across markets remains challenging due to stochastic nature.
[117] 2023	DQ-learning; sentiment; VADER	Developed a DQ-learning model incorporating sentiment and trend labeling to enhance stock trend prediction, achieving better portfolio value and Sharpe ratios.	- Developed a trading agent using a Deep Q-learning model enhanced with sentiment analysis from news titles and social media, employing VADER for sentiment extraction. - Incorporated stock trend prediction rather than direct price prediction. - Highlighted robustness during market crises, particularly outperforming during the Covid-19 pandemic. - Conducted comparative evaluations with machine learning classifiers (neural networks, decision trees, KNN, boosting, and SVM), demonstrating superior performance. - Demonstrated effectiveness in leveraging sentiment analysis for events like Tesla's Shanghai plant production, showcasing the model's ability to integrate real-time market news for enhanced trading decisions.
[118] 2022	Sentiment; VADER; experience replay; sentiment- driven trading	Integrated RL and sentiment analysis for stock trading decisions, demonstrating improved profitability by incorporating market sentiment into buy/sell/hold actions.	- VADER-generated sentiment scores of stock-related news. - RL agent, using a deep neural network with three fully connected hidden layers, optimizes buy, sell, and hold actions to maximize profitability. - Testing showed RL with sentiment analysis outperformed both RL without sentiment and benchmark models, highlighting the value of incorporating sentiment data.
[123] 2021	DRL; sentiment; Brazilian market; PPO; DDPG; Portuguese NLP	Developed a DRL-based asset trading system with sentiment and time-series prediction, improving strategies and returns in the Brazilian stock market context.	- Proposed a three-module system: price prediction (M1), sentiment prediction (M2), and automated trading (MT), integrated with a DRL agent. - Employed econometric models (ARIMA, SARIMAX), ML (SVR, AdaBoost), and DL (LSTM) for price prediction, and CNNs with GloVe embeddings for sentiment analysis. - Found that sentiment analysis improved volatility and loss-related metrics but had minimal impact on returns, better suited for risk-averse strategies in dynamic markets.

(Continued on next page)

Citation	Keywords	Objective/Purpose	Key Findings & Methods
[128] 2023	News-driven trading; RL; Spotlight-Trader; trajectory optimization; GPT-2; sparse attention	Developed SpotlightTrader, an RL-based News-driven Quantitative Trading (NQT) framework, focusing on decision trajectory optimization and spotlight-driven state trajectory modeling..	- Introduced a spotlight-driven state trajectory module using sparse attention to identify and leverage pivotal news while minimizing noise from irrelevant news. - Employed a novel pretrain-finetune training pipeline for adapting to non-stationary financial markets, incorporating trajectory-level replay buffers and prioritized sampling policies. - Validated effectiveness through experiments, outperforming baseline NQT methods in profitability, risk control, and adaptability to abrupt market changes. - Demonstrated coherent trading actions during case studies, such as Boeing (BA) stock during COVID-19, emphasizing the practical application of spotlight news filtering in guiding trading decisions.
[134] 2024	HFT; futures trading; PPO; News contextual embeddings; BERT; FinBERT; Llama 2; Mistral; Continuous action space	Proposed a reinforcement learning framework for high-frequency futures trading that leverages contextual news embeddings (from both transformer encoder and decoder-based language models) combined with minute-level price data, aiming to improve trading performance over purely price-driven or sentiment-based strategies.	- The authors replace sentiment-based news data with purely embeddings from BERT, FinBERT, Llama 2, and Mistral, demonstrating that deeper linguistic representation of news yields improved trading outcomes. - By mapping the RL agent's output $a_t \in [-1, +1]$ to integer lots via $\lfloor a_t \times \max_lots \rfloor$, the framework permits fine-grained control over the number of futures contracts traded. - To process sequences of technical indicators and news vectors, the model adopts a CNN architecture, capturing local temporal/contextual signals that feed into the PPO agent. - Experiments show that FinBERT lowers maximum drawdown while large LLMs (Llama 2, Mistral) achieve higher returns, collectively outperforming sentiment-only or price-only approaches.
[135] 2024	Multi-factor stock trading; DQN; Multi-BiGRU; multi-head ProbSparse attention; LightGBM turning-points; temporal vs. non-temporal models; sentiment	Proposed a multi-factor DQN-based trading strategy, combining BiGRU for bidirectional temporal learning, multi-head ProbSparse attention for efficient factor interaction, and LightGBM to detect turning points in daily stock data. By fusing financial-quality, valuation, and sentiment factors, the approach achieves higher returns across Chinese and US markets.	- Multi-BiGRU replaces simpler CNN/LSTM networks in DQN to capture forward/backward dependencies in multi-factor data. - Multi-head ProbSparse self-attention probabilistically filters "active" queries vs. "lazy" queries, focusing on high-impact factors. - LightGBM classifies absolute/relative technical indicators (ATI&RTI) to identify buyable vs. top turning points, guiding the final DQN policy. - Demonstrates statistically significant improvements over both temporal and non-temporal baselines, validated by boxplots, Friedman tests, and ablation studies.

5.2. Baseline Methods Comparison

Baseline methods provide a critical point of reference for evaluating the advancements offered by RL-NLP models in stock market trading. By comparing RL-NLP approaches against traditional methods or sentiment-free RL models, these studies highlight how integrated frameworks improve profitability, risk management, and adaptability. This section outlines the baseline methods considered in the reviewed papers and summarizes their comparative performance in Table 2.

Table 2: Baseline Methods Comparison

Citation	Baseline Methods Comparison
[1] 2021	Regression (W-LSTM, AZFinText), Classification (TSLDA, StockEmb, SN-HFA, MAN-SF, Chaotic), Ranking (R-LSTM, RankNet), RL (iRDGP, AlphaStock, S-Reward, SARL [119])
[122] 2022	Random Policy (random Buy, Sell, Hold actions), Baseline Portfolio Value (representing the starting portfolio value of \$1000 and 10 stocks), with sentiment and without sentiment
[125] 2019	No explicit baseline methods were provided in this study
[121] 2019	Price-only RL, news-only and pretrained sentiment classifier RL
[115] 2021	DRNN and MDRNN neural network/sentiment score variants (i.e., textual contents from titles, articles, or synthesized influence)
[129] 2021	Sentiment-free baseline and Buy-and-Hold (BandH) strategy
[119] 2022	CRP (Constant Rebalanced Portfolio), OLMAR (On-Line Moving Average Reversion), WMAMR (Weighted Moving Average Mean Reversion), EW (Equal Weight), DPM (Deep Portfolio Management)
[130] 2018	CDAX index, momentum trading strategies, portfolio trading, news-based strategies (e.g., simple news trading, sector-specific, combined news and momentum), supervised learning (i.e., random forest)
[131] 2022	DQN, DDQN, DDDQN compared against their Community Aware sentiment-enhanced counterparts: CA-DQN, CA-DDQN, and CA-DDDQN
[132] 2021	Adaptive DDPG, DDPG, and traditional mean-variance and minimum-variance portfolio analyses
[126] 2022	Constant Rebalanced Portfolio (CRP), Buy-and-hold (BAH), Exponential gradient portfolio (EG), Follow the regularized leader (FTRL) strategy, and adversarial deep reinforcement learning in portfolio management (Adversarial PG)
[127] 2022	Baseline (no technical indicators or sentiment), WithTechIndicators (technical indicators only), and WithSentiments (technical indicators combined with sentiment scores)
[120] 2023	DQN, DDQN, and Dueling DQN
[124] 2024	Buy and Hold, DSL model (Price, Price+Sentiment, DSL variants), price trailing, and DRL agents (Price, Price+Sentiment, Price+Full Sentiment)
[133] 2023	Buy-and-Hold benchmark, best stock benchmark, and Constantly Rebalanced Portfolios (CRPs)
[116] 2022	Buy-and-Hold, Dollar-Cost Averaging (DCA), Timed DCA, and various DDQN model configurations
[117] 2023	Traditional Deep Q-learning Model (DQN) and Holding Strategy
[118] 2022	Emulation of manual trading, RL without Sentiment, RL with Sentiment
[123] 2021	Buy-and-Hold (BH), MT models (MT1, MT2, MT3, MT3ta, MT4, MT4ta) incorporating OHLCV, price prediction, sentiment analysis, and technical indicators
[128] 2023	S&P 500 index, Regression (AZFinText, Game), Classification (StockNet, StockEmb, HTLSTM), Ranking (FAST, HISN), and Reinforcement Learning (SARL [119], Profit [1])
[134] 2024	Price-only RL baseline, sentiment-based RL baseline, and four text embedding RL variants (BERT, FinBERT, Llama 2, Mistral)
[135] 2024	Non-temporal DQN baselines (DQN, DoubleDQN, DQN_CNN, DuelingDQN), temporal RL baselines (TFJ-DRL, PMMRL, MSF-DRL), and a reference to Buy-and-Hold

5.3. Input Data/RL Environment, Data Sources, and Preprocessing

In reinforcement learning models for stock trading, the input data constitutes the states that define the RL environment. These states represent the agent’s perception of the market and are crucial for effective decision-making. Common state representations include prices, portfolio data, technical indicators, sentiment from news articles or social media, and broader market signals. Preprocessing these inputs—through techniques such as normalization, tokenization, or sentiment scoring—ensures that they are structured appropriately for integration into RL models. This section reviews the diverse data sources and preprocessing methods employed

in the surveyed studies. Table 3 provides a summary of these data types, sources, and their transformation processes into actionable states for RL agents.

Table 3: Input Data/RL Environment, Data Sources, and Preprocessing

Citation	Input Data/RL Environment	Data Sources	Preprocessing
[1] 2021	Tweets, news headlines, stock prices	Prices from Yahoo Finance, stocknet-dataset, and China & Hong Kong dataset	Tokenization and truncation of text; preprocessing tweets using NLTK (Twitter mode) to handle URLs, @ mentions, and hashtags; employing pre-trained BERT models (BERT-base-cased for English tweets, Chinese-BERT-base for Chinese news); character-based tokenization for Chinese headlines; aligning trading days by discarding incomplete data windows; temporal splits for training, validation, and testing across datasets.
[122] 2022	Current money, stock holdings, opening stock price, differences in opening price (5-day and 50-day averages), and average sentiment towards the company for the day	Yahoo Finance API for stock prices; historical news headlines scraped from Reuters Twitter account	Removed stopwords and tokenized headlines; used a knowledge graph with a pre-specified distance of 5 to identify relevant headlines; applied an ensemble sentiment analyzer (IBM Watson, TextBlob, NLTK) to classify sentiment as positive or negative based on the classifier with the highest confidence; aggregated daily sentiment using majority sentiment scoring.
[125] 2019	Current stock closing price, moving averages, available capital, number of stocks held, and stock trend prediction	Not specified	HTML markup removal; sentence tokenization; stopword removal; stemming; word indexing using Bag of Words; converting cleaned sentences into a list of indices; Bag of Words constructed from a corpus of financial news headlines.
[121] 2019	Prices (open, high, low, close) and news	Dow Jones Industrial Average (DJIA) dataset from Kaggle	Used a 2-layer CNN with batch normalization and dropout for sentiment classification; employed contextualized word embedding via BERT.
[115] 2021	Historical stock price return and news influence toward the market	Price data sourced from Wharton Research Data Services (WRDS) NYSE Trade and Quote database; News data from Bloomberg and Reuters financial news datasets (2006–2013) and additional news titles from Bloomberg (2014–2020)	Adjusting stock data frequency from milliseconds to minutes for trading during regular hours; Rearranging news items according to trading days; Sentiment conversion using VADER, calculating a feature vector including count, mean, standard deviation, and quartiles for sentiment scores by date; Fusion of price and sentiment features using LSTMs with early fusion.
[129] 2021	Closing bid price of each asset and the sentiment of the news headlines	Employed datasets include 5,267 hourly price data points between 2018 to 2020 (three years) collected from a specialized financial portal (Dukascopy). News data was sourced from a portal (MarketWatch), which aggregates articles from several outlets, including the Wall Street Journal	Extracted sentiment using a sentiment analyzer trained on the SemEval-2017 Task 5 Financial Sentiment Dataset. Headlines were assigned sentiment scores ranging from -1 (pessimistic) to 1 (optimistic) and aggregated using rolling look-back windows. Data alignment ensured synchronization between hourly price data and corresponding sentiment scores. Constructed state representations by combining price differences, sentiment features, and normalized time-of-day inputs.

(Continued on next page)

Citation	Input Data/RL Environment	Data Sources	Preprocessing
[119] 2022	Asset prices (e.g., stock prices) represent the internal features and financial news provides external information encoded into the final state. Predicted price movements are incorporated using an LSTM model.	Bitcoin Dataset: Prices of 10 cryptocurrencies recorded at half-hour intervals from 2015-06-30 to 2017-06-30, with 35,089 data points split into training (32,313) and testing (2,776). HighTech Dataset: Daily closing prices and financial news for nine companies with maximum news coverage from 2006-10-20 to 2013-11-20, including 4,810 articles covering 72.48% of trading days.	Not detailed about specific cleaning steps.
[130] 2018	Sentiment scores, rate-of-change of stock prices, performance indicators, and the CDAX index	Ad Hoc Announcements and financial market data, including stock prices and the CDAX index (German stock market index)	Not specified.
[131] 2022	Closing stock price, daily sentiment value, 5-day and 30-day averages for price growth and sentiment	Twitter data for sentiment and Yahoo Finance for price data	Applied L2 normalization to sentiment values; fine-tuned BERT model for sentiment classification; filtered tweets using knowledge graph relationships and influence scores (retweets, likes, replies); extracted relevant keywords from a knowledge graph for sentiment analysis.
[132] 2021	Daily closing prices of Dow Jones stocks and sentiment from Google News headlines and Twitter tweets	Stock price data sourced from the Compustat database accessed via WRDS; news articles collected using a Google News scraper; and tweets obtained via Selenium automation from Twitter	Sentiment analysis performed using VADER on both news headlines and tweets. Sentiment polarity scores for headlines and tweets were weighted by engagement metrics (e.g., likes and retweets) for tweets. Missing sentiment data was set to neutral (0). Sentiment scores from Google News and Twitter were combined to form a final sentiment file.
[126] 2022	Adjusted-close, open, high, low prices, volumes, sentiment, and news_buzz	Price data from QuoteMedia's End of Day US Stock Prices (EOD) (2013–2020) accessed via Nasdaq Data Link's API (EOD); sentiment and news_buzz from InfoTrie's FinSentS Web News Sentiment data (FinSentS)	Normalized all EOD data by dividing by the first-day value; filled missing FinSentS data (9.51% missing) with neutral sentiment scores (0); news_buzz and sentiment were scaled (-5 to 5 for sentiment, 1 to 10 for news_buzz). Buzz scores indicated volatility, while sentiment scores provided market mood insights.
[127] 2022	Prices (Date, Volume, Open, Close, Adjusted Close, High and Low prices), sentiment scores, and 10 technical indicators	Historical market daily prices retrieved from Yahoo Finance, financial news data compiled from Benzinga, Seeking Alpha, Zacks, and Reddit's worldNews Channel, with over 3.2M news headlines (2009–2021)	Used trading calendars to exclude weekends/holidays and ensure no gaps. Aligned historical data lengths across stocks. Technical indicators calculated from Close, High, Low prices, and Volume data with specific look-back windows.
[120] 2023	Stock price data and sentiment	Data retrieved from Yahoo Finance for SENSEX NSE and newsapi.org	Preprocessing included filtering null values and focusing on four main features: open, high, low, and close prices.
[124] 2024	Prices and extensive features about prices, sentiment analysis of news and social media	Online sources, including financial news articles and social media sites	Tables in the paper describe input features like rolling window price volatility and sentiment averages. Preprocessing was centered around standardizing values and feature engineering.

(Continued from previous page)

Citation	Input Data/RL Environment	Data Sources	Preprocessing
[133] 2023	Historical and predicted prices, sentiment analysis outputs, and technical indicators like relative strength index (RSI) and momentum (MOM)	Gold market data and stock prices from S&P, Yahoo Finance, NASDAQ, and news data from Kaggle	Preprocessed news by tokenizing, lowercasing, removing stopwords, stemming, and lemmatizing. Stock price continuity ensured by filling gaps with previous trading day data.
[116] 2022	Price features include returns over various periods (1-day, 2-days, 1-week, 2-weeks, 1-month) and financial indicators like Stochastic, Ultimate Oscillator, RSI, MACD, and ATR. Additional features encompass news/opinion sentiment scores (content and titles) over 1, 5, and 21 days, tweet sentiment metrics (keywords and cashtags), and counts of related tweets and retweets	Yahoo Finance API for prices, news articles from Investing.com, Reuters, and Bloomberg, and public social media platforms like Twitter	Text cleaning included lowercasing, removing URLs/whitespace. Sentiment analysis handled via FinBERT, which is robust without additional stopword removal or stemming.
[117] 2023	Binary stock price trends (up/down), sentiment scores, price indicators, and moving averages	Yahoo Finance API and news datasets from Kaggle	Preprocessing involved filling missing stock price data, binary labeling trends (up/down), and using adjusted closing prices. Sentiment was derived from headline data.
[118] 2022	OHLCV prices, sentiment, 5-day moving average price of the stock, stocks held, cash held, and sentiment score of the previous day	Stock news API and stock API for price data	Prices processed for moving averages. Sentiment preprocessing included tokenization, stopword removal, lowercasing, and lemmatization.

(Continued on next page)

(Continued from previous page)

Citation	Input Data/RL Environment	Data Sources	Preprocessing
[123] 2021	Prediction inputs include daily BOVA11 index OHLCV data, technical indicators (ADI, BB, MACD, RSI), and sentiment datasets (labeled D2-A, unlabeled D2-B). RL models used outputs from prediction modules: MT1 (price prediction signals), MT2 (sentiment signals), MT3 (OHLCV features), MT3ta (OHLCV with TIs), MT4 (combined price and sentiment signals), and MT4ta (combined with TIs)	Price data sourced via Yahoo Finance, sentiment predictions from sources like UOL Economia and Investing.com	Outlier detection, missing data handling, tokenization, lemmatization, and elimination of irrelevant titles.
[128] 2023	Account observations (balance, holdings), OHLC prices, sentiment embeddings	Datasets from Twitter (SP500), iFinD (CSI300), and Aylien (COVID-SP500)	Preprocessing included filling missing values, normalizing prices, and embedding news text using pre-trained FinBERT.
[134] 2024	OHLC price data turned into technical indicators (ADX, MACD, MOM, ATR, RSI, Slow %K, Williams %R, BBAND, EMA), combined with up to 60 minutes of recent financial news titles (embedded via BERT, FinBERT, Llama 2, or Mistral)	NIFTY 50 one-minute dataset from Kaggle (2010–2021) and Economic Times archive (filtered for financial news)	Technical indicator values were z-normalized; news titles were embedded using transformer models (BERT/FinBERT) or 4-bit-quantized LLMs (Llama 2/Mistral). A 60-minute window was used to mitigate news sparsity; price data was also taken over a rolling window of size w for each time step.
[135] 2024	Multiple factors (valuation, financial quality, leverage, size, momentum, volatility, turnover, shareholder, technical indicators), sentiment factor via BiLSTM, and LightGBM-based turning-point signals	Chinese A-share data via ITendx and third-party APIs (akshare, yfinance) for US markets; comment text from Eastmoney, Snowball Finance	Z-score normalization applied to all multi-factor features and sentiment scores; LightGBM labels “buyable” vs. “top” turning points; final RL environment uses daily data fused with turning-point signals and BiGRU attention layers.

5.4. Stock Exchange, Stocks Traded, and Ticker/Data Frequency

The context of stock market trading varies greatly depending on the exchanges, assets, and timeframes under consideration. This section explores the specific exchanges, stocks, and ticker data frequencies (e.g., daily,

intraday) used in the reviewed papers, as these choices significantly impact the models' design and evaluation. A detailed breakdown is presented in Table 4.

Table 4: Stock Exchange/Stocks Traded, and Ticker/Data Frequency

Citation	Stock Exchange/Stocks Traded	Frequency
[1] 2021	NASDAQ; Shanghai, Shenzhen, and the Hong Kong Stock Exchanges; A plethora of stocks from these exchanges	Daily
[122] 2022	NASDAQ; Microsoft, Amazon, and Tesla	Daily
[125] 2019	NASDAQ; GOOGL and GE	Daily
[121] 2019	does not specify exact stocks or exchanges	Daily
[115] 2021	NASDAQ; AAPL, GOOG, MSFT, AMZN, GE, MCO, CVX, AND PFE	Minutes
[129] 2021	NASDAQ; technology (AAPL, AMZN, FB, GOOGL, INTC, MSFT, NFLX), financial (JPM, MA, V), consumer discretionary (DIS, HD, JNJ, KO, PFE, PG), energy (XOM), ETF (SPY), industrial (BA) and Communication Services (T)	Hourly
[119] 2022	Bitcoin (cryptocurrencies); S&P 500 (HighTech companies); 10 cryptocurrencies and 9 companies with the most news coverage (BIDU, AAPL, MSFT, INTC, QCOM, AMD, AMZN, NVDA, GOOG)	Half-hourly and Daily
[130] year	Does not specify exact stocks or exchanges	Daily
[131] 2022	Istanbul Stock Exchange; Garanti Bank (GARAN), Akbank (AKBNK), Türkiye İş Bankası (ISCTR)	Daily
[132] 2021	Dow Jones Industrial Average (DJIA); 30 Dow Jones companies	Daily
[126] 2022	US stock market; one portfolio of tech (AAPL, AMD, GOOGL) and another of major US stocks (GOOGL, NVDA, TSLA)	Daily
[127] 2022	AAPL, MSFT, QCOM, IBM, RTX, PG, GS, NKE, DIS, and AXP	Daily
[120] 2023	Indian stock market; Reliance Industries Limited and Tata Motors Limited	Daily
[124] 2024	Cryptocurrencies USDT pairs; 14 cryptocurrency USDT pairs	Minute-level, resampled hourly
[133] 2023	Gold market; Gold prices	Daily
[116] 2022	Various stock exchanges; Selected stocks are in paper's Table 3.1	Daily
[117] 2023	Tesla stock prices	Daily
[118] 2022	Apple stock prices	Daily
[123] 2021	B3 (Brazil Bolsa Balcão); BOVA11 ETF, representing the Brazilian stock market	Daily
[128] 2023	Refer to Table 3	Unclear
[134] 2024	NIFTY 50 Index (India), near-month futures contracts	Minute-level
[135] 2024	Shanghai and Shenzhen Stock Exchanges (40 stocks), plus 30 mixed Chinese and US stocks	Daily

5.5. Training Setups, Evaluation Metrics, and Software

The reviewed RL-NLP stock trading papers reveal common patterns in model training and evaluation, alongside notable variations in hyperparameter tuning, data splits, and software choices. Below, we summarize these aspects in detail.

Training Setups and Data Splits

The studies reviewed describe a range of training setups to develop and evaluate RL models for stock trading. Hyperparameter configurations were determined through systematic techniques such as grid search, with adjustments made for model complexity, input sequence lengths, and optimization parameters. Common practices included using weight initialization methods and learning rate schedulers that adjusted dynamically during training. Data preparation involved temporal splits, with training sets spanning several years of historical data and validation and test sets covering more recent periods. Some studies used rolling window approaches to divide data into overlapping training and testing periods, reflecting evolving market conditions and enabling periodic retraining. Models incorporating sentiment analysis or some sort of prediction input stage usually followed a two-stage training process. For example, sentiment classifiers were trained separately to predict sentiment labels from financial text, and their outputs were included in the state representations for the RL models. This separation allowed for independent optimization of sentiment models, price prediction models, and RL agents. Exploration and overfitting prevention techniques included random parameter initialization, introducing noise in action policies, and employing alternative reward structures. Episodic simulations were commonly used, where agents interacted with the environment over extended sequences to capture long-term trading strategies. Overall, these setups utilized combinations of numerical and textual data to model market behavior and trading decisions under dynamic conditions.

Evaluation Metrics

The studies used a variety of financial and statistical metrics to evaluate the performance of RL models. The Sharpe Ratio (SR) was commonly used to assess risk-adjusted returns, with some studies employing the Sortino Ratio (StR) to account specifically for downside risk. Cumulative Return (CR) and Maximum Drawdown (MDD) were frequently reported to measure overall profitability and the largest observed loss. Additional metrics included Annualized Return (AR), Final Portfolio Value (FPV), and Portfolio Value (PV), which tracked changes in asset value over time. Some studies calculated daily returns, accumulated wealth rates, and measures of volatility to capture episodic and overall performance. Risk-related metrics, such as the Calmar Ratio and Mean Absolute Drawdown (MAD), were also reported. These metrics provided insights into the trade-offs between return and risk, while profit consistency was assessed using measures like Average Profit Return (APR). The evaluation metrics focused on capturing both profitability and risk-related aspects of trading strategies, providing multiple dimensions of model performance.

Software and Frameworks

Most studies employed Python as the primary programming language, leveraging deep learning frameworks like PyTorch or TensorFlow. Reinforcement learning libraries such as Stable-Baselines3, Keras-RL, and OpenAI Gym were commonly used for implementing RL algorithms and creating RL environments. GPU acceleration was widely reported, with hardware ranging from Google Colab to local NVIDIA GPUs.

Conclusion

The reviewed papers highlight several common practices and trends across training setups, evaluation metrics, and software usage. Periodic retraining using rolling windows emerged as a popular approach to incorporate new market data, while reward engineering techniques tailored models to incentivize stable, long-term profitability. Despite the stochastic nature of RL experiments, relatively few studies emphasized multiple random seed trials to ensure robust, reproducible results. These findings provide a structured view of how RL models for stock trading are trained and evaluated, highlighting the methodologies and tools employed across the reviewed literature.

6. Challenges, Limitations, and Future Directions

RL and NLP applications in stock trading present exciting opportunities but face several significant challenges and limitations. These obstacles span data quality, algorithmic complexity, market volatility, and practical

implementation. By synthesizing insights from reviewed papers and personal observations, this section outlines key themes and proposes areas for future research.

Data Quality and Preprocessing Challenges

A recurring theme in the reviewed literature is the difficulty of working with financial data that is noisy, imbalanced, and temporally inconsistent. Many papers highlight the uneven distribution of financial news across companies—larger firms like Apple often have disproportionately more news coverage than smaller entities like Baidu. Temporal gaps in data, such as missing news during weekends or holidays, further complicate modeling efforts. This issue is particularly acute when aligning data streams, such as combining minute-level stock prices with textual data like news articles or tweets, which introduces noise and requires extensive preprocessing.

Additionally, sentiment analysis models may often struggle with accuracy, leading to unreliable sentiment scores that can mislead RL agents. Many papers address this by proposing advanced preprocessing techniques but still potentially could fall short of fully capturing market nuances. Future research could benefit from exploring embeddings that bypass explicit sentiment scoring, allowing RL models to infer market sentiment directly from textual features.

Algorithmic Complexity and Model Design

Many RL-based trading studies restrict themselves to a single stock or a narrow market window, potentially benefiting from a supportive market phase but limiting real-world applicability. Additionally, these works often omit complexities such as transaction costs, liquidity constraints, and multi-asset portfolios—factors that significantly affect profitability in actual markets. While the integration of NLP insights (e.g., from news, sentiment, or social media) holds promise for richer decision-making, a lack of transparency about how features are extracted and fused into the RL pipeline complicates reproducibility and casts doubt on how components (CNNs, BERT embeddings, etc.) are tuned or combined. Expanding the state space to handle multiple assets, incorporating more granular price data (hourly or intraday), and explicitly modeling real-world constraints (transaction costs, liquidity, and slippage) can improve both the robustness and the realism of RL+NLP strategies. Thorough documentation of architecture design and data flows, alongside more rigorous treatment of environment complexities, remains critical for advancing state-of-the-art research in this domain.

Evaluation and Reproducibility

Evaluation methodologies in the reviewed papers often lack depth and standardization. Many rely solely on traditional metrics like Sharpe ratios or cumulative returns, which may not adequately capture the trade-offs between profitability, risk, and adaptability. Furthermore, several studies do not report key experimental details, such as random seed variations or hyperparameter tuning, raising concerns about reproducibility. Future work should emphasize robust evaluation protocols, including comprehensive benchmarking against baseline models, multi-seed experiments, and comparisons to traditional investment strategies.

Additional Observations and Future Directions

Building on these challenges, several opportunities for future research emerge:

- **Enhanced Data Representation:** Move beyond the simplicity of sentiment scores to raw embeddings, allowing RL agents to extract meaningful patterns directly. Incorporate macroeconomic indicators, technical signals, and alternative data sources like earnings calls or analyst reports for richer state representations.
- **Model Diversity and Customization:** Experiment with multiple RL architectures, as opposed to just one, to address limitations in existing methods. Customize policies with CNNs, LSTMs, or attention mechanisms could improve performance.
- **Dynamic Evaluation Metrics:** Develop evaluation metrics that balance risk and return based on specific trading styles, such as high-risk strategies or conservative portfolio management. Explore reward functions that incentivize maximizing dollar returns rather than avoiding losses.
- **Comprehensive Benchmarking:** Create a standardized dataset and benchmark for financial RL research, enabling more rigorous comparisons across models and methodologies.
- **Real-World Adaptability:** Extend the scope of experiments to multi-stock portfolios, real-time trading environments, and longer time horizons. Incorporate practical considerations like transaction costs, regulatory constraints, and portfolio rebalancing strategies.

7. Conclusion

The integration of RL and NLP in stock trading offers promising avenues for navigating the dynamic complexities of financial markets. Although the field has demonstrated significant potential, it remains at an early stage and faces critical challenges. Upcoming research should prioritize advanced NLP techniques, such as transformer-based models (e.g., BERT and GPT), for more accurate and context-aware feature extraction—moving beyond outdated NLP approaches.

Many existing studies could broaden their scope by experimenting with multiple RL architectures, adopting robust evaluation metrics, and ensuring reproducibility via multiple random seeds. Additionally, incorporating richer data sources (e.g., macroeconomic indicators, high-frequency price data) and more comprehensive datasets would enhance both realism and robustness in trading simulations. In particular, shifting from sentiment-based pipelines to text embeddings and attention mechanisms may offer a more nuanced understanding of financial language, reducing the risk of inaccuracies tied to sentiment scoring.

The lack of benchmark datasets stands out as a pressing obstacle, impeding standardized comparisons and reproducibility across studies. Future work must also explore evaluation frameworks aligned with real-world trading contexts, emphasizing risk-aware metrics, longer time horizons, and diverse portfolios. Developing realistic trading environments that integrate multi-modal data, economic trends, and conventional financial features can further advance this field.

By addressing these challenges—harnessing state-of-the-art NLP, creating benchmark datasets, and implementing rigorous experimental protocols—researchers can develop more robust and effective models for financial decision-making. Ultimately, these improvements have the potential to reshape trading practices and catalyze broader applications in complex, data-rich domains.

Acknowledgments

I extend my gratitude to my Ph.D. advisor, Bridget McInnes, for her guidance and support throughout this process, and to Daniela Scidá and Leslie Stratton for their helpful discussions and comments on this survey.

References

- [1] R. Sawhney, A. Wadhwa, S. Agarwal, R. Shah, Quantitative day trading from natural language using reinforcement learning, in: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021*, pp. 4018–4030.
- [2] B. Hambly, R. Xu, H. Yang, Recent advances in reinforcement learning in finance, *Mathematical Finance* 33 (2023) 437–503.
- [3] B. Huang, Y. Huan, L. D. Xu, L. Zheng, Z. Zou, Automated trading systems statistical and machine learning methods and hardware implementation: a survey, *Enterprise Information Systems* 13 (2019) 132–144.
- [4] L. Kanashiro Felizardo, F. Caio Lima Paiva, A. H. Reali Costa, E. Del-Moral-Hernandez, Reinforcement learning applied to trading systems: A survey, *arXiv e-prints* (2022) arXiv-2212.
- [5] R. Sawhney, A. Wadhwa, S. Agarwal, R. Shah, Fast: Financial news and tweet based time aware network for stock trading, in: *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume, 2021*, pp. 2164–2175.
- [6] N. E. Huang, M.-L. Wu, W. Qu, S. R. Long, S. S. Shen, Applications of hilbert–huang transform to non-stationary financial time series analysis, *Applied stochastic models in business and industry* 19 (2003) 245–268.
- [7] H. Xu, L. Chai, Z. Luo, S. Li, Stock movement prediction via gated recurrent unit network based on reinforcement learning with incorporated attention mechanisms, *Neurocomputing* 467 (2022) 214–228.
- [8] W. Dabney, G. Ostrovski, A. Barreto, Temporally-extended $\{\epsilon\}$ -greedy exploration, *arXiv preprint arXiv:2006.01782* (2020).
- [9] A. Millea, Deep reinforcement learning for trading—a critical survey, *Data* 6 (2021) 119.
- [10] S. Sun, R. Wang, B. An, Reinforcement learning for quantitative trading, *ACM Transactions on Intelligent Systems and Technology* 14 (2023) 1–29.
- [11] V. Singh, S.-S. Chen, M. Singhanian, B. Nanavati, A. Gupta, et al., How are reinforcement learning and deep learning algorithms used for big data based decision making in financial industries—a review and research agenda, *International Journal of Information Management Data Insights* 2 (2022) 100094.

- [12] W. Jiang, Applications of deep learning in stock market prediction: recent progress, *Expert Systems with Applications* 184 (2021) 115537.
- [13] P. C. Soon, T.-P. Tan, H. Y. Chan, K. H. Gan, A review on sentiment analysis in reinforcement learning model for stock market analysis, *International Journal of Asian Language Processing* 32 (2022) 2330001.
- [14] S. K. Sahu, A. Mokhadde, N. D. Bokde, An overview of machine learning, deep learning, and reinforcement learning-based techniques in quantitative finance: recent progress and challenges, *Applied Sciences* 13 (2023) 1956.
- [15] T. G. Fischer, Reinforcement learning in financial markets-a survey, Technical Report, FAU discussion papers in economics, 2018.
- [16] R. S. Sutton, A. G. Barto, Reinforcement learning: An introduction, MIT press, 2018.
- [17] L. Weng, An overview of reinforcement learning: Theory and applications, Blog post, 2018. URL: <https://lilianweng.github.io/posts/2018-02-19-rl-overview/>.
- [18] J. Moody, L. Wu, Y. Liao, M. Saffell, Performance functions and reinforcement learning for trading systems and portfolios, *Journal of forecasting* 17 (1998) 441–470.
- [19] J. E. Moody, M. Saffell, Y. Liao, L. Wu, Reinforcement learning for trading systems and portfolios., in: *KDD*, 1998, pp. 279–283.
- [20] J. Sadighian, Extending deep reinforcement learning frameworks in cryptocurrency market making, *arXiv preprint arXiv:2004.06985* (2020).
- [21] N.-F. Chen, R. Roll, S. A. Ross, Economic forces and the stock market, *Journal of business* (1986) 383–403.
- [22] S. R. Parvin, N. Panakaje, Factors influencing stock market participation: A review, *International Journal of Case Studies in Business, IT and Education (IJCSBE)* 6 (2022) 831–861.
- [23] A. Mahida, Reinforcement learning for real-world applications-a comprehensive review (2019).
- [24] C. J. Watkins, P. Dayan, Q-learning, *Machine learning* 8 (1992) 279–292.
- [25] H. R. Maei, C. Szepesvári, S. Bhatnagar, R. S. Sutton, Toward off-policy learning control with function approximation., in: *ICML*, volume 10, 2010, pp. 719–726.
- [26] S. Bhatnagar, R. S. Sutton, M. Ghavamzadeh, M. Lee, Naturalgradient actor-critic algorithms, *Automatica* (2007).
- [27] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, *arXiv preprint arXiv:1506.02438* (2015).
- [28] J. Schulman, S. Levine, P. Abbeel, M. Jordan, P. Moritz, Trust region policy optimization, in: *International conference on machine learning*, PMLR, 2015, pp. 1889–1897.
- [29] J. Achiam, Advanced policy gradient methods, Berkeley, RAIL Lab, 2017. URL: http://rail.eecs.berkeley.edu/deeprlcourse-fa17/f17docs/lecture_13_advanced_pg.pdf.
- [30] OpenAI, Trpo - spinning up documentation, Blog post, 2019. URL: <https://spinningup.openai.com/en/latest/algorithms/trpo.html#id9>.
- [31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347* (2017).
- [32] OpenAI, Ppo - spinning up documentation, Blog post, 2019. URL: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>.
- [33] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: *International conference on machine learning*, PMLR, 2016, pp. 1928–1937.
- [34] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, *nature* 518 (2015) 529–533.
- [35] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, *arXiv preprint arXiv:1312.5602* (2013).
- [36] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [37] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, N. Freitas, Dueling network architectures for deep reinforcement learning, in: *International conference on machine learning*, PMLR, 2016, pp. 1995–2003.
- [38] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, D. Silver, Rainbow: Combining improvements in deep reinforcement learning, in: *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [39] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, *arXiv preprint arXiv:1511.05952* (2015).
- [40] J. Weng, H. Chen, D. Yan, K. You, A. Duburcq, M. Zhang, Y. Su, H. Su, J. Zhu, Tianshou: A highly

- modularized deep reinforcement learning library, *Journal of Machine Learning Research* 23 (2022) 1–6. URL: <http://jmlr.org/papers/v23/21-1127.html>.
- [41] M. G. Bellemare, W. Dabney, R. Munos, A distributional perspective on reinforcement learning, in: *International conference on machine learning*, PMLR, 2017, pp. 449–458.
 - [42] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, *arXiv preprint arXiv:1509.02971* (2015).
 - [43] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, M. Riedmiller, Deterministic policy gradient algorithms, in: *International conference on machine learning*, Pmlr, 2014, pp. 387–395.
 - [44] OpenAI, Trpo - spinning up documentation, Blog post, 2018. URL: <https://spinningup.openai.com/en/latest/algorithms/ddpg.html>.
 - [45] S. Fujimoto, H. Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: *International conference on machine learning*, PMLR, 2018, pp. 1587–1596.
 - [46] OpenAI, Td3 - spinning up documentation, Blog post, 2018. URL: <https://spinningup.openai.com/en/latest/algorithms/td3.html>.
 - [47] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *International conference on machine learning*, PMLR, 2018, pp. 1861–1870.
 - [48] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al., Soft actor-critic algorithms and applications, *arXiv preprint arXiv:1812.05905* (2018).
 - [49] R. U. Community, Deep reinforcement learning practical tips, 2018. URL: https://www.reddit.com/r/reinforcementlearning/comments/7s8px9/deep_reinforcement_learning_practical_tips/, accessed: 2024-08-31.
 - [50] A. Fish, Reproducing deep rl, 2018. URL: <http://amid.fish/reproducing-deep-rl>, accessed: 2024-08-31.
 - [51] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
 - [52] P. Abbeel, J. Schulman, S. Levine, Deep rl bootcamp: Nuts and bolts of deep rl experimentation, 2017. URL: <https://www.youtube.com/watch?v=8EcdaCk9KaQ>, slides available at <https://rll.berkeley.edu/deeprlcourse/docs/nuts-and-bolts.pdf>.
 - [53] A. Irpan, Deep reinforcement learning doesn't work yet, 2018. URL: <https://www.alexirpan.com/2018/02/14/rl-hard.html>, accessed: 2024-08-31.
 - [54] S. Ivanov, 37 reasons why your neural network is not working, 2018. URL: <https://blog.slavv.com/37-reasons-why-your-neural-network-is-not-working-4020854bd607>, accessed: 2024-08-31.
 - [55] O. Flowers, How many random seeds should i use? statistical power analysis in deep reinforcement learning experiments, 2020. URL: <https://openlab-flowers.inria.fr/t/how-many-random-seeds-should-i-use-statistical-power-analysis-in-deep-reinforcement-learning-experiments/457>, accessed: 2024-08-31.
 - [56] A. Hill, Others, Problem in evaluation methodology in ddpg, 2018. URL: <https://github.com/hill-a/stable-baselines/issues/199>, accessed: 2024-08-31.
 - [57] D. Jurafsky, J. H. Martin, *Speech and Language Processing*, Prentice Hall, 2000.
 - [58] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, *arXiv preprint arXiv:2303.18223* (2023).
 - [59] T. Winograd, *Understanding Natural Language*, Academic Press, 1972.
 - [60] C. D. Manning, H. Schütze, *Foundations of Statistical Natural Language Processing*, MIT Press, 1999.
 - [61] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, in: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2013.
 - [62] J. Pennington, R. Socher, C. D. Manning, Glove: Global vectors for word representation, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.
 - [63] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Computation* 9 (1997) 1735–1780.
 - [64] A. Vaswani, Attention is all you need, *Advances in Neural Information Processing Systems* (2017).
 - [65] J. Devlin, Bert: Pre-training of deep bidirectional transformers for language understanding, *arXiv preprint arXiv:1810.04805* (2018).
 - [66] W. A. Qader, M. M. Ameen, B. I. Ahmed, An overview of bag of words; importance, implementation, applications, and challenges, in: *2019 international engineering conference (IEC)*, IEEE, 2019, pp. 200–204.
 - [67] M. Grootendorst, Bertopic: Neural topic modeling with a class-based tf-idf procedure, *arXiv preprint arXiv:2203.05794* (2022).
 - [68] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, N. Carlini, Deduplicating training

- data makes language models better, arXiv preprint arXiv:2107.06499 (2021).
- [69] A. L. Awad, S. M. Elkaftas, M. W. Fakh, Stock market prediction using deep reinforcement learning, *Applied System Innovation* 6 (2023) 106.
 - [70] K. A. Hambarde, H. Proenca, Information retrieval: recent advances and beyond, *IEEE Access* (2023).
 - [71] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, *Transactions of the association for computational linguistics* 5 (2017) 135–146.
 - [72] Y. Ye, H. Pei, B. Wang, P.-Y. Chen, Y. Zhu, J. Xiao, B. Li, Reinforcement-learning based portfolio management with augmented asset movement prediction states, in: *Proceedings of the AAAI conference on artificial intelligence*, volume 34, 2020, pp. 1112–1119.
 - [73] D. Bahdanau, Neural machine translation by jointly learning to align and translate, arXiv preprint arXiv:1409.0473 (2014).
 - [74] M.-T. Luong, Effective approaches to attention-based neural machine translation, arXiv preprint arXiv:1508.04025 (2015).
 - [75] L. Weng, Attention? attention!, Blog post, 2018. URL: <https://lilianweng.github.io/posts/2018-06-24-attention/>.
 - [76] J. Lei Ba, J. R. Kiros, G. E. Hinton, Layer normalization, *ArXiv e-prints* (2016) arXiv–1607.
 - [77] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
 - [78] Y. Wu, Google’s neural machine translation system: Bridging the gap between human and machine translation, arXiv preprint arXiv:1609.08144 (2016).
 - [79] V. Sanh, Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, arXiv preprint arXiv:1910.01108 (2019).
 - [80] C. Buciluă, R. Caruana, A. Niculescu-Mizil, Model compression, in: *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 535–541.
 - [81] G. Hinton, Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531 (2015).
 - [82] Z. Lan, Albert: A lite bert for self-supervised learning of language representations, arXiv preprint arXiv:1909.11942 (2019).
 - [83] I. Beltagy, M. E. Peters, A. Cohan, Longformer: The long-document transformer, arXiv preprint arXiv:2004.05150 (2020).
 - [84] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, et al., Big bird: Transformers for longer sequences, *Advances in neural information processing systems* 33 (2020) 17283–17297.
 - [85] Y. Liu, Roberta: A robustly optimized bert pretraining approach, arXiv preprint arXiv:1907.11692 364 (2019).
 - [86] P. He, X. Liu, J. Gao, W. Chen, Deberta: Decoding-enhanced bert with disentangled attention, in: *International Conference on Learning Representations*, 2021. URL: <https://openreview.net/forum?id=XPZlaotutsD>.
 - [87] P. He, J. Gao, W. Chen, Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing, arXiv preprint arXiv:2111.09543 (2021).
 - [88] K. Clark, Electra: Pre-training text encoders as discriminators rather than generators, arXiv preprint arXiv:2003.10555 (2020).
 - [89] T. B. Brown, Language models are few-shot learners, arXiv preprint arXiv:2005.14165 (2020).
 - [90] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, arXiv preprint arXiv:2303.08774 (2023).
 - [91] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).
 - [92] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al., Palm: Scaling language modeling with pathways, *Journal of Machine Learning Research* 24 (2023) 1–113.
 - [93] T. Le Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, et al., Bloom: A 176b-parameter open-access multilingual language model (2023).
 - [94] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, *Advances in neural information processing systems* 35 (2022) 27730–27744.
 - [95] OpenAI, GPT-4V(ision) System Card, 2023. URL: https://cdn.openai.com/papers/GPTV_System_Card.pdf.

- [96] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021).
- [97] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, D. Zhou, Mobilebert: a compact task-agnostic bert for resource-limited devices, arXiv preprint arXiv:2004.02984 (2020).
- [98] A. H. Huang, H. Wang, Y. Yang, Finbert: A large language model for extracting information from financial text, *Contemporary Accounting Research* 40 (2023) 806–841.
- [99] I. Beltagy, K. Lo, A. Cohan, Scibert: A pretrained language model for scientific text, arXiv preprint arXiv:1903.10676 (2019).
- [100] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [101] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv preprint arXiv:2312.10997 (2023).
- [102] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, J.-R. Wen, Large language models for information retrieval: A survey, arXiv preprint arXiv:2308.07107 (2023).
- [103] A. Petukhova, J. P. Matos-Carvalho, N. Fachada, Text clustering with llm embeddings, arXiv preprint arXiv:2403.15112 (2024).
- [104] V. Devarajan, R. Subramanian, Analyzing semantic similarity amongst textual documents to suggest near duplicates, *Indonesian Journal of Electrical Engineering and Computer Science* 25 (2022) 1703–1711.
- [105] H. Guo, S. Yuan, X. Wu, Logbert: Log anomaly detection via bert, in: *2021 international joint conference on neural networks (IJCNN)*, IEEE, 2021, pp. 1–8.
- [106] A. Akbik, T. Bergmann, D. Blythe, K. Rasul, S. Schweter, R. Vollgraf, Flair: An easy-to-use framework for state-of-the-art nlp, in: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics (demonstrations)*, 2019, pp. 54–59.
- [107] I. Garrido-Muñoz, A. Montejó-Ráez, F. Martínez-Santiago, L. A. Ureña-López, A survey on bias in deep nlp, *Applied Sciences* 11 (2021) 3184.
- [108] B. Hutchinson, V. Prabhakaran, E. Denton, K. Webster, Y. Zhong, S. Denuyl, Social biases in nlp models as barriers for persons with disabilities, arXiv preprint arXiv:2005.00813 (2020).
- [109] P. Kumar, Adversarial attacks and defenses for large language models (llms): methods, frameworks & challenges, *International Journal of Multimedia Information Retrieval* 13 (2024) 26.
- [110] A. K. Veldanda, F. Grob, S. Thakur, H. Pearce, B. Tan, R. Karri, S. Garg, Investigating hiring bias in large language models, in: *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*, 2023.
- [111] W. A. Khattak, F. Rabbi, Ethical considerations and challenges in the deployment of natural language processing systems in healthcare, *International Journal of Applied Health Care Analytics* 8 (2023) 17–36.
- [112] L. Wang, X. Zhang, H. Su, J. Zhu, A comprehensive survey of continual learning: theory, method and application, *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024).
- [113] H. Zhao, H. Chen, F. Yang, N. Liu, H. Deng, H. Cai, S. Wang, D. Yin, M. Du, Explainability for large language models: A survey, *ACM Transactions on Intelligent Systems and Technology* 15 (2024) 1–38.
- [114] C. Hutto, E. Gilbert, Vader: A parsimonious rule-based model for sentiment analysis of social media text, in: *Proceedings of the international AAAI conference on web and social media*, volume 8, 2014, pp. 216–225.
- [115] Y.-F. Chen, S.-H. Huang, Sentiment-influenced trading system based on multimodal deep reinforcement learning, *Applied Soft Computing* 112 (2021) 107788.
- [116] D. Benk, Stock trading using a deep reinforcement learning and text analysis (2022).
- [117] X. Li, H. Ming, Stock market prediction using reinforcement learning with sentiment analysis, *International Journal on Cybernetics & Informatics (IJCI)* 12 (2023) 1.
- [118] K. Ameen Suhail, S. Sankar, A. S. Kumar, T. Nestor, N. F. Soliman, A. D. Algarni, W. El-Shafai, A. El-Samie, E. Fathi, Stockmarket trading based onmarket sentiments and reinforcement learning., *Computers, Materials & Continua* 70 (2022).
- [119] Y. Ye, H. Pei, B. Wang, P.-Y. Chen, Y. Zhu, J. Xiao, B. Li, Reinforcement-learning based portfolio management with augmented asset movement prediction states, in: *Proceedings of the AAAI conference on artificial intelligence*, volume 34, 2020, pp. 1112–1119.
- [120] A. Bhakar, P. S. Deori, Y. V. Gautam, K. Srinivasa, Maximizing returns with reinforcement learning: A paradigm shift in stock market portfolio management, in: *TENCON 2023-2023 IEEE Region 10 Conference (TENCON)*, IEEE, 2023, pp. 393–398.

- [121] C. Zhang, Sentiment analysis and deep reinforcement learning for algorithmic trading, Semantic Scholar (2019).
- [122] A. Nan, A. Perumal, O. R. Zaiane, Sentiment and knowledge based algorithmic trading with deep reinforcement learning, in: International Conference on Database and Expert Systems Applications, Springer, 2022, pp. 167–180.
- [123] R. F. d. Silva, Automated stock trading system using deep reinforcement learning and price and sentiment prediction modules., Ph.D. thesis, Universidade de São Paulo, 2021.
- [124] L. Avramelou, P. Nousi, N. Passalis, A. Tefas, Deep reinforcement learning for financial trading using multi-modal features, Expert Systems with Applications 238 (2024) 121849.
- [125] A. R. Azhikodan, A. G. Bhat, M. V. Jadhav, Stock trading bot using deep reinforcement learning, in: Innovations in Computer Science and Engineering: Proceedings of the Fifth ICICSE 2017, Springer, 2019, pp. 41–49.
- [126] Z. Huang, F. Tanaka, Mspm: A modularized and scalable multi-agent reinforcement learning-based system for financial portfolio management, Plos one 17 (2022) e0263689.
- [127] T. Kabbani, E. Duman, Deep reinforcement learning approach for trading automation in the stock market, IEEE Access 10 (2022) 93564–93574.
- [128] M. Yang, M. Zhu, Q. Liang, X. Zheng, M. Wang, Spotlight news driven quantitative trading based on trajectory optimization., in: IJCAI, 2023, pp. 4930–4939.
- [129] F. C. Lima Paiva, L. K. Felizardo, R. A. d. C. Bianchi, A. H. R. Costa, Intelligent trading systems: A sentiment-aware reinforcement learning approach, in: Proceedings of the second ACM international conference on AI in finance, 2021, pp. 1–9.
- [130] S. Feuerriegel, H. Prendinger, News-based trading strategies, Decision Support Systems 90 (2016) 65–74.
- [131] A. B. Altuner, Z. H. Kilimci, A novel deep reinforcement learning based stock price prediction using knowledge graph and community aware sentiments, Turkish Journal of Electrical Engineering and Computer Sciences 30 (2022) 1506–1524.
- [132] P. Koratamaddi, K. Wadhwani, M. Gupta, S. G. Sanjeevi, Market sentiment-aware deep reinforcement learning approach for stock portfolio allocation, Engineering Science and Technology, an International Journal 24 (2021) 848–859.
- [133] A. L. Awad, S. M. Elkaffas, M. W. Fakhr, Stock market prediction using deep reinforcement learning, Applied System Innovation 6 (2023) 106.
- [134] S. Gangopadhyay, P. Majumder, Examining the effect of news context on algorithmic trading, in: Proceedings of the Eighth Financial Technology and Natural Language Processing and the 1st Agent AI for Scenario Planning, 2024, pp. 33–41.
- [135] W. Liu, Y. Gu, Y. Ge, Multi-factor stock trading strategy based on dqn with multi-bigru and multi-head probsparse self-attention, Applied Intelligence (2024) 1–24.

A. Appendix

A.1. List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
NLP	Natural Language Processing
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
MDRNN	Multimodal Deep Recurrent Neural Network
CNN	Convolutional Neural Network
RCNN	Recurrent Convolutional Neural Network

Continued on next page

Acronym	Definition
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
ANN	Artificial Neural Network
KNN	K-Nearest Neighbors
SVM	Support Vector Machine
PPO	Proximal Policy Optimization
DDPG	Deep Deterministic Policy Gradient
TD3	Twin Delayed Deep Deterministic Policy Gradient
DQN	Deep Q-Network
DDQN	Double Deep Q-Network
A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
SARSA	State-Action-Reward-State-Action Algorithm
MARL	Multi-Agent Reinforcement Learning
SAC	Soft Actor-Critic
TRPO	Trust Region Policy Optimization
MDP	Markov Decision Process
POMDP	Partially Observable Markov Decision Process
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Approach
FinBERT	Financial BERT
ALBERT	A Lite BERT
DistilBERT	Distilled BERT
DeBERTa	Decoding-enhanced BERT with Disentangled Attention
mDeBERTa	Multilingual DeBERTa
SciBERT	Scientific BERT
ELECTRA	Efficiently Learning an Encoder that Classifies Token Replacements Accurately
GloVe	Global Vectors for Word Representation
CBOW	Continuous Bag of Words
BoW	Bag-of-Words
TF-IDF	Term Frequency-Inverse Document Frequency
NER	Named Entity Recognition
NLTK	Natural Language Toolkit
VADER	Valence Aware Dictionary and Sentiment Reasoner
HFT	High-Frequency Trading
AT	Algorithmic Trading
PM	Portfolio Management
OE	Order Execution
QT	Quantitative Trading
NQT	News-Driven Quantitative Trading

Continued on next page

Acronym	Definition
ROI	Return on Investment
Sharpe Ratio	A Measure of Risk-Adjusted Return
StR	Sortino Ratio
CR	Cumulative Return
MDD	Maximum Drawdown
AR	Annualized Return
FPV	Final Portfolio Value
PV	Portfolio Value
RSI	Relative Strength Index
MOM	Momentum
MACD	Moving Average Convergence Divergence
ATR	Average True Range
OHLC	Open, High, Low, Close
OHLCV	Open, High, Low, Close, Volume
MC	Monte Carlo
TD	Temporal-Difference
DP	Dynamic Programming
KL Divergence	Kullback-Leibler Divergence
IS	Importance Sampling
IID	Independent and Identically Distributed
OU	Ornstein-Uhlenbeck
ARIMA	AutoRegressive Integrated Moving Average
SARIMA	Seasonal AutoRegressive Integrated Moving Average
AWS	Amazon Web Services
API	Application Programming Interface
RTD	Replaced Token Detection
GDES	Gradient-Disentangled Embedding Sharing
EAMs	Evolving Agent Modules
RAG	Retrieval-Augmented Generation

A.2. Advantage Function

Below describes why the **advantage function** $A^\pi(s_t, a_t)$ remains unbiased and reduces variance:

1. **Unbiasedness:** Recall that all probability distributions are normalized:

$$\int_x P_\theta(x) = 1. \quad (96)$$

Take the gradient of both sides of the normalization condition:

$$\nabla_\theta \int_x P_\theta(x) = \nabla_\theta 1 = 0. \quad (97)$$

Use the log derivative trick to get:

$$0 = \nabla_\theta \int_x P_\theta(x) \quad (98)$$

$$= \int_x \nabla_\theta P_\theta(x) \quad (99)$$

$$= \int_x P_\theta(x) \nabla_\theta \log P_\theta(x). \quad (100)$$

Therefore, the expectation of the baseline term $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t)b(s_t)]$ is zero because the baseline $b(s_t)$ does not depend on the action a_t :

$$\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t)b(s_t)] = b(s_t)\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t)] \quad (101)$$

$$= b(s_t) \cdot 0 \quad (102)$$

$$= 0. \quad (103)$$

Therefore, introducing $b(s_t)$ does not change the expectation of the gradient estimate, maintaining unbiasedness.

2. Variance Reduction: High variance in gradient estimates can occur when running the agent on a batch of episodes (trajectories from start to finish) and using the empirical expectation of the total reward at the end of each trajectory, leading to wide-ranging and unreliable performance outcomes, which makes learning unstable and slow. By subtracting the baseline $b(s_t)$, we reduce the variability of the estimate. The baseline is typically chosen to be the value function $V(s_t)$, which estimates the expected return from state s_t . By subtracting this baseline, we are effectively focusing on the advantage of taking a specific action over the average action. This subtraction reduces the fluctuations in the estimate, making the learning process more stable and efficient:

$$\text{Var}(\Psi_t - b(s_t)) < \text{Var}(\Psi_t). \quad (104)$$

A.3. Policy Gradient Theorem

The Policy Gradient Theorem is a foundational result in reinforcement learning that provides a way to compute the gradient of the expected reward with respect to the policy parameters. This theorem is crucial for policy optimization methods, as it enables the direct adjustment of policy parameters to maximize the expected return. The theorem's main idea is that we can approximate the gradient ascent direction for optimizing policy parameters by using the expected value of certain calculations involving the policy's actions and their respective action values. This approach helps in efficiently improving the policy to achieve better performance.

The formal derivation of the Policy Gradient Theorem, including the specific mathematical notations and steps, is provided in the following section.

A.3.1. Derivation for Basic Policy Gradient

Let's start by understanding the gradient of the expected value of a function $f(x)$ with respect to parameters θ :

$$\nabla_\theta \mathbb{E}_{p_\theta}[f(x)] \quad (105)$$

We express this expectation as an integral:

$$\nabla_\theta \mathbb{E}_{p_\theta}[f(x)] = \nabla_\theta \int p_\theta(x) f(x) dx \quad (106)$$

Explanation: The expectation $\mathbb{E}_{p_\theta}[f(x)]$ is written as an integral over the probability density function $p_\theta(x)$. Next, we move the gradient inside the integral:

$$= \int f(x) \nabla_\theta p_\theta(x) dx \quad (107)$$

Explanation: The key idea is that the expectation of a function with respect to a parameterized probability distribution can be differentiated by differentiating the probability distribution itself.

We introduce a clever trick by multiplying and dividing by $p_\theta(x)$:

$$= \int f(x) \frac{\nabla_{\theta} p_{\theta}(x)}{p_{\theta}(x)} p_{\theta}(x) dx \quad (108)$$

Explanation: Multiplying and dividing by $p_{\theta}(x)$ sets up the use of the logarithm in the next step. Using the logarithm property:

$$= \int p_{\theta}(x) f(x) \nabla_{\theta} \log p_{\theta}(x) dx \quad (109)$$

Explanation: The fraction $\frac{\nabla_{\theta} p_{\theta}(x)}{p_{\theta}(x)}$ is written as $\nabla_{\theta} \log p_{\theta}(x)$.

Finally, we rewrite it as an expectation:

$$= \mathbb{E}_{p_{\theta}} [f(x) \nabla_{\theta} \log p_{\theta}(x)] \quad (110)$$

Explanation: The gradient of the expected value of $f(x)$ is now expressed as the expectation of $f(x)$ times the gradient of the log-probability of x .

Another important aspect is the gradient of the log probability of a trajectory, which will be needed when substituting x with a trajectory τ . The computation of $\log p_{\theta}(\tau)$ is as follows:

$$\nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} \log \left(\mu(s_0) \prod_{t=0}^T \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t) \right) \quad (111)$$

Explanation: We start with the log probability of a trajectory τ , which is the product of the initial state distribution $\mu(s_0)$, the policy $\pi_{\theta}(a_t | s_t)$, and the state transition probabilities $P(s_{t+1} | s_t, a_t)$.

Next, we use the logarithm property to separate the product into a sum:

$$= \nabla_{\theta} \left(\log \mu(s_0) + \sum_{t=0}^T (\log \pi_{\theta}(a_t | s_t) + \log P(s_{t+1} | s_t, a_t)) \right) \quad (112)$$

Explanation: The log of a product is the sum of the logs, which simplifies the expression by breaking it into separate terms.

Since $\mu(s_0)$ and $P(s_{t+1} | s_t, a_t)$ do not depend on θ , their gradients are zero:

$$= \nabla_{\theta} \sum_{t=0}^T \log \pi_{\theta}(a_t | s_t) \quad (113)$$

Explanation: Only the policy $\pi_{\theta}(a_t | s_t)$ depends on θ , so we focus on these terms.

Therefore, in reinforcement learning, we want to find the gradient of the expected reward with respect to the policy parameters θ . This is given by:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (114)$$

Explanation: The equation shows the gradient of the expected reward $J(\theta)$ with respect to the policy parameters θ as the expectation of the sum over all time steps of the return Ψ_t times the gradient of the log-probability of the action a_t given state s_t .

Where:

- $J(\theta)$ is the expected reward or objective function we aim to maximize.
- $\mathbb{E}$ denotes the expectation over all possible trajectories.
- $\sum_{t=0}^T$ is the sum over all time steps.
- Ψ_t is the return (cumulative reward) from time step t onward.
- $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ is the gradient of the log-probability of taking action a_t given state s_t with respect to the policy parameters θ .

Step-by-Step Explanation

1. Objective Function (Expected Reward):

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \right] \quad (115)$$

Explanation: The objective function $J(\theta)$ represents the expected sum of future rewards (returns) Ψ_t from time step t onward.

2. Gradient of the Objective Function:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E} \left[\sum_{t=0}^T \Psi_t \right] \quad (116)$$

Explanation: We want to find how the expected reward changes as we adjust the policy parameters θ .

3. Using the Log-Derivative Trick:

$$\nabla_{\theta} \mathbb{E} \left[\sum_{t=0}^T \Psi_t \right] = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (117)$$

Explanation: By applying the log-derivative trick, we convert the gradient of the expectation into the expectation of the gradient, involving the log-probability of the policy.

4. Final Gradient Expression:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (118)$$

Explanation: This final equation shows that the gradient of the expected reward with respect to the policy parameters θ can be computed as the expected value of the product of the returns Ψ_t and the gradient of the log-probability of the actions taken under the policy π_{θ} .

A.4. Example of Self-Attention Mechanism with Pretend Numbers

Let's walk through the steps of self-attention using a simple example with pretend numbers. We'll use a very basic sentence: "I like dogs."

Step 1: Create Query, Key, and Value Vectors

For each word in the sentence, three vectors are created: **Query (Q)**, **Key (K)**, and **Value (V)**. Let's assume we have word embeddings of size 3 (just to keep it simple). We'll also assume that the weight matrices W^Q , W^K , and W^V are already trained and multiply the word embeddings to produce these vectors.

Let's pretend we have these word embeddings for simplicity:

- "I": [1, 0, 1]
- "like": [0, 1, 1]
- "dogs": [1, 1, 0]

After multiplying by the weight matrices, we get the query, key, and value vectors:

- "I":

$$q_1 = [0.5, 1.0, 0.3]$$

$$k_1 = [0.7, 0.9, 0.2]$$

$$v_1 = [1.2, 0.8, 0.5]$$

- "like":

$$q_2 = [0.6, 0.4, 0.9]$$

$$k_2 = [0.5, 0.3, 0.7]$$

$$v_2 = [0.9, 1.1, 0.4]$$

- "dogs":

$$q_3 = [0.9, 0.7, 0.6]$$

$$k_3 = [0.4, 0.6, 1.0]$$

$$v_3 = [1.0, 0.7, 0.3]$$

Step 2: Calculating the Attention Scores

Next, we calculate the attention scores by taking the dot product of each word's query vector with every other word's key vector. This gives us scores that represent how much focus each word should have on the others.

For "I", we compute the dot products:

$$\text{Score between "I" and "I"} = q_1 \cdot k_1 = 0.5 \times 0.7 + 1.0 \times 0.9 + 0.3 \times 0.2 = 0.35 + 0.9 + 0.06 = 1.31$$

$$\text{Score between "I" and "like"} = q_1 \cdot k_2 = 0.5 \times 0.5 + 1.0 \times 0.3 + 0.3 \times 0.7 = 0.25 + 0.3 + 0.21 = 0.76$$

$$\text{Score between "I" and "dogs"} = q_1 \cdot k_3 = 0.5 \times 0.4 + 1.0 \times 0.6 + 0.3 \times 1.0 = 0.2 + 0.6 + 0.3 = 1.1$$

So, the attention scores for "I" are [1.31, 0.76, 1.1].

We do the same for "like" and "dogs":

For "like":

$$\text{Score between "like" and "I"} = q_2 \cdot k_1 = 0.6 \times 0.7 + 0.4 \times 0.9 + 0.9 \times 0.2 = 0.42 + 0.36 + 0.18 = 0.96$$

$$\text{Score between "like" and "like"} = q_2 \cdot k_2 = 0.6 \times 0.5 + 0.4 \times 0.3 + 0.9 \times 0.7 = 0.3 + 0.12 + 0.63 = 1.05$$

$$\text{Score between "like" and "dogs"} = q_2 \cdot k_3 = 0.6 \times 0.4 + 0.4 \times 0.6 + 0.9 \times 1.0 = 0.24 + 0.24 + 0.9 = 1.38$$

For "dogs":

$$\text{Score between "dogs" and "I"} = q_3 \cdot k_1 = 0.9 \times 0.7 + 0.7 \times 0.9 + 0.6 \times 0.2 = 0.63 + 0.63 + 0.12 = 1.38$$

$$\text{Score between "dogs" and "like"} = q_3 \cdot k_2 = 0.9 \times 0.5 + 0.7 \times 0.3 + 0.6 \times 0.7 = 0.45 + 0.21 + 0.42 = 1.08$$

$$\text{Score between "dogs" and "dogs"} = q_3 \cdot k_3 = 0.9 \times 0.4 + 0.7 \times 0.6 + 0.6 \times 1.0 = 0.36 + 0.42 + 0.6 = 1.38$$

Step 3: Scaling and SoftMax

We then scale these scores by dividing by the square root of the key vector dimension $d_k = 3$, which is approximately 1.73. This prevents the dot products from becoming too large.

For "I", the scaled scores are:

$$[1.31/1.73, 0.76/1.73, 1.1/1.73] = [0.76, 0.44, 0.64]$$

We apply the SoftMax function to these scaled scores to get probabilities:

$$\text{SoftMax}(0.76, 0.44, 0.64) = [0.38, 0.28, 0.34]$$

We do the same for "like" and "dogs" to get the SoftMax probabilities for those as well.

Step 4: Weighting the Value Vectors

Next, we use the SoftMax probabilities to weight the value vectors. For "I", we multiply each value vector by the corresponding SoftMax score:

$$\text{Weighted value for "I"} = 0.38 \times v_1 = 0.38 \times [1.2, 0.8, 0.5] = [0.456, 0.304, 0.19]$$

$$\text{Weighted value for "like"} = 0.28 \times v_2 = 0.28 \times [0.9, 1.1, 0.4] = [0.252, 0.308, 0.112]$$

$$\text{Weighted value for "dogs"} = 0.34 \times v_3 = 0.34 \times [1.0, 0.7, 0.3] = [0.34, 0.238, 0.102]$$

Step 5: Summing the Weighted Value Vectors

Finally, we sum the weighted value vectors for "I" to get the final representation for "I":

$$[0.456, 0.304, 0.19] + [0.252, 0.308, 0.112] + [0.34, 0.238, 0.102] = [1.048, 0.85, 0.404]$$

This new vector is the context-aware representation of the word "I", incorporating information from "like" and "dogs". This same process is repeated for every word in the sentence, allowing the self-attention mechanism to build rich, context-aware representations for each word based on its relationship with the other words in the sentence. The key difference from a real-world application is the reduced dimensionality and assumed simplicity of the vectors. In an actual implementation, the dimensions of the word embeddings and attention weights would be much larger, and the weight matrices W^Q , W^K , and W^V would be learned parameters from the model's training process.

A.5. Example Calculation for DistilBERT Loss Components

Pretend example calculation for the combined loss function used in DistilBERT, illustrating the Cross-Entropy Distillation Loss (L_{ce}), Masked Language Modeling Loss (L_{mlm}), and Cosine Embedding Loss (L_{cos}). The weights for each component are $\alpha = 0.5$, $\beta = 0.3$, and $\gamma = 0.2$.

Cross-Entropy Distillation Loss (L_{ce})

In knowledge distillation, the teacher model's output probabilities are softened using a temperature parameter T . Given logits $z_A = 2.0$, $z_B = 1.0$, and $z_C = 0.1$ for three classes (A, B, and C), we calculate the probabilities at temperature $T = 2$ as follows:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Calculating each softened probability:

$$\begin{aligned} p_A &= \frac{\exp(2.0/2)}{\exp(2.0/2) + \exp(1.0/2) + \exp(0.1/2)} = \frac{\exp(1.0)}{\exp(1.0) + \exp(0.5) + \exp(0.05)} \approx 0.49, \\ p_B &= \frac{\exp(1.0/2)}{\exp(1.0) + \exp(0.5) + \exp(0.05)} \approx 0.30, \\ p_C &= \frac{\exp(0.1/2)}{\exp(1.0) + \exp(0.5) + \exp(0.05)} \approx 0.19. \end{aligned}$$

Assume the student model's predicted probabilities are $s_A = 0.6$, $s_B = 0.25$, and $s_C = 0.15$. The Cross-Entropy Distillation Loss is calculated as:

$$\begin{aligned} L_{ce} &= - \sum_i p_i \log(s_i) \\ L_{ce} &= -(0.49 \cdot \log(0.6) + 0.30 \cdot \log(0.25) + 0.19 \cdot \log(0.15)) \approx 1.0289. \end{aligned}$$

Masked Language Modeling Loss (L_{mlm})

For the Masked Language Modeling Loss, assume the correct label for a masked token is "finance" (index 2) and the student's predicted probability distribution is $[0.05, 0.1, 0.8, 0.03, 0.02]$. Using cross-entropy for the MLM loss:

$$L_{mlm} = -\log(0.8) \approx 0.223.$$

Cosine Embedding Loss (L_{cos})

Let the hidden state vectors for the teacher and student models be $[0.8, 0.6]$ and $[0.7, 0.5]$, respectively. Cosine similarity is given by:

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

Calculating the dot product:

$$a \cdot b = (0.8)(0.7) + (0.6)(0.5) = 0.86$$

and magnitudes:

$$\|a\| = \sqrt{(0.8)^2 + (0.6)^2} = 1, \quad \|b\| = \sqrt{(0.7)^2 + (0.5)^2} \approx 0.86.$$

Thus, $\cos(\theta) \approx 1$, and the Cosine Embedding Loss is:

$$L_{cos} = 1 - \cos(\theta) = 1 - 1 = 0.$$

Total Loss Calculation

The final loss L is computed as a weighted sum of all three components:

$$L = \alpha \cdot L_{ce} + \beta \cdot L_{mlm} + \gamma \cdot L_{cos}$$

$$L = (0.5 \cdot 1.0289) + (0.3 \cdot 0.223) + (0.2 \cdot 0) = 0.51445 + 0.0669 + 0 = 0.58135.$$

Thus, the total combined loss for this example is approximately $L = 0.58135$.

A.6. Example of Disentangled Attention Mechanism with Pretend Numbers

Let's walk through the steps of DeBERTa's disentangled attention using a simplified example based on the sentence: "Dogs bark at passing cars." We will use $k = 2$ and ensure all calculations are consistent with this value. In DeBERTa, k is set to 512, which makes more intuitive sense to have a "far-away" embedding, this is just a simple example. We assume:

- Initial input embeddings for each word in the sentence: "Dogs bark at passing cars."
- Learnable weight matrices $W_{Q_c}, W_{K_c}, W_{V_c}$ for content embeddings and W_{Q_r}, W_{K_r} for position embeddings.
- A relative distance matrix δ representing the distances between tokens.
- Relative position embeddings P based on those distances.

Initial Embeddings and Matrices

Sentence Tokens and Indices:

- "Dogs" $\rightarrow i = 0$
- "bark" $\rightarrow i = 1$
- "at" $\rightarrow i = 2$
- "passing" $\rightarrow i = 3$
- "cars" $\rightarrow i = 4$

Initial Input Embeddings X :

For simplicity, we'll use an embedding size of 2.

- "Dogs": $X_{\text{Dogs}} = [0.8, 1.0]$
- "bark": $X_{\text{bark}} = [0.6, 0.4]$
- "at": $X_{\text{at}} = [0.9, 0.3]$
- "passing": $X_{\text{passing}} = [0.7, 1.2]$
- "cars": $X_{\text{cars}} = [1.0, 0.5]$

Relative Position Embeddings P :

With $k = 2$, we need embeddings for relative distances from -2 to $+2$, and special embeddings for distances beyond that.

We map relative distances to indices using the formula:

$$\delta(i, j) = \begin{cases} 0 & \text{if } i - j \leq -k \\ 2k & \text{if } i - j \geq k \\ i - j + k & \text{otherwise} \end{cases}$$

So, $\delta(i, j)$ ranges from 0 to $2k = 4$.

Relative position embeddings P :

- P_0 (distance ≤ -2): $[0.5, 0.1]$
- P_1 (distance -1): $[0.08, 0.03]$
- P_2 (distance 0): $[0.1, -0.05]$
- P_3 (distance $+1$): $[0.84, -1.45]$
- P_4 (distance $\geq +2$): $[1.05, -0.73]$

Learnable Weight Matrices:

- Content weights:

$$W_{Q_c} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.7 \end{bmatrix}, \quad W_{K_c} = \begin{bmatrix} 0.1 & 0.8 \\ -0.4 & 0.6 \end{bmatrix}, \quad W_{V_c} = \begin{bmatrix} 0.3 & 0.2 \\ -0.1 & 0.5 \end{bmatrix}$$

- Position weights:

$$W_{Q_r} = \begin{bmatrix} -0.2 & 0.4 \\ 0.6 & -0.3 \end{bmatrix}, \quad W_{K_r} = \begin{bmatrix} 0.4 & -0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

Step 1: Compute Content and Position Vectors

Content Vectors:

For each token, compute content queries Q_c , keys K_c , and values V_c .

Example for "Dogs":

$$Q_{c,\text{Dogs}} = X_{\text{Dogs}} \times W_{Q_c} = [0.8, 1.0] \times \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.7 \end{bmatrix} = [0.6, 0.7]$$

Similarly, compute K_c and V_c for "Dogs" and other tokens.

Position Vectors:

For each relative position index δ , compute position queries Q_r and keys K_r .

Example for $\delta = 1$ (distance -1):

$$\begin{aligned} Q_{r,1} &= P_1 \times W_{Q_r} = [0.08, 0.03] \times \begin{bmatrix} -0.2 & 0.4 \\ 0.6 & -0.3 \end{bmatrix} \\ &= [(0.08 \times -0.2) + (0.03 \times 0.6), (0.08 \times 0.4) + (0.03 \times -0.3)] = [-0.002, 0.025] \end{aligned}$$

Similarly, compute K_r for all δ .

Step 2: Calculate Relative Distance Matrix $\delta(i, j)$

Using the formula and $k = 2$, we compute $\delta(i, j)$ for each pair (i, j) :

- For $i = 0$ ("Dogs"):
 - $j = 0$: $i - j = 0$, so $\delta(0, 0) = 0 + 2 = 2$
 - $j = 1$: $i - j = -1$, so $\delta(0, 1) = -1 + 2 = 1$
 - $j = 2$: $i - j = -2$, $i - j \leq -k$, so $\delta(0, 2) = 0$
 - $j = 3$: $i - j = -3$, $i - j \leq -k$, so $\delta(0, 3) = 0$
 - $j = 4$: $i - j = -4$, $i - j \leq -k$, so $\delta(0, 4) = 0$
- For $i = 1$ ("bark"):
 - $j = 0$: $i - j = 1$, so $\delta(1, 0) = 1 + 2 = 3$
 - $j = 1$: $i - j = 0$, so $\delta(1, 1) = 0 + 2 = 2$
 - $j = 2$: $i - j = -1$, so $\delta(1, 2) = -1 + 2 = 1$
 - $j = 3$: $i - j = -2$, $i - j \leq -k$, so $\delta(1, 3) = 0$
 - $j = 4$: $i - j = -3$, $i - j \leq -k$, so $\delta(1, 4) = 0$

Thus, the relative distance matrix $\delta(i, j)$ is:

$$\delta(i, j) = \begin{bmatrix} 2 & 1 & 0 & 0 & 0 \\ 3 & 2 & 1 & 0 & 0 \\ 4 & 3 & 2 & 1 & 0 \\ 4 & 4 & 3 & 2 & 1 \\ 4 & 4 & 4 & 3 & 2 \end{bmatrix}$$

Step 3: Calculate Attention Scores

For each pair (i, j) , compute:

$$\tilde{A}_{i,j} = Q_{c,i} \cdot K_{c,j}^\top + Q_{c,i} \cdot K_{r,\delta(i,j)}^\top + Q_{r,\delta(j,i)} \cdot K_{c,j}^\top$$

Example calculation for $i = 0$ ("Dogs") attending to $j = 1$ ("bark"):

- **Content-to-Content Term:**

$$\begin{aligned} Q_{c,\text{Dogs}} \cdot K_{c,\text{bark}}^\top &= [0.6, 0.7] \cdot [-0.1, 0.5]^\top \\ &= (0.6)(-0.1) + (0.7)(0.5) = -0.06 + 0.35 = 0.29 \end{aligned}$$

- **Content-to-Position Term:**

$$\begin{aligned} \delta(0, 1) = 1 &\implies K_{r,1}, \quad Q_{c,\text{Dogs}} \cdot K_{r,1}^\top = [0.6, 0.7] \cdot [0.029, 0.01]^\top \\ &= (0.6)(0.029) + (0.7)(0.01) = 0.0174 + 0.007 = 0.0244 \end{aligned}$$

- **Position-to-Content Term:**

$$\begin{aligned} \delta(1, 0) = 3 &\implies Q_{r,3}, \quad Q_{r,3} \cdot K_{c,\text{bark}}^\top = [0.3, -0.4] \cdot [-0.1, 0.5]^\top \\ &= (0.3 \times -0.1) + (-0.4 \times 0.5) = -0.03 + (-0.2) = -0.23 \end{aligned}$$

(Here, $Q_{r,3}$ is computed from P_3 and W_{Q_r} , similar to earlier steps.)

Combine the terms to get the final attention score:

$$\tilde{A}_{0,1} = 0.29 + 0.0244 - 0.23 = 0.0844$$

Similarly, compute $\tilde{A}_{i,j}$ for other pairs.

Summary

By using $k = 2$, we have demonstrated how DeBERTa's disentangled attention mechanism calculates attention scores incorporating relative positions within a specified range, and uses special embeddings for positions beyond that range. This approach allows the model to focus on relevant nearby tokens while efficiently handling distant tokens.